

基于语音声学特征的帕金森病辅助诊断与潜在语音表型分析

摘要：帕金森病常伴随发声稳定性、声带振动和谱结构等方面的改变，语音声学特征为辅助筛查和探索性分型提供了低侵入、可重复采集的数据基础。本文围绕两个含重复录音结构的帕金森病语音数据集，建立受试者级二分类辅助诊断模型、判别性声学特征综合评价模型以及 PD 内部潜在语音表型无监督分型模型。针对同一受试者多次录音可能导致的样本单位错误，本文将受试者作为核心分析单位；在监督诊断模型中采用按受试者分组的交叉验证，并将标准化、模型训练和相关特征处理限制在训练折内部完成。

对问题一和问题三，本文构建并比较 Logistic Regression、SVM-RBF 和 Random Forest 三类二分类辅助诊断模型，并以轻量 XGBoost 作为补充集成模型证据。Dataset 1 主分析采用受试者级指标，Random Forest 的准确率和 AUC 分别为 0.8290 和 0.8511，但其特异度为 0.4385，显示模型更偏向识别 PD 类；SVM-RBF 的 balanced accuracy 为 0.7118，综合表现较稳健。补充 XGBoost 在 Dataset 1 acoustic-only 受试者级结果中 accuracy 为 0.8331、AUC 为 0.8741，但 specificity 为 0.4372，同样提示模型更偏向 PD 识别，因此不改变主推荐口径。Dataset 2 按受试者 ID 分组交叉验证，SVM-RBF 达到 accuracy 0.8375、balanced accuracy 0.8375、AUC 0.8906、specificity 0.8750，作为该数据集的主推荐模型。

对问题二，本文在 Dataset 1 受试者级数据上整合 Mann-Whitney U 检验、Welch t-test、FDR-BH 校正、Cohen's d 、Cliff's delta、L1 Logistic 稳定选择、ExtraTrees 重要性和 held-out permutation importance，得到候选语音生物标志物综合排序；同时采用 XGBoost 特征重要性和 held-out-fold SHAP 平均绝对贡献作为补充解释证据。Top20 特征主要包括 Delta / Delta-delta 动态谱波动特征与 TQWT 多尺度特征；Top50 中 TQWT 特征 31 个、Delta / Delta-delta 特征 16 个、Shimmer、MFCC 和 Nonlinear 特征各 1 个。SHAP 与 XGBoost 重要性进一步支持 Delta / Delta-delta 与 TQWT 特征的判别贡献，但不参与主排序计算，也不进入后续 PD 内部分型。上述结果仅表示判别性声学特征或候选语音生物标志物，不构成临床因果解释。

对问题四和问题五，由于数据集中没有真实运动型/非运动型标签，也没有静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆、睡眠障碍六类临床症状标签，本文仅在 Dataset 1 的 188 名 PD 受试者内部进行无监督潜在语音表型分析。二类潜在语音表型采用多候选报告口径，其中 top20_biomarkers / pca_5 / KMeans 的权重敏感性 top-1 频率最高，为 37.3%，因此作为相对推荐候选方案，而不作为唯一稳健最优结论。六类潜在语音表型采用 top20_biomarkers / pca_5 / AgglomerativeClustering，其权重敏感性 top-1 频率为 87.1%，可作为 $k = 6$ 潜在语音表型聚类的稳健主推荐方案。上述分型结果均不应解释为已经由临床标签确认的真实亚型或症状类别。

本文在建模流程中保持受试者分组、训练折内标准化、主声学模型排除非声学变量和无监督标签不回流输入等约束，以降低样本单位错误和信息泄漏风险。综上，语音声学特征对帕金森病辅助筛查、判别性特征识别和探索性潜在语音表型分析具有一定价值，但本文模型不应解释为临床确诊工具，仍需更大样本和真实临床症状标签验证。

关键词：帕金森病；语音特征；受试者级交叉验证；候选语音生物标志物；潜在语音表型；权重敏感性分析

一 问题重述

帕金森病是一类常见神经退行性疾病，临床表现既包括静止性震颤、运动迟缓、肌强直等运动症状，也常伴随睡眠、认知、情绪和自主神经等非运动症状^[1]。除上述临床表现外，患者还可能出现发声稳定性下降、音强控制异常、音调波动和语音清晰度下降等语音障碍。已有研究表明，发声障碍和非侵入语音测试可用于帕金森病监测和辅助评估^[2, 3]。语音采集成本较低，适合用于辅助筛查和持续随访，因此本题要求基于给定声学数据建立帕金森病特征识别与声音诊断模型。

题目包含五个相互关联的问题。第一问要求利用 Dataset 1 建立健康人与帕金森病患者的二分类辅助诊断模型；第二问要求识别与帕金森病判别相关的关键语音特征；第三问要求利用 Dataset 2 建立声音特征智能诊断模型；第四问希望区分运动型与非运动型帕金森病；第五问希望进一步区分六类临床症状。由于附件数据没有提供第四问所需的真实运动型/非运动型标签，也没有第五问所需的六类临床症状标签，本文不构建真实临床症状监督诊断模型，而是在 PD 受试者内部进行无监督探索，得到基于语音特征的二类和六类潜在表型。

本文工作的边界是基于既有数据和已确定的计算结果形成完整论文表达，不引入新的实验、不改变已有模型输出。所有模型结论均限于语音声学数据的统计关联和预测表现，不能解释为临床因果结论或临床确诊结论。

二 问题分析与模型假设

两个数据集均包含同一受试者的多次录音。若直接把录音记录作为完全独立样本进行行级随机划分，同一受试者的不同录音可能同时进入训练集与测试集，从而高估模型性能。因此，监督建模必须以受试者为分组单位，Dataset 1 的主分析也应采用 subject-level 口径报告结果。

五个问题对应的方法路线如下。问题一使用 Dataset 1 的声学特征建立 PD/Healthy 二分类辅助诊断模型，并以受试者级交叉验证结果作为主结论。问题二基于 Dataset 1 受试者级数据，从统计差异、效应量、稳定选择和模型重要性等多个角度综合识别判别性声学特征。问题三单独使用 Dataset 2 建立诊断模型，并按 subject ID 分组交叉验证。问题四和问题五均缺少真实临床症状标签，因此仅在 Dataset 1 的 PD 受试者内部进行无监督聚类，分别形成二类和六类潜在语音表型。

为保证建模问题可计算并与数据结构一致，作如下假设：第一，同一受试者的多次录音是同一统计个体的重复观测，模型评估时不能跨训练集和测试集泄漏；第二，给定标签仅表示 Healthy/PD 二分类状态，不含运动型/非运动型或六类症状真标签；第三，声学特征的标准化参数、模型参数和特征选择参数只能由训练折估计；第四，聚类标签只表示声学空间中的潜在结构，不预设临床亚型含义。

设第 i 名受试者的声学特征向量为 $\mathbf{x}_i \in \mathbb{R}^p$ ，二分类标签为 $y_i \in \{0, 1\}$ ，其中 $y_i = 1$ 表示 PD。监督诊断模型学习映射

$$f: \mathbf{x}_i \mapsto \hat{p}_i = P(y_i = 1 | \mathbf{x}_i), \quad (1)$$

并由阈值得到类别预测 $\hat{y}_i$ 。主要评价指标定义为

$$\text{Sensitivity} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP}, \quad (2)$$

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}, \quad F_1 = \frac{2TP}{2TP + FP + FN}. \quad (3)$$

特征识别模型输出候选语音生物标志物排序。无监督分型模型在 PD 受试者集合上学习聚类映射 $g(\mathbf{x}_i) = c_i$ ，并使用 silhouette、Davies-Bouldin、簇大小比例和重采样稳定性 ARI 评价声学空间划分质量。由于聚类标签来自声学特征内部结构，后续聚类标签复现模型只能用于复现聚类规则，不能解释为真实临床症状诊断模型。

三 数据预处理与质量控制

3.1 数据结构审计

Dataset 1 原始数据规模为 756 行、755 列，包含 252 名受试者，每名受试者理论上有 3 次录音。数据包含 752 个纯声学特征，元数据列为 `id`、`gender` 和 `class`。该数据集来源于 Sakar 等构建的多类型录音帕金森语音数据集^[4]。审计发现 1 条完全重复记录，去重后建模准备文件为 755 行；受试者级聚合后为 252 行，其中 Healthy 64 名、PD 188 名。Dataset 1 主分析使用 subject-level 口径，避免把重复录音当作完全独立样本。

Dataset 2 原始数据规模为 240 行、48 列，包含 80 名受试者，每人 3 次录音，共 44 个声学特征。元数据列为 `ID`、`Recording`、`Status` 和 `Gender`。该数据集及其重复录音结构在 Naranjo 等关于录音重复对帕金森病检测影响的研究中已有说明^[5]；后续相关工作也讨论了重复录音条件下的变量选择与分类建模^[6]。受试者级类别分布为 Healthy 40 名、PD 40 名，类别均衡。

表 1 两个数据集的基本结构

数据集	原始规模	受试者数	录音结构	声学特征数	受试者级类别分布
Dataset 1	756 行、755 列	252	每人约 3 次录音	752	Healthy 64, PD 188
Dataset 2	240 行、48 列	80	每人 3 次录音	44	Healthy 40, PD 40

3.2 性别变量统一与主模型边界

两个数据集原始性别编码方向不同。为便于描述统计，预处理时统一构造 `sex_male`：Dataset 1 中 `sex_male = gender`，Dataset 2 中 `sex_male = 1 - Gender`。该变量仅用于描述统计和补充敏感性分析，主声学模型不使用 `sex_male`，从而避免把非声学人口学变量混入主要声学诊断结论。

3.3 交叉验证与标准化原则

监督诊断模型使用 `StratifiedGroupKFold`，并以受试者 ID 作为分组变量。标准化、模型训练和特征选择等涉及数据分布学习的步骤均在交叉验证训练折内部完成。Dataset 2 的准备文件只完成元数据清理和性别编码，声学特征没有在交叉验证前做全局 z-score 标准化。

四 模型一：帕金森病二分类辅助诊断模型

4.1 模型输入与方法

模型一输入为两个数据集的 acoustic-only 特征。Dataset 1 使用去重后的建模准备文件，并在报告主结果时按受试者聚合；Dataset 2 使用准备后的 240 条录音，并在交叉验证中按受试者 ID 分组。语音特征结合机器学习用于帕金森病识别已有较多研究基础，Little 等和 Tsanas 等分别从发声障碍测量与非侵入语音测试角度验证了语音特征在帕金森病远程监测中的价值^[2, 3]；Naranjo 等进一步讨论了重复录音条件下的分类问题^[5, 6]。

本文比较的主模型包括 Logistic Regression、SVM-RBF 和 Random Forest，并加入轻量 XGBoost 作为补充集成模型证据。其中 SVM 适合处理非线性间隔分类问题^[7]，Random Forest 通过多棵决策树集成提高稳定性^[8]，XGBoost 通过正则化梯度提升树刻画非线性特征交互^[10]。模型实现采用 scikit-learn 机器学习库^[9]；XGBoost 仅作为补充比较，不替代主推荐模型。对 Logistic Regression 与 SVM-RBF，标准化嵌入 sklearn Pipeline，并且只在训练折内拟合。

评价指标包括 accuracy、balanced accuracy、AUC、sensitivity、specificity 和 F1。其中 sensitivity 表示 PD 类召回率，specificity 表示 Healthy 类召回率。由于 Dataset 1 的 PD 受试者多于 Healthy 受试者，balanced accuracy 与 specificity 对判断模型是否过度偏向 PD 类尤为重要。

4.2 受试者级主结果

表 2 Dataset 1 subject-level acoustic-only 主结果

模型	Accuracy	Balanced Acc.	AUC	Sensitivity	Specificity	F1
Logistic Regression	0.7893	0.6989	0.8353	0.8825	0.5154	0.8616
Random Forest	0.8290	0.7004	0.8511	0.9623	0.4385	0.8936
SVM-RBF	0.8171	0.7118	0.8234	0.9249	0.4987	0.8825
XGBoost (补充)	0.8331	0.7025	0.8741	0.9679	0.4372	0.8965

Dataset 1 中 Random Forest 的 accuracy 和 AUC 较高，分别为 0.8290 和 0.8511，但 specificity 仅为 0.4385，说明该模型在类别不均衡背景下更倾向于判为 PD。补充 XGBoost 的 accuracy 和 AUC 分别为 0.8331 和 0.8741，但 specificity 为 0.4372，呈现与 Random Forest 相似的类别偏向。SVM-RBF 的 balanced accuracy 为 0.7118，在敏感度和特异度之间相对均衡；Logistic Regression 具有较好的可解释性，但综合指标略低。因此，XGBoost 结果用于支持非线性集成模型的补充判别能力，而不改变本文对 Dataset 1 主结果的审慎表述。

表 3 Dataset 2 subject-level acoustic-only 主结果

模型	Accuracy	Balanced Acc.	AUC	Sensitivity	Specificity	F1
Logistic Regression	0.7875	0.7875	0.8781	0.8250	0.7500	0.7905
Random Forest	0.7875	0.7875	0.8656	0.7250	0.8500	0.7748
SVM-RBF	0.8375	0.8375	0.8906	0.8000	0.8750	0.8300
XGBoost (补充)	0.7875	0.7875	0.8563	0.7250	0.8500	0.7748

Dataset 2 中 SVM-RBF 的 accuracy、balanced accuracy 和 AUC 均最高，specificity

也达到 0.8750，因此在该数据集上作为主推荐诊断模型。补充 XGBoost 的综合表现与 Random Forest 接近，未超过 SVM-RBF。Random Forest 的 specificity 较高但 sensitivity 较低，提示其对 Healthy 类更保守；Logistic Regression 的 sensitivity 较高但整体 accuracy 低于 SVM-RBF。

五 模型二：关键语音生物标志物识别

5.1 输入、方法与输出

模型二使用 Dataset 1 subject-level 数据，共 252 名受试者，其中 Healthy 64 名、PD 188 名。输入特征为 752 个声学特征，排除 `id`、`gender`、`class` 和 `sex_male`。候选特征识别方法包括 Mann-Whitney U 检验、Welch t-test、FDR-BH 多重检验校正^[12]、Cohen's *d*、Cliff's delta、L1 Logistic 稳定选择、ExtraTrees 重要性和 held-out permutation importance。L1 Logistic 的稀疏选择思想与 LASSO 正则化一脉相承^[13]。最终排序综合单变量差异、效应量、稳定性和模型重要性，而不是依赖单一树模型重要性。

为进一步检查候选特征解释是否与非线性集成模型一致，本文在相同受试者级折划分下训练轻量 XGBoost，并计算 held-out fold 上的 SHAP 平均绝对贡献值。XGBoost 特征重要性和 SHAP 加性解释框架^[10, 11]仅作为补充解释证据，不纳入综合 mean-rank 公式，因此不会改变 Top10、Top20 或 Top50 候选特征集合。

帕金森病语音研究中常用的 jitter、shimmer、MFCC、谐噪比和非线性复杂度等声学特征，已被用于发声障碍测量和疾病进展监测^[2, 3]。因此，本文将统计差异和模型解释结果统一表述为候选语音生物标志物或判别性声学特征，而不作临床因果外推。

5.2 Top20 候选语音生物标志物

表 4 Dataset 1 Top20 候选语音生物标志物

序号	特征	类别	Mean Rank
1	std_delta_delta_log_energy	Delta / Delta-delta	5.6667
2	std_6th_delta_delta	Delta / Delta-delta	15.5000
3	std_8th_delta	Delta / Delta-delta	20.6667
4	std_9th_delta_delta	Delta / Delta-delta	70.5833
5	std_7th_delta_delta	Delta / Delta-delta	79.4167
6	tqwt_TKE0_std_dec_12	TQWT	80.8333
7	tqwt_minValue_dec_12	TQWT	85.0000
8	std_9th_delta	Delta / Delta-delta	85.6667
9	mean_MFCC_2nd_coef	MFCC	86.1667
10	tqwt_TKE0_std_dec_11	TQWT	95.0833
11	std_10th_delta	Delta / Delta-delta	96.0833
12	tqwt_maxValue_dec_11	TQWT	97.4167
13	std_7th_delta	Delta / Delta-delta	97.6667
14	tqwt_kurtosisValue_dec_36	TQWT	101.5000
15	std_6th_delta	Delta / Delta-delta	101.9167
16	tqwt_kurtosisValue_dec_27	TQWT	103.7500
17	tqwt_energy_dec_12	TQWT	112.5833
18	DFA	Nonlinear	118.4167
19	tqwt_entropy_shannon_dec_17	TQWT	128.0833
20	tqwt_stdValue_dec_7	TQWT	131.6667

5.3 结果解释

Top20 中 Delta / Delta-delta 动态谱波动特征和 TQWT 多尺度特征占主要部分。Top50 类别分布为：TQWT 31 个，Delta / Delta-delta 16 个，Shimmer 1 个，MFCC

1 个, Nonlinear 1 个。该分布提示 PD/Healthy 的声学差异主要体现在动态谱波动和多尺度时频结构上。

补充 XGBoost 与 SHAP 结果提供了与主排序相互印证的非线性解释证据。SHAP 平均绝对贡献前三位均属于 Delta / Delta-delta 动态谱波动特征; XGBoost 重要性排名靠前的特征则以 TQWT 和 Delta / Delta-delta 为主。这说明补充集成模型同样将主要判别贡献集中在 Delta / Delta-delta 与 TQWT 两类声学维度上, 与 Top20/Top50 的类别构成一致。

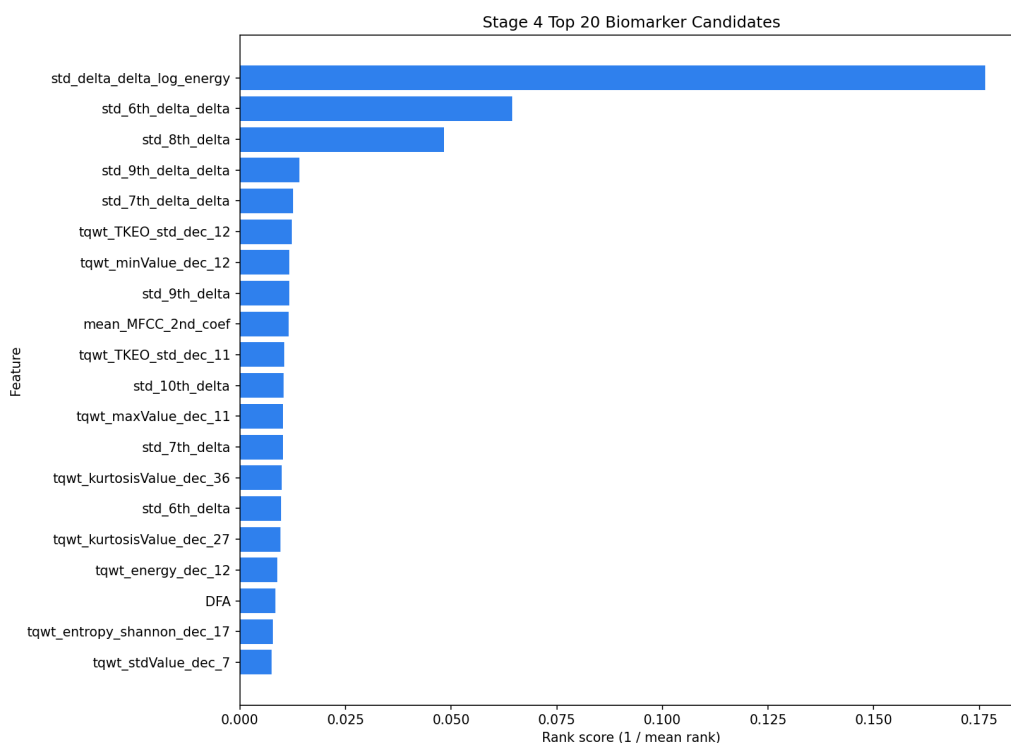


图 1 Top20 候选语音生物标志物综合排序

图中展示综合排名得分最高的 20 个候选声学特征。排名靠前的特征并非由单一模型重要性决定, 而是综合了单变量统计差异、效应量、稳定选择、树模型重要性和置换重要性等证据。排名前列的特征集中于 Delta / Delta-delta 和 TQWT, 说明 PD/Healthy 的主要判别信息与动态谱变化和多尺度时频结构有关。该图支持候选特征排序的稳定解释, 但不能说明这些声学特征是疾病发生发展的临床原因。

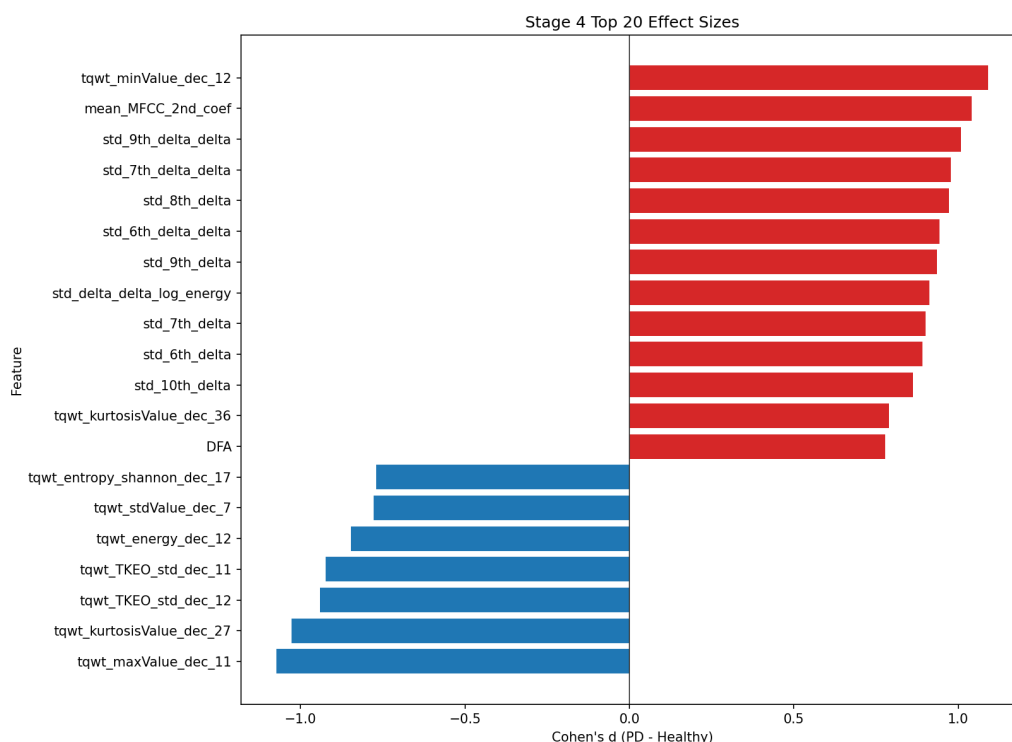


图 2 Top20 候选特征效应量

该图展示 Top20 特征的效应量方向和大小。正负方向表示 PD 组和 Healthy 组在该特征上的均值差异方向，绝对值越大表示组间差异越明显。效应量图与综合排序图共同说明，部分动态谱波动特征具有较明显的组间差异，可为二分类模型提供判别信息。需要强调的是，效应量只能说明统计关联和判别能力，不能推出该特征是帕金森病的临床致病机制。

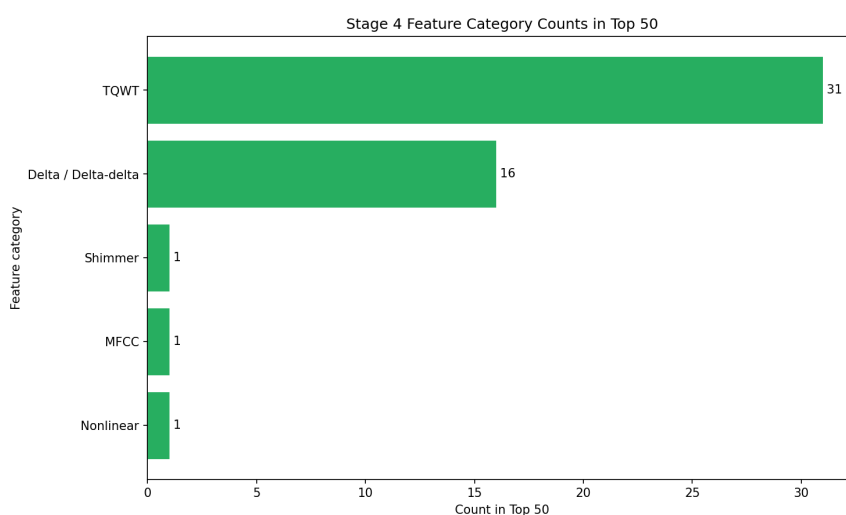


图 3 Top50 候选特征类别分布

该图显示 Top50 候选特征的类别构成，其中 TQWT 和 Delta / Delta-delta 占主导。这一结果与 Top20 排名一致，说明候选语音生物标志物不是零散分布在所有特征类别中，而是集中在多尺度声学波动和动态谱变化相关特征中。该集中性增强了模型解释的一致性，但仍不能替代独立数据集和临床机制验证。

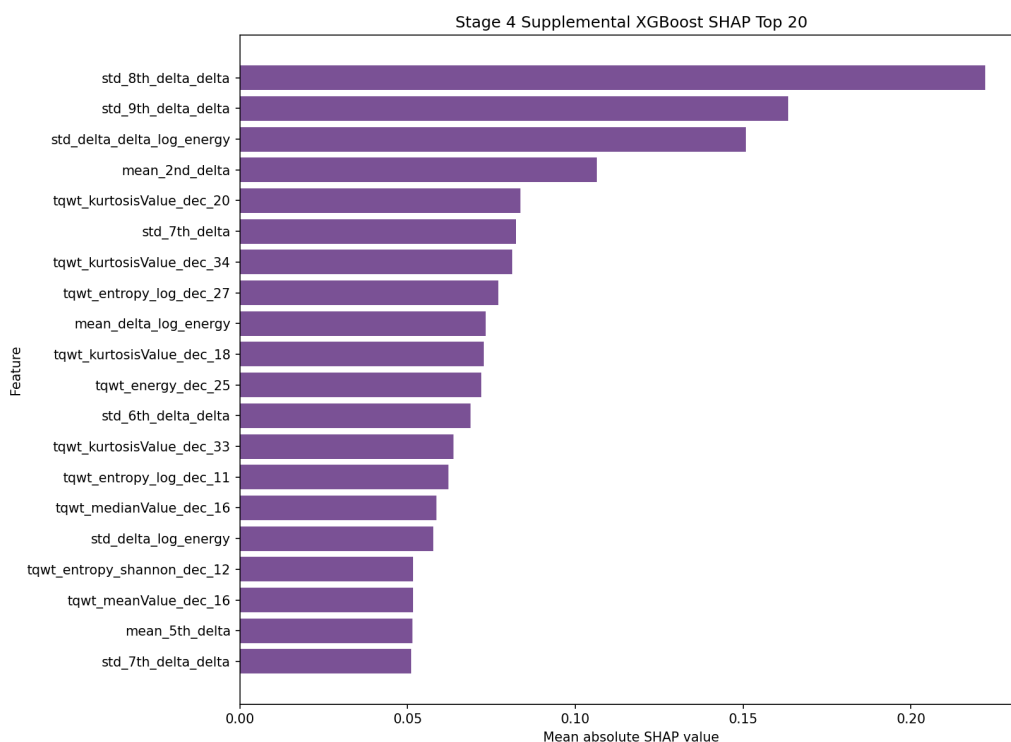


图 4 XGBoost 补充模型的 SHAP 平均绝对贡献 Top20

该图展示 XGBoost 补充模型在 held-out fold 上得到的 SHAP 平均绝对贡献。排名靠前的特征仍主要来自 Delta / Delta-delta 和 TQWT，说明非线性集成模型对判别信息来源的解释与综合排序结果基本一致。需要强调的是，SHAP 值表示模型预测层面的局部加性贡献，不表示临床因果效应；本文仅将其作为补充解释证据，而不作为候选特征主排序或后续 PD 内部分型输入。

上述特征只能称为候选语音生物标志物或判别性声学特征。由于本文没有临床机制实验和外部因果验证，不能作因果机制外推。

六 模型三：Dataset 2 声音特征智能诊断

Dataset 2 包含 80 名受试者、240 条录音，每名受试者 3 次录音，受试者级 Healthy 与 PD 各 40 名，类别均衡。该数据集的声学特征数为 44，与 Dataset 1 的高维 TQWT 特征空间不同，因此本文将其作为独立数据集单独建模和解释，而不与 Dataset 1 直接合并训练。

建模流程与模型一保持一致：输入为 acoustic-only 特征，主模型包括 Logistic Regression、SVM-RBF 和 Random Forest，并以 XGBoost 作为补充集成模型参考。交叉验证采用 StratifiedGroupKFold 并按 ID 分组。标准化嵌入 Pipeline，仅在训练折内拟合。输出为受试者级 PD/Healthy 辅助诊断结果。

从表 2 可见，SVM-RBF 在 Dataset 2 上取得 accuracy 0.8375、balanced accuracy 0.8375、AUC 0.8906、sensitivity 0.8000、specificity 0.8750 和 F1 0.8300，综合表现最好。补充 XGBoost 的 accuracy 和 balanced accuracy 均为 0.7875，AUC 为 0.8563，未超过 SVM-RBF。Random Forest 的 specificity 较高但 sensitivity 较低，提示其对 Healthy 类更保守；Logistic Regression 的 sensitivity 较高但整体 accuracy 低于 SVM-RBF。

Dataset 2 样本量较小，但类别均衡、每名受试者录音次数一致，适合作为独立验证风格的参考分析。由于两个数据集特征空间不完全一致，本文不将 Dataset 2 结果解释为 Dataset 1 模型的严格外部验证，而作为同一研究问题下的独立声学诊断建模证据。

七 模型四：二类潜在语音表型辅助分型

7.1 建模边界

题目第四问涉及运动型和非运动型帕金森病，但给定数据没有真实运动型/非运动型临床标签。因此，本文不训练真实临床亚型监督分类器，而是在 Dataset 1 的 188 名 PD 受试者内部进行 $k = 2$ 无监督聚类，得到基于语音特征的二类潜在表型。该结果可作为探索性辅助分型线索，不能等同于真实临床亚型诊断。

7.2 输入、方法与结果

候选输入特征集包括 Top20、Top50、all acoustic 以及 PD-only 内部筛选得到的相关性去冗余、高 IQR/MAD 和 SparsePCA 表示方案。综合内部聚类指标、簇均衡性和 10000 组 Dirichlet 权重敏感性分析后，本文采用多候选报告口径。top20_biomarkers / pca_5 / KMeans 的 top-1 频率最高，为 37.3%，低于 60% 的稳健主推荐阈值，因此表述为“相对推荐候选方案”，不作为唯一最优结论。

所有候选方案先执行标准化，再按方案进行 PCA 和聚类。聚类输入不包含 id、gender、class、sex_male、record_count 或任何标签列。聚类质量通过 silhouette、Calinski-Harabasz、Davies-Bouldin 和重采样 ARI 等指标综合评价^[14, 15, 16, 17]。

表 5 二类潜在语音表型相对推荐候选方案

候选方案	特征 / 表示 / 聚类方法	top-1 频率	top-3 频率	判定
候选 1	top20_biomarkers / pca_5 / KMeans	37.3%	63.2%	相对推荐
候选 2	top50_biomarkers / pca_5 / KMeans	22.4%	35.2%	可接受候选
候选 3	pd_high_iqr_top50 / pca_5 / AgglomerativeClustering	21.0%	28.9%	可接受候选

表 6 二类潜在语音表型相对推荐方案指标

项目	数值
相对推荐候选方案	top20_biomarkers / pca_5 / KMeans
PD 受试者数	188
Cluster 0 人数	62
Cluster 1 人数	126
Smaller cluster ratio	0.3298
Silhouette	0.3674
Stability ARI mean	0.9897
权重敏感性 top-1 频率	37.3%

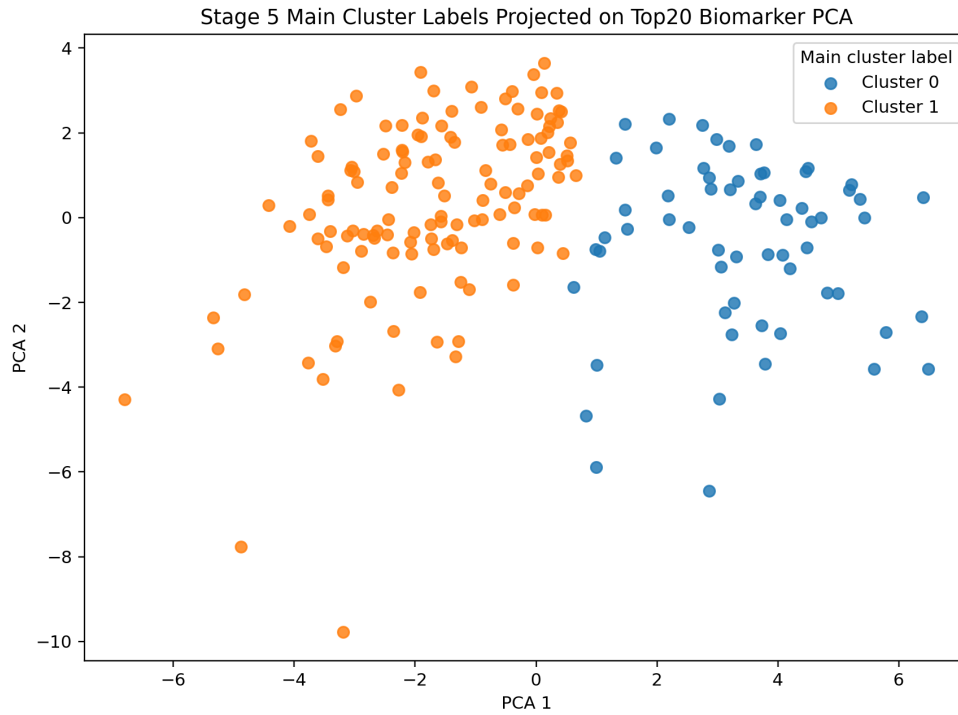


图 5 Dataset 1 PD 受试者二类潜在语音表型 PCA 可视化

该图将相对推荐候选方案得到的二类聚类标签投影到 Top20 特征的前两个 PCA 方向上。图中两类样本在二维空间中呈现较明显的左右分离趋势，说明 Top20 候选特征能够支持 PD 受试者内部二类潜在语音表型划分。但 PCA 二维图只是可视化投影，不能替代 silhouette、簇规模和稳定性 ARI 等定量指标，也不能直接说明真实临床亚型存在。

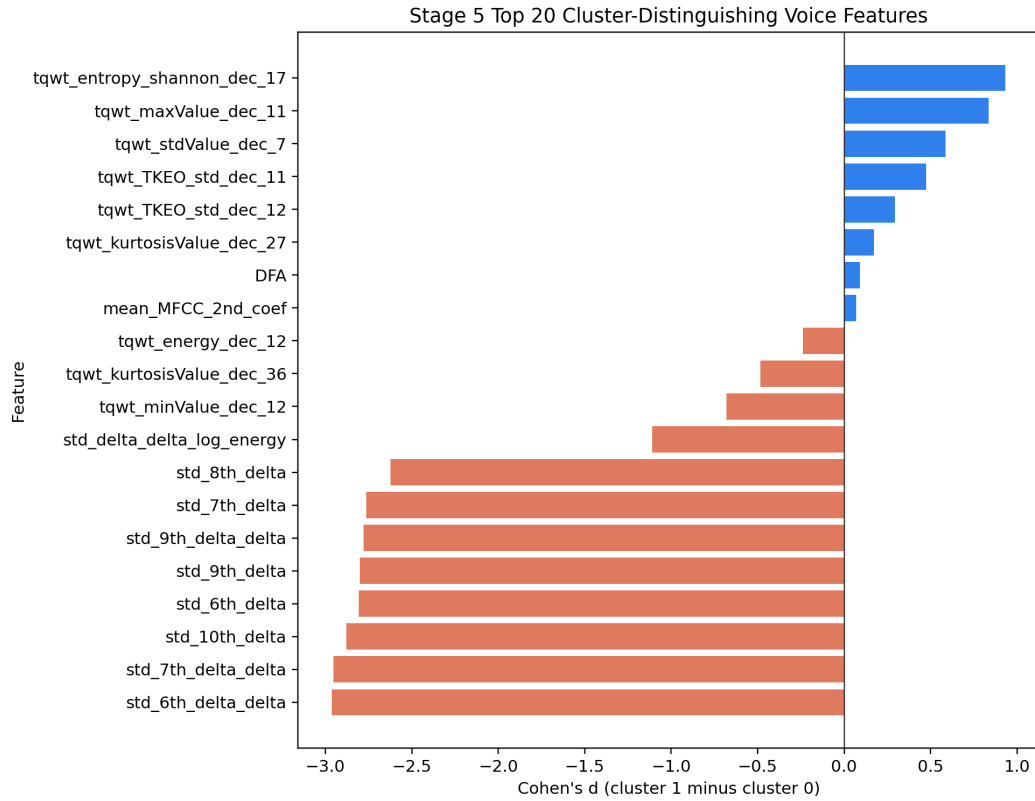


图 6 二类潜在语音表型的 Top20 特征差异

该图以 Cohen's d 展示二类潜在表型在 Top20 特征上的差异方向。横轴为 cluster 1 minus cluster 0, 因此负值表示 cluster 0 更高, 正值表示 cluster 1 更高。图中 cluster 0 在多项 Delta / Delta-delta 动态谱波动特征上更高, cluster 1 在若干 TQWT 特征上更高, 提示两簇主要反映动态谱波动和多尺度声学结构的差异。该差异只能解释为语音表型差异, 不能直接解释为真实运动型/非运动型临床差异。

7.3 表型解释

相对推荐候选方案得到 cluster 0 共 62 人、cluster 1 共 126 人。cluster 0 在多项 Delta / Delta-delta 动态谱波动特征上更高, 例如 std_6th_delta_delta、std_7th_delta_delta、std_10th_delta、std_6th_delta 等。cluster 1 在部分 TQWT 多尺度特征上更高, 例如 tqwt_entropy_shannon_dec_17、tqwt_maxValue_dec_11、tqwt_stdValue_dec_7、tqwt_TKEO_std_dec_11 等。

后续聚类标签复现模型学习的是上述无监督聚类标签, 用于将探索性分型规则模型化。该分类器不是根据真实运动型/非运动型标签训练, 不能解释为真实临床亚型诊断性能。

八 模型五：六类潜在语音表型辅助分型

8.1 建模边界

题目第五问涉及六类帕金森病症状，但数据集中没有静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆、睡眠障碍六类真实临床症状标签。因此，本文只在 Dataset 1 的 188 名 PD 受试者内部固定 $k = 6$ 进行无监督聚类，输出基于语音特征的六类潜在语音表型。六个 cluster 编号没有直接临床症状含义，仍需真实临床症状标签进一步验证。

8.2 输入、方法与结果

六类潜在语音表型建模对 Top20、Top50、all acoustic 以及 PD-only 内部筛选得到的特征表示进行候选方案比较，并做 $k = 2$ 到 $k = 8$ 的参考扫描。在不对 Top20/Top50 设置解释性奖励、且不把二类聚类一致性作为评分项的条件下，top20_biomarkers / pca_5 / AgglomerativeClustering 的权重敏感性 top-1 频率为 87.1%，达到 60% 的稳健主推荐阈值。因此，本文将其作为 $k = 6$ 潜在语音表型聚类的稳健主推荐方案。

所有表示均先进行标准化，PCA 和聚类仅使用声学特征；二类聚类标签没有进入六类聚类输入。聚类评价同样结合 silhouette、Calinski-Harabasz、Davies-Bouldin 和 ARI 等指标^[14, 15, 16, 17]。

表 7 六类潜在语音表型稳健主推荐方案指标

项目	数值
稳健主推荐方案	top20_biomarkers / pca_5 / AgglomerativeClustering
PD 受试者数	188
六簇人数	13、55、34、28、25、33
最小簇比例	0.0691
Silhouette	0.1823
Stability ARI mean	0.4582
权重敏感性 top-1 频率	87.1%
二类与六类表型 NMI	0.4089
Collapsed majority ARI	0.8128

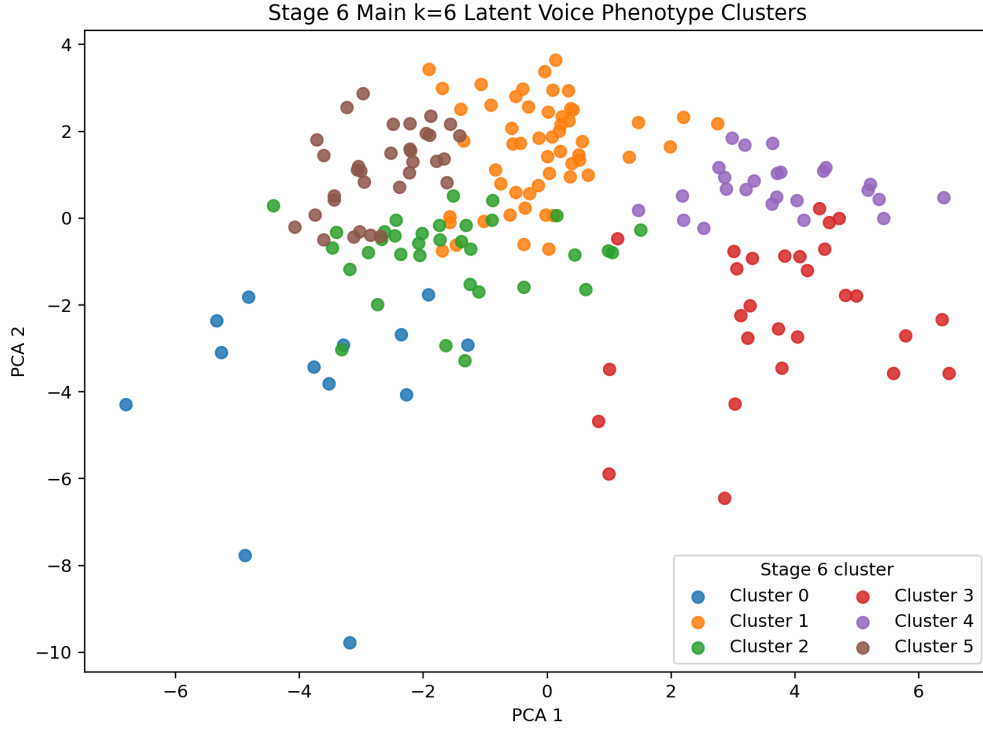


图 7 Dataset 1 PD 受试者六类潜在语音表型 PCA 可视化

该图将六类潜在语音表型投影到 PCA 二维平面。部分类别在平面上具有较清晰的区域分布，但也存在一定重叠，说明 $k = 6$ 划分比 $k = 2$ 更细，同时稳定性和分离度更弱。因此该图支持“探索性六类语音表型”的表述，而不能支持“六类临床症状完全可分”的结论。

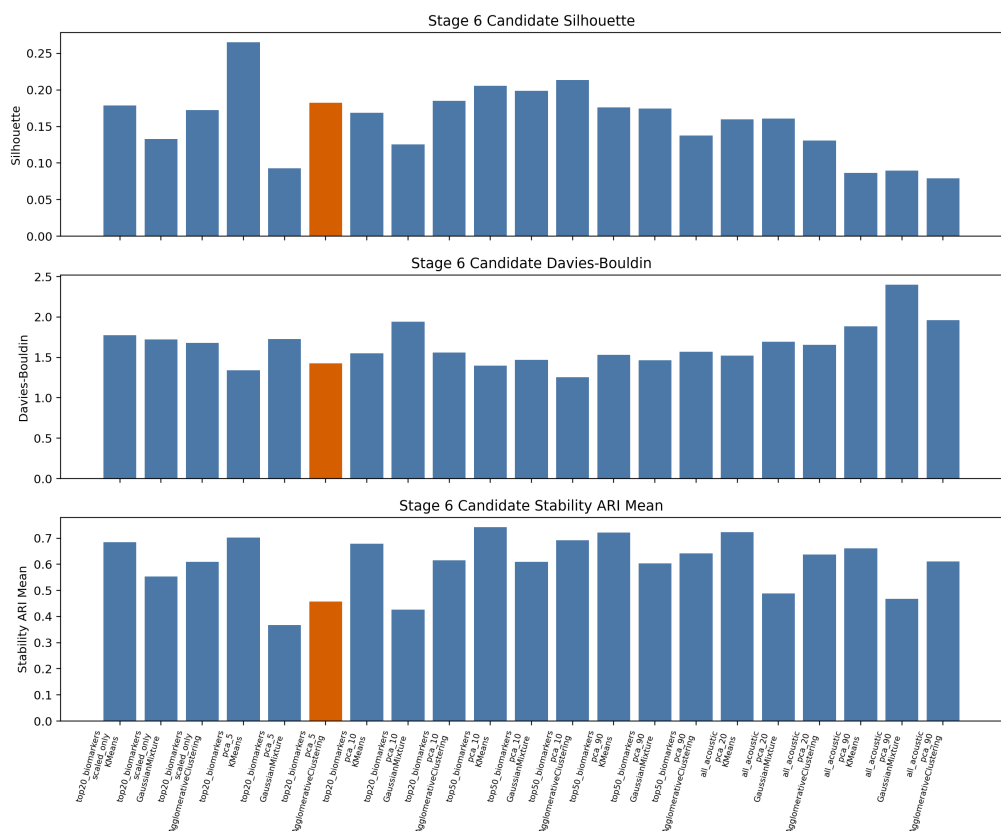


图 8 六类潜在语音表型候选方案指标比较

该候选指标图用于说明主推荐方案不是单一指标最优的机械选择，而是在簇规模约束、稳定性和内部聚类指标之间折中选择。权重敏感性分析进一步显示，该方案在无解释性奖励和无二类聚类一致性评分项的设定下仍保持较高 top-1 频率。该图和敏感性分析均不能说明六类表型具有外部临床标签验证。

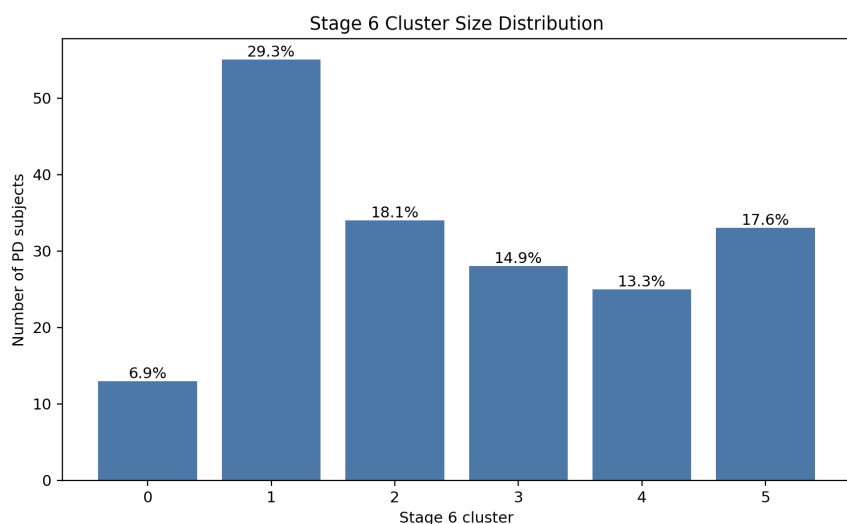


图 9 六类潜在语音表型簇规模分布

该图展示六个簇的人数分布。主方案最小簇为 13 人，占 6.9%，没有出现 1 人或 2 人小簇，说明该方案在固定 $k = 6$ 的前提下避免了极端碎片化划分。簇规模分布只能说明无监督划分在样本数量上的可解释性，不能说明各簇对应真实临床症状类别。

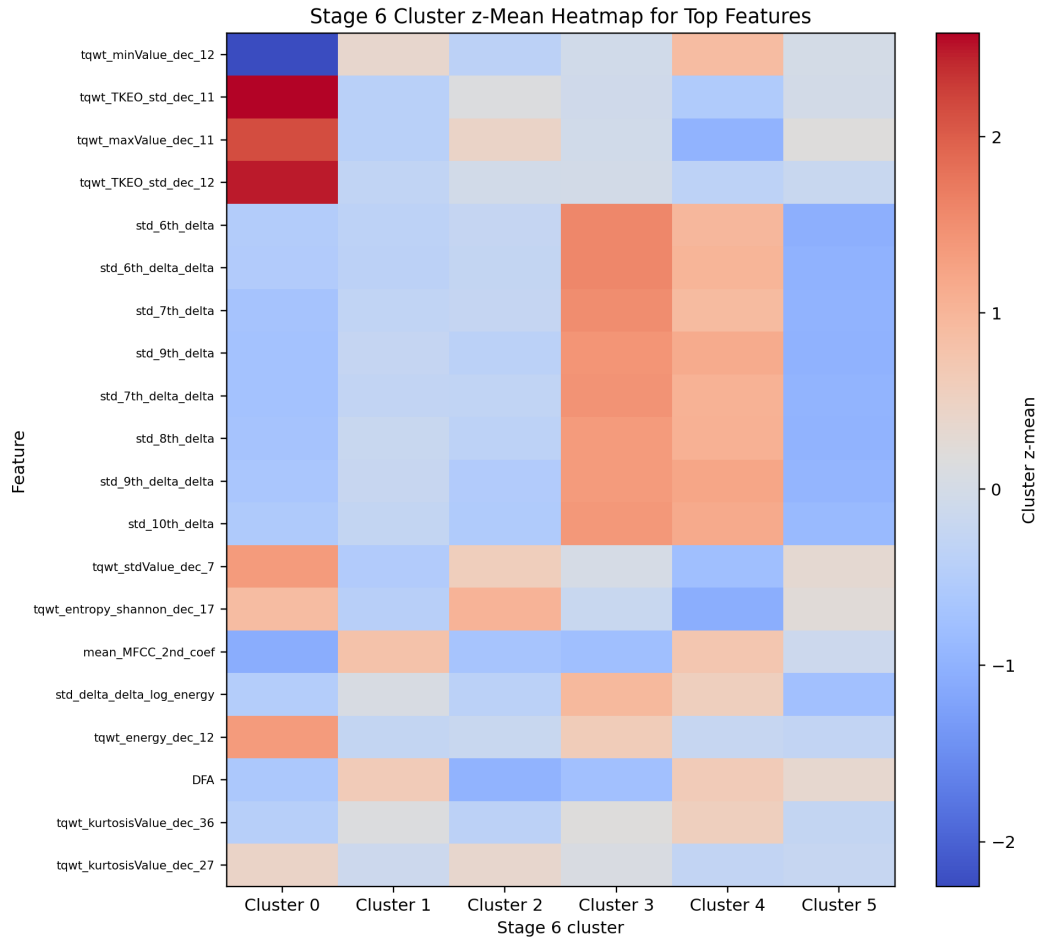


图 10 六类潜在语音表型特征热图

该热图展示六个簇在主要区分特征上的标准化均值差异，颜色深浅反映各簇在对应特征上的相对高低。Cluster 0 在部分 TQWT 特征上明显较高，Cluster 3 和 Cluster 4 在多项 Delta / Delta-delta 特征上较高，Cluster 5 在动态谱波动特征上相对较低。该图提供了六类潜在语音表型的声学画像基础，但不能说明这些画像对应具体临床症状。

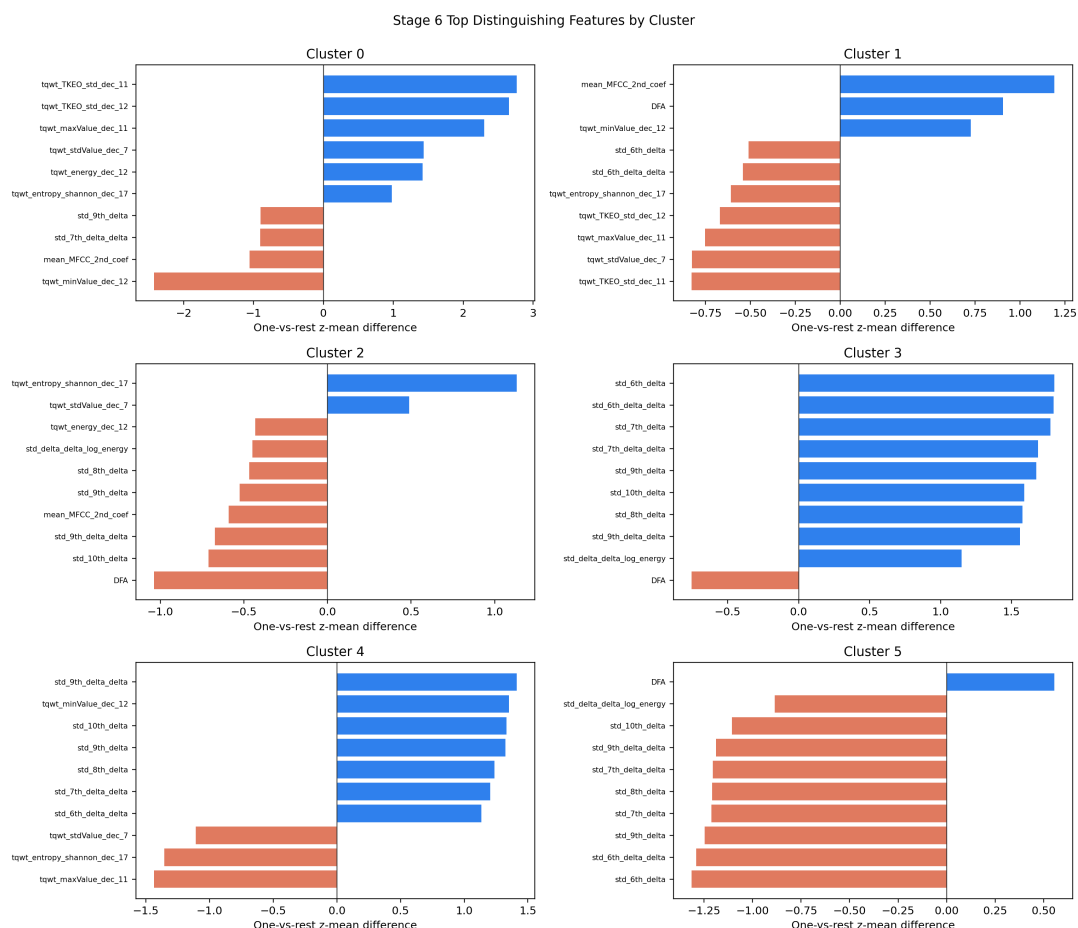


图 11 六类潜在语音表型各簇 Top 区分特征

该图进一步给出不同簇的主要区分特征，有助于观察各类潜在语音表型的具体声学画像。不同簇的 Top 特征集中在 TQWT 与 Delta / Delta-delta 相关特征，说明六类划分主要反映多尺度时频结构和动态谱波动模式的差异。该图只能解释声学空间内的聚类画像，不能作为临床症状命名依据。

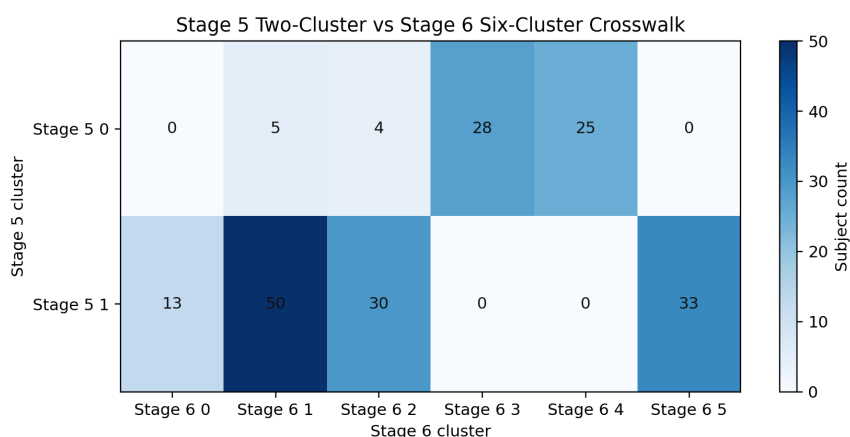


图 12 二类潜在表型与六类潜在表型交叉关系

该图展示二类表型与六类表型之间的交叉关系。图中六类表型的部分簇主要落在二类表型 cluster 0，另一些簇主要落在二类表型 cluster 1，说明六类潜在表型在一定程

度上可以看作二类潜在表型的进一步细分。结合 NMI 0.4089 和 collapsed majority ARI 0.8128, 可说明两级语音表型之间存在中等程度对应关系。但这只是声学聚类标签之间的关系, 不能解释为运动型/非运动型到六类临床症状的真实层级关系。

8.3 结果解释

六类潜在语音表型主要由 Delta / Delta-delta 与 TQWT 特征区分。部分簇表现为动态谱波动特征较高, 部分簇则表现为 TQWT 多尺度能量、熵或统计量差异。与二类方案相比, 六类方案的 silhouette 下降到 0.1823, stability ARI mean 下降到 0.4582, 说明固定 $k = 6$ 后聚类结构更细但稳定性和分离度较弱。

六类潜在表型与二类潜在表型存在一定嵌套关系, NMI 为 0.4089, 按六簇多数映射回二簇后的 collapsed majority ARI 为 0.8128。该交叉关系只能说明六类潜在表型与二类潜在表型在声学空间中存在一定对应, 不能解释为真实临床分层诊断。

六类聚类标签复现模型预测的是六类聚类标签, 不是真实静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆或睡眠障碍标签。因此, 其指标只表示复现无监督聚类划分的能力, 不表示六类临床症状诊断能力。

九 PD 内部分型特征扩展与权重敏感性分析

考虑到 Healthy/PD 判别特征可能主要刻画疾病有无或总体异常程度，本文在 PD 受试者内部进一步构建分型特征方案，包括相关性去冗余、高 IQR/MAD 特征筛选和 SparsePCA 表示。为避免把解释便利性混入客观评价，推荐评分不对 Top20/Top50 设置额外奖励，也不把二类聚类与六类聚类的一致性作为六类方案的评分项。所有候选方案均在 Dataset 1 的 188 名 PD 受试者内部比较，不引入 Healthy 样本或记录级录音样本。

权重敏感性分析同时考虑 silhouette、Calinski-Harabasz、反向 Davies-Bouldin、稳定性 ARI 和簇均衡性五个指标，并随机生成 10000 组 Dirichlet 权重。每组权重下重新计算综合分并记录排名第一和排名前三的候选方案，由此判断推荐结论是否依赖某一组主观权重。

二类潜在语音表型采用多候选报告口径。top20_biomarkers / pca_5 / KMeans 的 top-1 频率最高，为 37.3%，但低于 60% 的稳健主推荐阈值，因此作为相对推荐候选方案。与此同时，top50_biomarkers / pca_5 / KMeans 和 pd_high_iqr_top50 / pca_5 / AgglomerativeClustering 也作为可接受候选同步报告。这一结果说明二类潜在语音表型对权重设定仍有一定敏感性，不宜写作唯一稳健最优。

六类潜在语音表型采用 top20_biomarkers / pca_5 / AgglomerativeClustering 作为稳健主推荐方案。该方案 top-1 频率为 87.1%，在无解释性奖励、无二类聚类一致性评分项的设定下仍达到稳健主推荐阈值。因此，本文将该方案限定表述为 $k = 6$ 潜在语音表型聚类，而不写作由临床症状标签监督得到的诊断模型。

表 8 二类潜在语音表型权重敏感性主要候选

候选	特征 / 表示 / 聚类方法	top-1 频率	top-3 频率
1	top20_biomarkers / pca_5 / KMeans	37.3%	63.2%
2	top50_biomarkers / pca_5 / KMeans	22.4%	35.2%
3	pd_high_iqr_top50 / pca_5 / AgglomerativeClustering	21.0%	28.9%

表 9 六类潜在语音表型权重敏感性主要候选

候选	特征 / 表示 / 聚类方法	top-1 频率	判定
1	top20_biomarkers / pca_5 / AgglomerativeClustering	87.1%	稳健主推荐
2	all_acoustic / pca_90 / GaussianMixture	12.9%	可接受候选

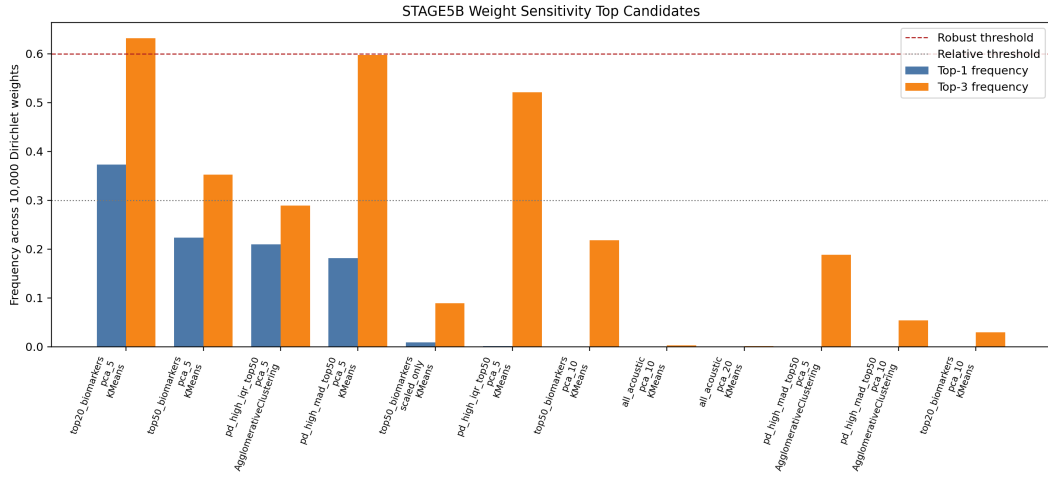


图 13 二类潜在语音表型权重敏感性主要候选

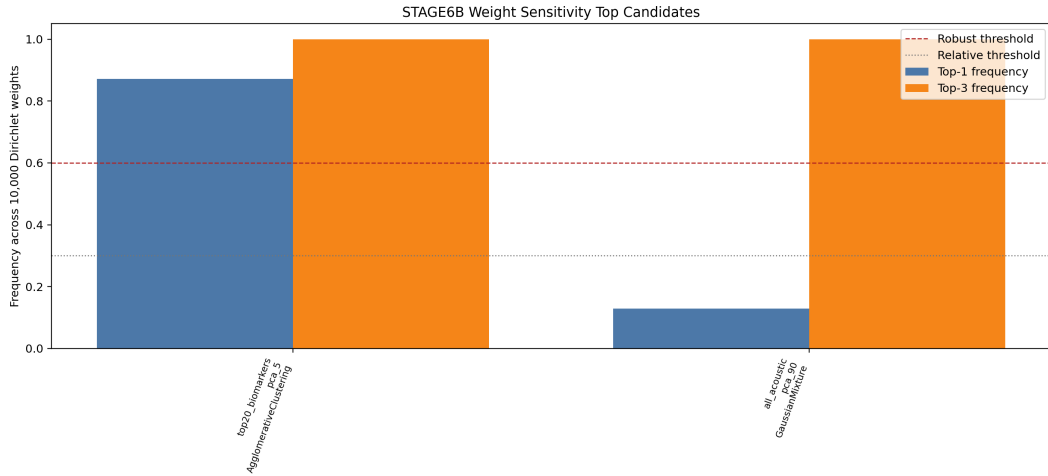


图 14 六类潜在语音表型权重敏感性主要候选

综上，二类潜在语音表型结果采用“相对推荐候选方案 + 多候选并列报告”的表述；六类潜在语音表型结果采用“稳健主推荐方案”的表述，但严格限定为声学空间中的 $k = 6$ 潜在语音表型聚类。

十 模型优缺点分析

10.1 优点

第一，本文严格识别了两个数据集的重复录音结构，监督学习中按受试者分组交叉验证，并以 subject-level 指标作为主结果，降低了样本单位错误带来的性能高估风险。第二，主声学模型排除了 `sex_male`，避免将人口学变量混入声学诊断结论。第三，候选语音生物标志物识别同时结合统计检验、效应量、稳定选择和模型重要性，并以 XGBoost/SHAP 作补充解释一致性检查，使特征解释不依赖单一方法。第四，对缺少真实标签的第四问和第五问，本文采用无监督潜在表型分析，并明确其探索性边界。第五，全流程一致性检查对分组、泄漏和指标一致性进行了系统控制。

10.2 局限性

本文也存在明显局限。Dataset 1 受试者数有限且类别不均衡，导致部分模型特异度偏低；补充 XGBoost 虽提高了部分判别指标，但同样表现出 specificity 偏低的问题，不能据此写成临床确诊模型。Dataset 2 虽类别均衡，但样本量较小。两个数据集的特征空间不完全一致，不能简单合并或直接外推。SHAP 解释反映的是模型预测贡献，不等同于临床因果机制。第四问和第五问缺少真实临床症状标签，因此聚类结果只能称为潜在语音表型，不能直接对应运动型/非运动型或六类临床症状。聚类表型仍需要真实临床标签、纵向随访和外部样本验证。

十一 结论

针对问题一,本文在 Dataset 1 上建立受试者级 PD/Healthy 辅助诊断模型。Random Forest 的 accuracy 和 AUC 较高,但 specificity 偏低;补充 XGBoost 的 accuracy 和 AUC 分别达到 0.8331 和 0.8741,但 specificity 仍为 0.4372;SVM-RBF 的 balanced accuracy 更稳健。该结果说明语音声学特征具有一定辅助筛查价值,但不能替代临床诊断。

针对问题二,综合多种统计和模型解释证据,本文得到 Top20 候选语音生物标志物。其中特征主要集中于 Delta / Delta-delta 动态谱波动和 TQWT 多尺度声学结构;补充 XGBoost/SHAP 解释也将主要贡献集中在这两类特征上,增强了判别性特征解释的一致性。但上述证据仍只能说明模型预测和统计关联,不能解释为临床因果结论。

针对问题三, Dataset 2 按受试者 ID 分组交叉验证后, SVM-RBF 取得 accuracy 0.8375、balanced accuracy 0.8375 和 AUC 0.8906,是该数据集上的主推荐模型;补充 XGBoost 未超过 SVM-RBF。该结论支持语音特征在独立数据集中的辅助诊断价值,但仍受样本量限制。

针对问题四,本文在 188 名 PD 受试者内部得到二类潜在语音表型。权重敏感性分析显示, top20_biomarkers / pca_5 / KMeans 的 top-1 频率最高但仅为 37.3%,因此应表述为相对推荐候选方案,并同时报告 top50_biomarkers / pca_5 / KMeans 和 pd_high_iqr_top50 / pca_5 / AgglomerativeClustering 等可接受候选。该结构主要由动态谱波动特征和部分 TQWT 特征区分,但不是真实运动型/非运动型诊断。

针对问题五,本文固定 $k = 6$ 得到六类潜在语音表型。权重敏感性分析显示, top20_biomarkers / pca_5 / AgglomerativeClustering 的 top-1 频率为 87.1%,可作为稳健主推荐方案。六簇主要由 Delta / Delta-delta 与 TQWT 特征区分,但其标签只表示声学空间中的潜在结构,不能写作由六类真实症状标签验证的诊断结果。后续若获得真实六类临床症状标签,应重新建立受试者级监督验证框架。

参考文献

- [1] Greenland J C, Barker R A. The differential diagnosis of Parkinson's disease[M]// Stoker T B, Greenland J C, editors. Parkinson's Disease: Pathogenesis and Clinical Aspects. Brisbane: Codon Publications, 2018.
- [2] Little M A, McSharry P E, Hunter E J, Spielman J, Ramig L O. Suitability of dysphonia measurements for telemonitoring of Parkinson's disease[J]. IEEE Transactions on Biomedical Engineering, 2009, 56(4): 1015–1022.
- [3] Tsanas A, Little M A, McSharry P E, Ramig L O. Accurate telemonitoring of Parkinson's disease progression by noninvasive speech tests[J]. IEEE Transactions on Biomedical Engineering, 2010, 57(4): 884–893.
- [4] Sakar B E, Isenkul M E, Sakar C O, Sertbas A, Gorgen F, Delil S, Apaydin H, Kursun O. Collection and analysis of a Parkinson speech dataset with multiple types of sound recordings[J]. IEEE Journal of Biomedical and Health Informatics, 2013, 17(4): 828–834.
- [5] Naranjo L, Perez C J, Campos-Roca Y, Martin J. Addressing voice recording replications for Parkinson's disease detection[J]. Expert Systems with Applications, 2016, 46: 286–292.
- [6] Naranjo L, Perez C J, Martin J, Campos-Roca Y. A two-stage variable selection and classification approach for Parkinson's disease detection by using voice recording replications[J]. Computer Methods and Programs in Biomedicine, 2017, 142: 147–156.
- [7] Cortes C, Vapnik V. Support-vector networks[J]. Machine Learning, 1995, 20(3): 273–297.
- [8] Breiman L. Random forests[J]. Machine Learning, 2001, 45(1): 5–32.
- [9] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, et al. Scikit-learn: Machine learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825–2830.
- [10] Chen T, Guestrin C. XGBoost: A scalable tree boosting system[C]// Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016: 785–794.
- [11] Lundberg S M, Lee S I. A unified approach to interpreting model predictions[C]// Advances in Neural Information Processing Systems. 2017, 30: 4765–4774.
- [12] Benjamini Y, Hochberg Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing[J]. Journal of the Royal Statistical Society: Series B, 1995, 57(1): 289–300.
- [13] Tibshirani R. Regression shrinkage and selection via the lasso[J]. Journal of the Royal Statistical Society: Series B, 1996, 58(1): 267–288.
- [14] Rousseeuw P J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis[J]. Journal of Computational and Applied Mathematics, 1987, 20: 53–65.

- [15] Calinski T, Harabasz J. A dendrite method for cluster analysis[J]. Communications in Statistics, 1974, 3(1): 1–27.
- [16] Davies D L, Bouldin D W. A cluster separation measure[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1979, PAMI-1(2): 224–227.
- [17] Hubert L, Arabie P. Comparing partitions[J]. Journal of Classification, 1985, 2: 193–218.

A 附录：程序代码

本附录给出建模、特征识别、潜在表型分析与权重敏感性分析所用的完整代码。XGBoost 补充二分类模型位于代码二，XGBoost 特征重要性与 SHAP 解释性分析位于代码三。为避免正文被程序实现细节打断，代码统一置于附录。

A.1 代码一：数据预处理与质量控制

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import matplotlib
6
7 matplotlib.use("Agg")
8
9 import matplotlib.pyplot as plt
10 import pandas as pd
11
12
13 ROOT = Path(__file__).resolve().parents[1]
14 DATA_DIR = ROOT / "data"
15 RESULTS_DIR = ROOT / "results"
16 FIGURES_DIR = RESULTS_DIR / "figures"
17
18 CSV_ENCODING = "utf-8"
19 OUTPUT_CSV_ENCODING = "utf-8-sig"
20
21 DATASET_CONFIG = {
22     "dataset1": {
23         "display_name": "Dataset 1",
24         "file_name": "pd_speech_features.csv",
25         "read_kwargs": {"header": 1, "encoding": CSV_ENCODING},
26         "read_method": 'pd.read_csv(path, header=1, encoding="utf-8")',
27         "id_col": "id",
28         "label_col": "class",
29         "gender_col": "gender",
30         "recording_col": None,
31         "metadata_cols": ["id", "gender", "class"],
32         "sex_male_rule": "sex_male = gender",
33     },
34     "dataset2": {
35         "display_name": "Dataset 2",
36         "file_name": "ReplicatedAcousticFeatures-ParkinsonDatabase.csv",
37         "read_kwargs": {"encoding": CSV_ENCODING},
38         "read_method": 'pd.read_csv(path, encoding="utf-8")',
39         "id_col": "ID",
40         "label_col": "Status",
41         "gender_col": "Gender",
42         "recording_col": "Recording",
43         "metadata_cols": ["ID", "Recording", "Status", "Gender"],
44         "sex_male_rule": "sex_male = 1 - Gender",
45     },
46 }
47
48 STAT_COLUMNS = ["mean", "std", "median", "min", "max", "IQR"]
49
50
51 def rel(path: Path) -> str:
52     return path.relative_to(ROOT).as_posix()
53
54
55 def find_source_csvs() -> dict[str, Path]:
56     paths = {path.name: path for path in DATA_DIR.rglob("*.csv")}
57     missing = [
58         config["file_name"]
59         for config in DATASET_CONFIG.values()
60         if config["file_name"] not in paths
61     ]
62     if missing:
63         raise FileNotFoundError(f"Missing source CSV files: {missing}")
64     return paths
65
66
67 def add_sex_male(df: pd.DataFrame, dataset_key: str) -> pd.DataFrame:
68     cfg = DATASET_CONFIG[dataset_key]
69     out = df.copy()
70     if dataset_key == "dataset1":
71         out["sex_male"] = out[cfg["gender_col"]]
72     elif dataset_key == "dataset2":
73         out["sex_male"] = 1 - out[cfg["gender_col"]]
74     else:
75         raise ValueError(f"Unknown dataset: {dataset_key}")
76     return out
77
78
79 def feature_columns(df: pd.DataFrame, dataset_key: str) -> list[str]:
80     cfg = DATASET_CONFIG[dataset_key]
81     excluded = set(cfg["metadata_cols"]) | {"sex_male"}
```

```

82     return [
83         column
84         for column in df.select_dtypes(include="number").columns
85         if column not in excluded
86     ]
87
88
89 def describe_features(
90     df: pd.DataFrame,
91     features: list[str],
92     dataset_key: str,
93     scope: str,
94     label_value: object = "",
95 ) -> pd.DataFrame:
96     numeric = df[features]
97     q1 = numeric.quantile(0.25)
98     q3 = numeric.quantile(0.75)
99     stats = pd.DataFrame(
100         {
101             "dataset": dataset_key,
102             "scope": scope,
103             "label_value": label_value,
104             "feature": features,
105             "n": numeric.count().reindex(features).astype(int).values,
106             "mean": numeric.mean().reindex(features).values,
107             "std": numeric.std(ddof=1).reindex(features).values,
108             "median": numeric.median().reindex(features).values,
109             "min": numeric.min().reindex(features).values,
110             "max": numeric.max().reindex(features).values,
111             "IQR": (q3 - q1).reindex(features).values,
112         }
113     )
114     return stats[["dataset", "scope", "label_value", "feature", "n"] + STAT_COLUMNS]
115
116
117 def record_level_stats(
118     df: pd.DataFrame, dataset_key: str, features: list[str]
119 ) -> pd.DataFrame:
120     cfg = DATASET_CONFIG[dataset_key]
121     frames = [describe_features(df, features, dataset_key, "record_overall")]
122     for label_value, group in df.groupby(cfg["label_col"], dropna=False):
123         frames.append(
124             describe_features(
125                 group, features, dataset_key, "record_by_label", label_value
126             )
127         )
128     return pd.concat(frames, ignore_index=True)
129
130
131 def subject_table(
132     df: pd.DataFrame, dataset_key: str, features: list[str]
133 ) -> pd.DataFrame:
134     cfg = DATASET_CONFIG[dataset_key]
135     id_col = cfg["id_col"]
136     label_col = cfg["label_col"]
137     gender_col = cfg["gender_col"]
138     feature_means = df.groupby(id_col, dropna=False)[features].mean().reset_index()
139     subject_meta = (
140         df.groupby(id_col, dropna=False)
141         .agg(
142             record_count=(id_col, "size"),
143             label_unique_count=(label_col, lambda s: s.nunique(dropna=False)),
144             gender_unique_count=(gender_col, lambda s: s.nunique(dropna=False)),
145             sex_male_unique_count=("sex_male", lambda s: s.nunique(dropna=False)),
146             label_value=(label_col, "first"),
147             raw_gender_value=(gender_col, "first"),
148             sex_male=("sex_male", "first"),
149         )
150         .reset_index()
151     )
152     return subject_meta.merge(feature_means, on=id_col, how="left")
153
154
155 def subject_level_stats(
156     subjects: pd.DataFrame, dataset_key: str, features: list[str]
157 ) -> pd.DataFrame:
158     frames = [describe_features(subjects, features, dataset_key, "subject_overall")]
159     for label_value, group in subjects.groupby("label_value", dropna=False):
160         frames.append(
161             describe_features(
162                 group, features, dataset_key, "subject_by_label", label_value
163             )
164         )
165     return pd.concat(frames, ignore_index=True)
166
167
168 def save_crosstab(
169     df: pd.DataFrame,
170     label_col: str,
171     output_path: Path,
172     index_name: str,
173 ) -> pd.DataFrame:

```

```

174 table = pd.crosstab(df[label_col], df["sex_male"], margins=True)
175 table.index.name = index_name
176 table.columns.name = "sex_male"
177 table.to_csv(output_path, encoding=OUTPUT_CSV_ENCODING)
178 return table
179
180
181 def save_distribution_figure(
182     record_counts: pd.Series,
183     subject_counts: pd.Series,
184     title: str,
185     x_label: str,
186     output_path: Path,
187 ) -> None:
188     fig, axes = plt.subplots(1, 2, figsize=(12, 6), dpi=120)
189     panels = [
190         ("Record level", record_counts, "#2f80ed"),
191         ("Subject level", subject_counts, "#27ae60"),
192     ]
193     for ax, (panel_title, counts, color) in zip(axes, panels):
194         counts = counts.sort_index()
195         labels = [str(label) for label in counts.index]
196         ax.bar(labels, counts.values, color=color)
197         ax.set_title(panel_title)
198         ax.set_xlabel(x_label)
199         ax.set_ylabel("Count")
200         ax.spines["top"].set_visible(False)
201         ax.spines["right"].set_visible(False)
202         for index, value in enumerate(counts.values):
203             ax.text(index, value, str(int(value)), ha="center", va="bottom")
204     fig.suptitle(title, fontsize=16, fontweight="bold")
205     fig.tight_layout(rect=(0, 0, 1, 0.94))
206     fig.savefig(output_path)
207     plt.close(fig)
208
209
210 def write_csv(df: pd.DataFrame, path: Path) -> None:
211     df.to_csv(path, encoding=OUTPUT_CSV_ENCODING, index=False)
212
213
214 def main() -> None:
215     RESULTS_DIR.mkdir(exist_ok=True)
216     FIGURES_DIR.mkdir(exist_ok=True)
217     source_paths = find_source_csvs()
218
219     loaded: dict[str, pd.DataFrame] = {}
220     subjects_by_dataset: dict[str, pd.DataFrame] = {}
221     feature_counts: dict[str, int] = {}
222     duplicate_counts: dict[str, int] = {}
223     label_conflicts: dict[str, int] = {}
224     gender_conflicts: dict[str, int] = {}
225     sex_conflicts: dict[str, int] = {}
226     crosstab_outputs: list[Path] = []
227     report_lines: list[str] = [
228         "## 第二阶段描述性统计与质量控制报告",
229         "",
230         "本報告由 `src/stage2_qc_pipeline.py` 生成，所有文本使用 UTF-8 写入，CSV 使用 utf-8-sig 写入以兼容 Excel。",
231         "",
232         "## 统一数据配置",
233         "",
234         "```python",
235         "DATASET_CONFIG = {",
236         "    'dataset1': {'id_col': 'id', 'label_col': 'class', 'gender_col': 'gender', 'metadata_cols': ['id', 'gender',
237         ↪ 'class']},",
238         "    'dataset2': {'id_col': 'ID', 'label_col': 'Status', 'gender_col': 'Gender', 'recording_col': 'Recording',
239         ↪ 'metadata_cols': ['ID', 'Recording', 'Status', 'Gender']},",
240         "}",
241         "```",
242         "",
243         "## 性别编码统一",
244         "",
245         "- 两个数据集原始性别编码方向不同。",
246         "- 为避免解释混乱，本阶段新增统一变量 `sex_male`，不覆盖原始 `gender/Gender`。",
247         "- `sex_male=1` 表示男性，`sex_male=0` 表示女性。",
248         "- Dataset 1: 保留原始列 `gender`，新增 `sex_male = gender`。",
249         "- Dataset 2: 保留原始列 `Gender`，新增 `sex_male = 1 - Gender`。",
250         "- 后续跨数据集分析和交叉表均使用 `sex_male`：原始 `gender/Gender` 只用于数据审计和溯源，不进入统一建模特征。",
251         "- 主模型应只使用语音特征；`sex_male` 不直接进入主模型，仅用于补充模型或敏感性分析，以检验性别混杂影响。",
252         "",
253     ]
254
255     for dataset_key, cfg in DATASET_CONFIG.items():
256         path = source_paths[cfg["file_name"]]
257         df_raw = pd.read_csv(path, **cfg["read_kwargs"])
258         df = add_sex_male(df_raw, dataset_key)
259         loaded[dataset_key] = df
260         features = feature_columns(df, dataset_key)
261         feature_counts[dataset_key] = len(features)
262         duplicate_counts[dataset_key] = int(df_raw.duplicated().sum())
263
264     record_stats = record_level_stats(df, dataset_key, features)
265     subjects = subject_table(df, dataset_key, features)

```

```

264 subjects_by_dataset[dataset_key] = subjects
265 subject_stats = subject_level_stats(subjects, dataset_key, features)
266
267 write_csv(record_stats, RESULTS_DIR / f"descriptive_statistics_{dataset_key}.csv")
268 write_csv(
269     subject_stats,
270     RESULTS_DIR / f"grouped_subject_statistics_{dataset_key}.csv",
271 )
272
273 for obsolete in RESULTS_DIR.glob(f"{dataset_key}_class_gender_crosstab*.csv"):
274     obsolete.unlink()
275 record_crosstab_path = (
276     RESULTS_DIR / f"{dataset_key}_class_sex_male_crosstab_record.csv"
277 )
278 subject_crosstab_path = (
279     RESULTS_DIR / f"{dataset_key}_class_sex_male_crosstab_subject.csv"
280 )
281 save_crosstab(
282     df,
283     cfg["label_col"],
284     record_crosstab_path,
285     cfg["label_col"],
286 )
287 save_crosstab(
288     subjects.rename(columns={"label_value": cfg["label_col"]}),
289     cfg["label_col"],
290     subject_crosstab_path,
291     cfg["label_col"],
292 )
293 crosstab_outputs.extend([record_crosstab_path, subject_crosstab_path])
294
295 label_conflicts[dataset_key] = int(
296     (subjects["label_unique_count"] > 1).sum()
297 )
298 gender_conflicts[dataset_key] = int(
299     (subjects["gender_unique_count"] > 1).sum()
300 )
301 sex_conflicts[dataset_key] = int(
302     (subjects["sex_male_unique_count"] > 1).sum()
303 )
304
305 save_distribution_figure(
306     df[cfg["label_col"]].value_counts(),
307     subjects["label_value"].value_counts(),
308     f"{cfg['display_name']} class distribution",
309     cfg["label_col"],
310     FIGURES_DIR / f"class_distribution_{dataset_key}.png",
311 )
312 save_distribution_figure(
313     df["sex_male"].value_counts(),
314     subjects["sex_male"].value_counts(),
315     f"{cfg['display_name']} sex_male distribution",
316     "sex_male",
317     FIGURES_DIR / f"gender_distribution_{dataset_key}.png",
318 )
319
320 report_lines.extend(
321     [
322         f"### {cfg['display_name']}",
323         "",
324         f"- 源文件: `{rel(path)}`",
325         f"- 读取方式: `{cfg['read_method']}`",
326         f"- 原始形状: `{df_raw.shape}`",
327         f"- 元信息列: `{', '.join(cfg['metadata_cols'])}`",
328         f"- 统一性别变量规则: `{cfg['sex_male_rule']}`",
329         f"- 数值语音特征数量: `{len(features)}`",
330         f"- 完全重复行数量: `{duplicate_counts[dataset_key]}`",
331         f"- 受试者数量: `{subjects.shape[0]}`",
332         f"- 按受试者标签不一致数量: `{label_conflicts[dataset_key]}`",
333         f"- 按受试者原始性别不一致数量: `{gender_conflicts[dataset_key]}`",
334         f"- 按受试者 `sex_male` 不一致数量: `{sex_conflicts[dataset_key]}`",
335         f"- 记录级类别分布: `{df[cfg['label_col']].value_counts().sort_index().to_dict()}`",
336         f"- 受试者级类别分布: `{subjects['label_value'].value_counts().sort_index().to_dict()}`",
337         f"- 记录级 `sex_male` 分布: `{df['sex_male'].value_counts().sort_index().to_dict()}`",
338         f"- 受试者级 `sex_male` 分布: `{subjects['sex_male'].value_counts().sort_index().to_dict()}`",
339         "",
340         f"### 输出",
341         f"- 记录级描述统计: `results/descriptive_statistics_{dataset_key}.csv`",
342         f"- 受试者聚合描述统计: `results/grouped_subject_statistics_{dataset_key}.csv`",
343         f"- 记录级类别 × `sex_male` 交叉表: `results/{record_crosstab_path.name}`",
344         f"- 受试者级类别 × `sex_male` 交叉表: `results/{subject_crosstab_path.name}`",
345         "",
346     ]
347 )
348
349 if cfg["recording_col"]:
350     recording_col = cfg["recording_col"]
351     recording_counts = df[recording_col].value_counts().sort_index()
352     per_subject_recording = df.groupby(cfg["id_col"])[recording_col].apply(
353         lambda s: tuple(sorted(s.dropna().tolist()))
354     )
355     expected = (1, 2, 3)

```

```

356         report_lines.extend(
357             [
358                 "### Recording 分布检查",
359                 f"- 记录级 `Recording` 分布: `{recording_counts.to_dict()}`",
360                 f"- 每名受试者均有且仅有一次 `Recording=1/2/3` 的数量: `{int((per_subject_recording ==
361                     expected).sum())}`",
362                 f"- 不满足上述条件的受试者数量: `{int((per_subject_recording != expected).sum())}`",
363                 ""
364             ]
365         )
366
367     dataset1 = loaded["dataset1"]
368     dataset2 = loaded["dataset2"]
369     dataset1_dedup = dataset1.drop_duplicates()
370     dataset1_dedup_path = RESULTS_DIR / "model_ready_dataset1_deduplicated.csv"
371     dataset2_prepared_path = RESULTS_DIR / "model_ready_dataset2_prepared.csv"
372     write_csv(dataset1_dedup, dataset1_dedup_path)
373     write_csv(dataset2, dataset2_prepared_path)
374
375     d1_dedup_record_counts = (
376         dataset1_dedup.groupby(DATASET_CONFIG["dataset1"]["id_col"])
377         .size()
378         .value_counts()
379         .sort_index()
380         .to_dict()
381     )
382
383     report_lines.extend(
384         [
385             "## Dataset 1 去重说明",
386             "",
387             "- 原始版保留 756 行, 用于数据描述。",
388             "- 去重版 755 行用于避免完全重复样本对建模产生重复加权。",
389             "- 使用去重版后, 不再严格满足每个受试者 3 条记录, 后续受试者级聚合必须显式说明。",
390             f"- 去重后受试者记录数分布: `{d1_dedup_record_counts}`。",
391             f"- 去重建模准备版: `results/{dataset1_dedup_path.name}`。",
392             f"- Dataset 2 prepared 建模准备版: `results/{dataset2_prepared_path.name}`。",
393             "",
394             "## 图像输出",
395             "",
396             "- `results/figures/class_distribution_dataset1.png`",
397             "- `results/figures/class_distribution_dataset2.png`",
398             "- `results/figures/gender_distribution_dataset1.png`",
399             "- `results/figures/gender_distribution_dataset2.png`",
400             "",
401             "## 边界说明",
402             "",
403             "本阶段只生成描述统计、受试者聚合、编码统一、交叉表、去重准备数据和基础分布图, 不生成预测结果、降维结果、合成采样结果、
404             ↪ 无监督分组结果或特征排序结果。",
405             ""
406         ]
407     )
408
409     (RESULTS_DIR / "descriptive_report.md").write_text(
410         "\n".join(report_lines), encoding="utf-8"
411     )
412
413     print("Generated stage 2 QC outputs:")
414     for path in [
415         RESULTS_DIR / "descriptive_statistics_dataset1.csv",
416         RESULTS_DIR / "descriptive_statistics_dataset2.csv",
417         RESULTS_DIR / "grouped_subject_statistics_dataset1.csv",
418         RESULTS_DIR / "grouped_subject_statistics_dataset2.csv",
419         *crosstab_outputs,
420         dataset1_dedup_path,
421         dataset2_prepared_path,
422         RESULTS_DIR / "descriptive_report.md",
423     ]:
424         print(f"- {rel(path)}")
425
426 if __name__ == "__main__":
427     main()

```

A.2 代码二：二分类辅助诊断模型与 XGBoost 补充模型

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from pathlib import Path
5 from typing import Any
6
7 import numpy as np
8 import pandas as pd
9 from sklearn.ensemble import RandomForestClassifier
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.metrics import (
12     accuracy_score,
13     balanced_accuracy_score,
14     confusion_matrix,
15     f1_score,
16     precision_score,
17     recall_score,
18     roc_auc_score,
19 )
20 from sklearn.model_selection import GroupKFold
21 from sklearn.pipeline import Pipeline
22 from sklearn.preprocessing import StandardScaler
23 from sklearn.svm import SVC
24
25 try:
26     from xgboost import XGBClassifier
27
28     HAS_XGBOOST = True
29     XGBOOST_IMPORT_ERROR = ""
30 except ImportError as exc:
31     XGBClassifier = None
32     HAS_XGBOOST = False
33     XGBOOST_IMPORT_ERROR = str(exc)
34
35 try:
36     from sklearn.model_selection import StratifiedGroupKFold
37
38     HAS_STRATIFIED_GROUP_KFOLD = True
39     STRATIFIED_GROUP_KFOLD_ERROR = ""
40 except ImportError as exc:
41     StratifiedGroupKFold = None
42     HAS_STRATIFIED_GROUP_KFOLD = False
43     STRATIFIED_GROUP_KFOLD_ERROR = str(exc)
44
45
46 ROOT = Path(__file__).resolve().parents[1]
47 RESULTS_DIR = ROOT / "results"
48 OUTPUT_CSV_ENCODING = "utf-8-sig"
49 RANDOM_STATE = 42
50 N_SPLITS = 5
51 POS_LABEL = 1
52 NEG_LABEL = 0
53 THRESHOLD = 0.5
54
55
56 @dataclass(frozen=True)
57 class DatasetConfig:
58     key: str
59     display_name: str
60     path: Path
61     id_col: str
62     label_col: str
63     metadata_cols: tuple[str, ...]
64     expected_record_count_distribution: dict[int, int] | None = None
65
66
67 DATASET_CONFIGS = (
68     DatasetConfig(
69         key="dataset1",
70         display_name="Dataset 1",
71         path=RESULTS_DIR / "model_ready_dataset1_deduplicated.csv",
72         id_col="id",
73         label_col="class",
74         metadata_cols=("id", "gender", "class"),
75         expected_record_count_distribution={2: 1, 3: 251},
76     ),
77     DatasetConfig(
78         key="dataset2",
79         display_name="Dataset 2",
80         path=RESULTS_DIR / "model_ready_dataset2_prepared.csv",
81         id_col="ID",
82         label_col="Status",
83         metadata_cols=("ID", "Recording", "Status", "Gender"),
84     ),
85 )
86
87
88 METRIC_COLUMNS = (
89     "accuracy",
90     "balanced_accuracy",
```



```

91     "roc_auc",
92     "sensitivity",
93     "specificity",
94     "precision",
95     "recall",
96     "f1",
97 )
98
99
100 def rel(path: Path) -> str:
101     return path.relative_to(ROOT).as_posix()
102
103
104 def write_csv(df: pd.DataFrame, path: Path) -> None:
105     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
106
107
108 def load_dataset(config: DatasetConfig) -> pd.DataFrame:
109     if not config.path.exists():
110         raise FileNotFoundError(f"Missing input file: {config.path}")
111     df = pd.read_csv(config.path)
112     required = set(config.metadata_cols) | {config.id_col, config.label_col, "sex_male"}
113     missing = sorted(required - set(df.columns))
114     if missing:
115         raise ValueError(f"{config.display_name} is missing required columns: {missing}")
116     if df[config.label_col].isna().any():
117         raise ValueError(f"{config.display_name} has missing labels")
118     if df[config.id_col].isna().any():
119         raise ValueError(f"{config.display_name} has missing subject IDs")
120     if df.isna().sum().sum() > 0:
121         raise ValueError(f"{config.display_name} contains missing values")
122     label_unique_by_subject = df.groupby(config.id_col)[config.label_col].nunique(
123         dropna=False
124     )
125     if int(label_unique_by_subject.max()) != 1:
126         raise ValueError(f"{config.display_name} has subjects with inconsistent labels")
127     return df
128
129
130 def acoustic_columns(df: pd.DataFrame, config: DatasetConfig) -> list[str]:
131     excluded = set(config.metadata_cols) | {"sex_male"}
132     return [
133         column
134         for column in df.select_dtypes(include="number").columns
135         if column not in excluded
136     ]
137
138
139 def feature_sets(df: pd.DataFrame, config: DatasetConfig) -> dict[str, list[str]]:
140     acoustic = acoustic_columns(df, config)
141     if not acoustic:
142         raise ValueError(f"{config.display_name} has no acoustic feature columns")
143     return {
144         "acoustic_only": acoustic,
145         "acoustic_plus_sex": acoustic + ["sex_male"],
146     }
147
148
149 def make_models() -> dict[str, Pipeline]:
150     models = {
151         "Logistic Regression": Pipeline(
152             steps=[
153                 ("scaler", StandardScaler()),
154                 (
155                     "model",
156                     LogisticRegression(max_iter=5000, random_state=RANDOM_STATE),
157                 ),
158             ]
159         ),
160         "SVM-RBF": Pipeline(
161             steps=[
162                 ("scaler", StandardScaler()),
163                 (
164                     "model",
165                     SVC(
166                         kernel="rbf",
167                         probability=True,
168                         random_state=RANDOM_STATE,
169                     ),
170                 ),
171             ]
172         ),
173         "Random Forest": Pipeline(
174             steps=[
175                 (
176                     "model",
177                     RandomForestClassifier(
178                         n_estimators=500,
179                         random_state=RANDOM_STATE,
180                         n_jobs=-1,
181                     ),
182                 )

```

```

183     ]
184     ),
185 }
186 if HAS_XGBOOST:
187     models["XGBoost (supplemental)"] = Pipeline(
188         steps=[
189             (
190                 "model",
191                 XGBClassifier(
192                     n_estimators=120,
193                     max_depth=2,
194                     learning_rate=0.05,
195                     subsample=0.8,
196                     colsample_bytree=0.8,
197                     reg_alpha=0.1,
198                     reg_lambda=5.0,
199                     objective="binary:logistic",
200                     eval_metric="logloss",
201                     tree_method="hist",
202                     random_state=RANDOM_STATE,
203                     n_jobs=1,
204                 ),
205             )
206         ]
207     )
208     return models
209
210 def make_splitter() -> tuple[Any, str, str]:
211     if HAS_STRATIFIED_GROUP_KFOLD:
212         return (
213             StratifiedGroupKFold(
214                 n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE
215             ),
216             "StratifiedGroupKFold",
217             "",
218         )
219     return (
220         GroupKFold(n_splits=N_SPLITS),
221         "GroupKFold",
222         STRATIFIED_GROUP_KFOLD_ERROR,
223     )
224
225
226 def distribution(values: pd.Series | np.ndarray) -> dict[int, int]:
227     series = pd.Series(values).astype(int)
228     return {
229         int(label): int(count)
230         for label, count in series.value_counts().sort_index().items()
231     }
232
233
234 def subject_label_distribution(
235     df: pd.DataFrame, id_col: str, label_col: str
236 ) -> dict[int, int]:
237     labels = df.groupby(id_col, dropna=False)[label_col].first()
238     return distribution(labels)
239
240
241 def add_split_audit(
242     audit_rows: list[dict[str, Any]],
243     config: DatasetConfig,
244     df: pd.DataFrame,
245     train_idx: np.ndarray,
246     test_idx: np.ndarray,
247     fold: int,
248     splitter_name: str,
249     fallback_reason: str,
250 ) -> None:
251     train_df = df.iloc[train_idx]
252     test_df = df.iloc[test_idx]
253     train_subjects = set(train_df[config.id_col])
254     test_subjects = set(test_df[config.id_col])
255     overlap = train_subjects & test_subjects
256     if overlap:
257         raise ValueError(
258             f"{config.display_name} fold {fold} has subject leakage: {sorted(overlap)[:5]}"
259         )
260     audit_rows.append(
261         {
262             "dataset": config.key,
263             "dataset_display_name": config.display_name,
264             "fold": fold,
265             "splitter": splitter_name,
266             "fallback_reason": fallback_reason,
267             "train_subject_count": len(train_subjects),
268             "test_subject_count": len(test_subjects),
269             "train_record_count": len(train_df),
270             "test_record_count": len(test_df),
271             "train_record_class_distribution": distribution(train_df[config.label_col]),
272             "test_record_class_distribution": distribution(test_df[config.label_col]),
273             "train_subject_class_distribution": subject_label_distribution(

```

```

275         train_df, config.id_col, config.label_col
276     ),
277     "test_subject_class_distribution": subject_label_distribution(
278         test_df, config.id_col, config.label_col
279     ),
280     "overlap_subject_count": len(overlap),
281 }
282 )
283
284
285 def positive_probability(model: Pipeline, x_test: pd.DataFrame) -> np.ndarray:
286     if hasattr(model, "predict_proba"):
287         classes = list(model.classes_)
288         if POS_LABEL not in classes:
289             raise ValueError(f"Positive label {POS_LABEL} not found in model classes")
290         return model.predict_proba(x_test)[:, classes.index(POS_LABEL)]
291     if hasattr(model, "decision_function"):
292         scores = model.decision_function(x_test)
293         return 1.0 / (1.0 + np.exp(-scores))
294     raise ValueError("Model does not expose predict_proba or decision_function")
295
296
297 def compute_metrics(
298     y_true: pd.Series | np.ndarray, y_prob: np.ndarray
299 ) -> tuple[dict[str, float], dict[str, int]]:
300     y_true_array = np.asarray(y_true).astype(int)
301     y_pred = (np.asarray(y_prob) >= THRESHOLD).astype(int)
302     tn, fp, fn, tp = confusion_matrix(
303         y_true_array, y_pred, labels=[NEG_LABEL, POS_LABEL]
304     ).ravel()
305     roc_auc = (
306         float(roc_auc_score(y_true_array, y_prob))
307         if len(np.unique(y_true_array)) == 2
308         else np.nan
309     )
310     metrics = {
311         "accuracy": float(accuracy_score(y_true_array, y_pred)),
312         "balanced_accuracy": float(balanced_accuracy_score(y_true_array, y_pred)),
313         "roc_auc": roc_auc,
314         "sensitivity": float(tp / (tp + fn)) if (tp + fn) else np.nan,
315         "specificity": float(tn / (tn + fp)) if (tn + fp) else np.nan,
316         "precision": float(
317             precision_score(y_true_array, y_pred, pos_label=POS_LABEL, zero_division=0)
318         ),
319         "recall": float(
320             recall_score(y_true_array, y_pred, pos_label=POS_LABEL, zero_division=0)
321         ),
322         "f1": float(f1_score(y_true_array, y_pred, pos_label=POS_LABEL, zero_division=0)),
323     }
324     matrix = {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)}
325     return metrics, matrix
326
327
328 def subject_level_predictions(
329     df_test: pd.DataFrame,
330     config: DatasetConfig,
331     record_prob: np.ndarray,
332 ) -> pd.DataFrame:
333     pred_df = pd.DataFrame(
334         {
335             "subject_id": df_test[config.id_col].to_numpy(),
336             "y_true": df_test[config.label_col].astype(int).to_numpy(),
337             "y_prob": record_prob,
338         }
339     )
340     subject_df = (
341         pred_df.groupby("subject_id", dropna=False)
342         .agg(y_true=("y_true", "first"), y_prob=("y_prob", "mean"))
343         .reset_index()
344     )
345     label_counts = pred_df.groupby("subject_id", dropna=False)["y_true"].nunique(
346         dropna=False
347     )
348     if int(label_counts.max()) != 1:
349         raise ValueError(f"{config.display_name} test fold has inconsistent labels")
350     return subject_df
351
352
353 def metric_row(
354     config: DatasetConfig,
355     feature_set: str,
356     model_name: str,
357     fold: int | str,
358     summary: str,
359     n_records: int,
360     n_subjects: int,
361     metrics: dict[str, float],
362 ) -> dict[str, Any]:
363     row: dict[str, Any] = {
364         "dataset": config.key,
365         "dataset_display_name": config.display_name,
366         "feature_set": feature_set,

```

```

367         "model": model_name,
368         "fold": fold,
369         "summary": summary,
370         "n_records": n_records,
371         "n_subjects": n_subjects,
372     }
373     row.update(metrics)
374     return row
375
376
377 def confusion_row(
378     config: DatasetConfig,
379     feature_set: str,
380     model_name: str,
381     level: str,
382     fold: int | str,
383     summary: str,
384     matrix: dict[str, int],
385 ) -> dict[str, Any]:
386     return {
387         "dataset": config.key,
388         "dataset_display_name": config.display_name,
389         "feature_set": feature_set,
390         "model": model_name,
391         "level": level,
392         "fold": fold,
393         "summary": summary,
394         **matrix,
395     }
396
397
398 def append_summary_rows(metrics_df: pd.DataFrame) -> pd.DataFrame:
399     group_cols = ["dataset", "dataset_display_name", "feature_set", "model"]
400     summary_rows: list[dict[str, Any]] = []
401     fold_rows = metrics_df[metrics_df["summary"] == "fold"]
402     for values, group in fold_rows.groupby(group_cols, dropna=False):
403         base = dict(zip(group_cols, values, strict=True))
404         for summary, values_by_metric in (
405             ("mean", group[list(METRIC_COLUMNS)].mean(numeric_only=True)),
406             ("std", group[list(METRIC_COLUMNS)].std(ddof=1, numeric_only=True)),
407         ):
408             row = {
409                 **base,
410                 "fold": "",
411                 "summary": summary,
412                 "n_records": "",
413                 "n_subjects": "",
414             }
415             row.update({metric: values_by_metric[metric] for metric in METRIC_COLUMNS})
416             summary_rows.append(row)
417     return pd.concat([metrics_df, pd.DataFrame(summary_rows)], ignore_index=True)
418
419
420 def append_confusion_summary_rows(confusion_df: pd.DataFrame) -> pd.DataFrame:
421     group_cols = ["dataset", "dataset_display_name", "feature_set", "model", "level"]
422     matrix_cols = ["tn", "fp", "fn", "tp"]
423     summary_rows: list[dict[str, Any]] = []
424     fold_rows = confusion_df[confusion_df["summary"] == "fold"]
425     for values, group in fold_rows.groupby(group_cols, dropna=False):
426         row = dict(zip(group_cols, values, strict=True))
427         row.update(
428             {
429                 "fold": "",
430                 "summary": "aggregate",
431                 **{
432                     column: int(group[column].sum())
433                     for column in matrix_cols
434                 },
435             }
436         )
437         summary_rows.append(row)
438     return pd.concat([confusion_df, pd.DataFrame(summary_rows)], ignore_index=True)
439
440
441 def markdown_table(df: pd.DataFrame, floatfmt: str = ".4f") -> str:
442     headers = list(df.columns)
443     rows = []
444     for _, row in df.iterrows():
445         values = []
446         for column in headers:
447             value = row[column]
448             if isinstance(value, float) and not pd.isna(value):
449                 values.append(format(value, floatfmt))
450             elif pd.isna(value):
451                 values.append("")
452             else:
453                 values.append(str(value))
454         rows.append(values)
455
456     table = [
457         "| " + " | ".join(headers) + " |",
458         "| " + " | ".join(["---"] * len(headers)) + " |",

```

```

459     ]
460     table.extend("| " + " | ".join(row) + " |" for row in rows)
461     return "\n".join(table)
462
463
464 def run_dataset(
465     config: DatasetConfig,
466     record_metric_rows: list[dict[str, Any]],
467     subject_metric_rows: list[dict[str, Any]],
468     audit_rows: list[dict[str, Any]],
469     confusion_rows: list[dict[str, Any]],
470     report_lines: list[str],
471 ) -> None:
472     df = load_dataset(config)
473     features_by_name = feature_sets(df, config)
474     models = make_models()
475     splitter, splitter_name, fallback_reason = make_splitter()
476
477     y = df[config.label_col].astype(int)
478     groups = df[config.id_col]
479     splits = list(splitter.split(df, y, groups))
480     if len(splits) != N_SPLITS:
481         raise ValueError(f"{config.display_name} expected {N_SPLITS} folds")
482
483     record_count_distribution = (
484         df.groupby(config.id_col).size().value_counts().sort_index().to_dict()
485     )
486     if config.expected_record_count_distribution is not None:
487         expected = config.expected_record_count_distribution
488         if record_count_distribution != expected:
489             raise ValueError(
490                 f"{config.display_name} record count distribution changed: "
491                 f"{record_count_distribution}, expected {expected}"
492             )
493
494     report_lines.extend(
495         [
496             f"## {config.display_name}",
497             "",
498             f"- Input: `{rel(config.path)}`",
499             f"- Splitter: `{splitter_name}`",
500             f"- Subjects: `{df[config.id_col].nunique()}`",
501             f"- Records: `{len(df)}`",
502             f"- Subject record count distribution: `{record_count_distribution}`",
503             f"- Record-level class distribution: `{distribution(df[config.label_col])}`",
504             f"- Subject-level class distribution: "
505             f"`{subject_label_distribution(df, config.id_col, config.label_col)}`",
506             f"- Acoustic feature count: `{len(features_by_name['acoustic_only'])}`",
507             "- Feature sets: `acoustic_only` is the main analysis; "
508             "`acoustic_plus_sex` is supplemental.",
509         ]
510     )
511     if HAS_XGBOOST:
512         report_lines.append(
513             "- XGBoost is included as a supplemental baseline and does not replace "
514             "the primary baseline interpretation."
515         )
516     else:
517         report_lines.append(
518             f"- XGBoost supplemental baseline was skipped because `xgboost` is unavailable: `{XGBOOST_IMPORT_ERROR}`."
519         )
520     if config.key == "dataset2":
521         report_lines.append(
522             "- Dataset 2 prepared file only contains metadata cleanup and sex_male "
523             "encoding; acoustic features are not globally z-score standardized before "
524             "cross-validation. Scaling for Logistic Regression and SVM-RBF is "
525             "performed only within sklearn Pipeline in each training fold."
526         )
527     report_lines.extend(["", "### Fold audit", ""])
528     if fallback_reason:
529         report_lines.extend(
530             [
531                 f"- StratifiedGroupKFold unavailable; fallback reason: `{fallback_reason}`",
532                 "",
533             ]
534         )
535
536     for fold, (train_idx, test_idx) in enumerate(splits, start=1):
537         add_split_audit(
538             audit_rows,
539             config,
540             df,
541             train_idx,
542             test_idx,
543             fold,
544             splitter_name,
545             fallback_reason,
546         )
547         train_df = df.iloc[train_idx]
548         test_df = df.iloc[test_idx]
549         report_lines.append(
550             f"- Fold {fold}: train subjects `{train_df[config.id_col].nunique()}`, "

```

```

551         f"test subjects `{test_df[config.id_col].nunique()}`", "
552         f"train records `{len(train_df)}`", test records `{len(test_df)}`", "
553         f"test subject classes "
554         f"`${subject_label_distribution(test_df, config.id_col, config.label_col)}`"
555     )
556
557     for feature_set_name, columns in features_by_name.items():
558         x_train = train_df[columns]
559         x_test = test_df[columns]
560         y_train = train_df[config.label_col].astype(int)
561         y_test = test_df[config.label_col].astype(int)
562
563         for model_name, model in models.items():
564             model.fit(x_train, y_train)
565             record_prob = positive_probability(model, x_test)
566
567             record_metrics, record_matrix = compute_metrics(y_test, record_prob)
568             record_metric_rows.append(
569                 metric_row(
570                     config,
571                     feature_set_name,
572                     model_name,
573                     fold,
574                     "fold",
575                     len(test_df),
576                     test_df[config.id_col].nunique(),
577                     record_metrics,
578                 )
579             )
580             confusion_rows.append(
581                 confusion_row(
582                     config,
583                     feature_set_name,
584                     model_name,
585                     "record",
586                     fold,
587                     "fold",
588                     record_matrix,
589                 )
590             )
591
592             subject_df = subject_level_predictions(test_df, config, record_prob)
593             subject_metrics, subject_matrix = compute_metrics(
594                 subject_df["y_true"], subject_df["y_prob"].to_numpy()
595             )
596             subject_metric_rows.append(
597                 metric_row(
598                     config,
599                     feature_set_name,
600                     model_name,
601                     fold,
602                     "fold",
603                     len(test_df),
604                     len(subject_df),
605                     subject_metrics,
606                 )
607             )
608             confusion_rows.append(
609                 confusion_row(
610                     config,
611                     feature_set_name,
612                     model_name,
613                     "subject",
614                     fold,
615                     "fold",
616                     subject_matrix,
617                 )
618             )
619         report_lines.append("")
620
621
622     def write_report(
623         report_lines: list[str],
624         record_metrics: pd.DataFrame,
625         subject_metrics: pd.DataFrame,
626     ) -> None:
627         report_lines.extend(
628             [
629                 "## Main subject-level results",
630                 "",
631                 "The primary model comparison uses `acoustic_only` subject-level mean metrics.",
632                 "",
633             ]
634         )
635         main = subject_metrics[
636             (subject_metrics["summary"] == "mean")
637             & (subject_metrics["feature_set"] == "acoustic_only")
638         ].copy()
639         display_cols = [
640             "dataset_display_name",
641             "model",
642             "accuracy",

```

```

643         "balanced_accuracy",
644         "roc_auc",
645         "sensitivity",
646         "specificity",
647         "precision",
648         "recall",
649         "f1",
650     ]
651     report_lines.append(markdown_table(main[display_cols], floatfmt=".4f"))
652     report_lines.extend(
653         [
654             "",
655             "## Output files",
656             "",
657             "- `results/stage3_baseline_metrics_record_level.csv`",
658             "- `results/stage3_baseline_metrics_subject_level.csv`",
659             "- `results/stage3_fold_split_audit.csv`",
660             "- `results/stage3_confusion_matrices.csv`",
661             "- `results/stage3_baseline_report.md`",
662             "",
663             "## Constraints",
664             "",
665             "- No row-wise random `train_test_split` was used.",
666             "- All train/test splits are grouped by subject ID.",
667             "- Standardization is only applied inside sklearn `Pipeline` objects.",
668             "- No SMOTE, PCA, feature selection, SHAP, clustering, or large grid search was used.",
669             "- XGBoost is reported as a supplemental baseline only.",
670             "- `sex_male` is excluded from the main `acoustic_only` models.",
671             "",
672         ]
673     )
674     (RESULTS_DIR / "stage3_baseline_report.md").write_text(
675         "\n".join(report_lines), encoding="utf-8"
676     )
677
678 def main() -> None:
679     RESULTS_DIR.mkdir(exist_ok=True)
680     record_metric_rows: list[dict[str, Any]] = []
681     subject_metric_rows: list[dict[str, Any]] = []
682     audit_rows: list[dict[str, Any]] = []
683     confusion_rows: list[dict[str, Any]] = []
684     report_lines = [
685         "# Stage 3 Baseline PD/Healthy Classification Report",
686         "",
687         "This report is generated by `src/stage3_baseline_models.py`.",
688         "",
689         "Positive class: `1` = PD; negative class: `0` = Healthy.",
690         "",
691         "Models: Logistic Regression, SVM-RBF, Random Forest, and optional XGBoost (supplemental).",
692         "",
693     ]
694
695     for config in DATASET_CONFIGS:
696         run_dataset(
697             config,
698             record_metric_rows,
699             subject_metric_rows,
700             audit_rows,
701             confusion_rows,
702             report_lines,
703         )
704
705     record_metrics = append_summary_rows(pd.DataFrame(record_metric_rows))
706     subject_metrics = append_summary_rows(pd.DataFrame(subject_metric_rows))
707     audit = pd.DataFrame(audit_rows)
708     confusion = append_confusion_summary_rows(pd.DataFrame(confusion_rows))
709
710     write_csv(
711         record_metrics, RESULTS_DIR / "stage3_baseline_metrics_record_level.csv"
712     )
713     write_csv(
714         subject_metrics, RESULTS_DIR / "stage3_baseline_metrics_subject_level.csv"
715     )
716     write_csv(audit, RESULTS_DIR / "stage3_fold_split_audit.csv")
717     write_csv(confusion, RESULTS_DIR / "stage3_confusion_matrices.csv")
718     write_report(report_lines, record_metrics, subject_metrics)
719
720 if __name__ == "__main__":
721     main()

```

A.3 代码三：判别性声学特征识别与 XGBoost/SHAP 解释性分析

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Any
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import mannwhitneyu, ttest_ind
14 from sklearn.ensemble import ExtraTreesClassifier
15 from sklearn.metrics import balanced_accuracy_score
16 from sklearn.linear_model import LogisticRegressionCV
17 from sklearn.model_selection import StratifiedKFold
18 from sklearn.pipeline import Pipeline
19 from sklearn.preprocessing import StandardScaler
20
21 try:
22     import shap
23     from xgboost import XGBClassifier
24
25     HAS_XGBOOST_SHAP = True
26     XGBOOST_SHAP_IMPORT_ERROR = ""
27 except ImportError as exc:
28     shap = None
29     XGBClassifier = None
30     HAS_XGBOOST_SHAP = False
31     XGBOOST_SHAP_IMPORT_ERROR = str(exc)
32
33
34 ROOT = Path(__file__).resolve().parents[1]
35 RESULTS_DIR = ROOT / "results"
36 FIGURES_DIR = RESULTS_DIR / "figures"
37 DATASET1_PATH = RESULTS_DIR / "model_ready_dataset1_deduplicated.csv"
38 DATASET2_REFERENCE_PATH = RESULTS_DIR / "model_ready_dataset2_prepared.csv"
39 OUTPUT_CSV_ENCODING = "utf-8-sig"
40
41 RANDOM_STATE = 42
42 N_SPLITS = 5
43 POS_LABEL = 1
44 NEG_LABEL = 0
45 NEAR_CONSTANT_STD_THRESHOLD = 1e-12
46 L1_CS = [0.01, 0.03, 0.1, 0.3, 1.0]
47 COEF_EPSILON = 1e-8
48 N_ESTIMATORS = 500
49 PERMUTATION_REPEATS = 1
50 XGBOOST_N_ESTIMATORS = 120
51
52 METADATA_COLUMNS = {"id", "gender", "class", "sex_male"}
53
54
55 def rel(path: Path) -> str:
56     return path.relative_to(ROOT).as_posix()
57
58
59 def write_csv(df: pd.DataFrame, path: Path) -> None:
60     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
61
62
63 def load_dataset1() -> pd.DataFrame:
64     if not DATASET1_PATH.exists():
65         raise FileNotFoundError(f"Missing input file: {DATASET1_PATH}")
66     df = pd.read_csv(DATASET1_PATH)
67     required = {"id", "gender", "class", "sex_male"}
68     missing = sorted(required - set(df.columns))
69     if missing:
70         raise ValueError(f"Dataset 1 is missing required columns: {missing}")
71     if df.isna().sum().sum() > 0:
72         raise ValueError("Dataset 1 contains missing values before aggregation")
73     if int(df.groupby("id")["class"].nunique(dropna=False).max()) != 1:
74         raise ValueError("Dataset 1 has subjects with inconsistent class labels")
75     if int(df.groupby("id")["sex_male"].nunique(dropna=False).max()) != 1:
76         raise ValueError("Dataset 1 has subjects with inconsistent sex_male values")
77     return df
78
79
80 def acoustic_columns(df: pd.DataFrame) -> list[str]:
81     columns = [
82         column
83         for column in df.select_dtypes(include="number").columns
84         if column not in METADATA_COLUMNS
85     ]
86     if len(columns) != 752:
87         raise ValueError(f"Expected 752 acoustic features, found {len(columns)}")
88     forbidden = METADATA_COLUMNS & set(columns)
89     if forbidden:
90         raise ValueError(f"Forbidden metadata columns in acoustic features: {forbidden}")
```



```

91     return columns
92
93
94 def build_subject_table(df: pd.DataFrame, features: list[str]) -> pd.DataFrame:
95     feature_means = df.groupby("id", dropna=False)[features].mean().reset_index()
96     subject_meta = (
97         df.groupby("id", dropna=False)
98         .agg(
99             class_label=("class", "first"),
100             sex_male=("sex_male", "first"),
101             record_count=("id", "size"),
102         )
103         .reset_index()
104         .rename(columns={"class_label": "class"})
105     )
106     subject_table = subject_meta.merge(feature_means, on="id", how="left")
107     if subject_table.shape[0] != 252:
108         raise ValueError(f"Expected 252 subjects, found {subject_table.shape[0]}")
109     return subject_table
110
111
112 def audit_subject_table(
113     subject_table: pd.DataFrame, features: list[str]
114 ) -> dict[str, Any]:
115     feature_audit = build_feature_audit_table(subject_table, features)
116     return {
117         "subject_count": int(subject_table.shape[0]),
118         "class_distribution": {
119             int(k): int(v)
120             for k, v in subject_table["class"].value_counts().sort_index().items()
121         },
122         "acoustic_feature_count": len(features),
123         "missing_value_count": int(subject_table.isna().sum().sum()),
124         "constant_feature_count": int(feature_audit["is_constant"].sum()),
125         "near_constant_feature_count": int(
126             feature_audit["is_near_constant"].sum()
127         ),
128         "near_constant_features": feature_audit.loc[
129             feature_audit["is_near_constant"], "feature"
130         ].tolist(),
131     }
132
133
134 def build_feature_audit_table(
135     subject_table: pd.DataFrame, features: list[str]
136 ) -> pd.DataFrame:
137     feature_values = subject_table[features]
138     stds = feature_values.std(ddof=0)
139     unique_counts = feature_values.nunique(dropna=False)
140     missing_counts = feature_values.isna().sum()
141     audit = pd.DataFrame(
142         {
143             "feature": features,
144             "category": [categorize_feature(feature) for feature in features],
145             "subject_level_std": stds.reindex(features).to_numpy(),
146             "subject_level_unique_count": unique_counts.reindex(features).to_numpy(),
147             "subject_level_missing_count": missing_counts.reindex(features).to_numpy(),
148         }
149     )
150     audit["is_constant"] = audit["subject_level_unique_count"] <= 1
151     audit["is_near_constant"] = (
152         audit["subject_level_std"] <= NEAR_CONSTANT_STD_THRESHOLD
153     )
154     return audit
155
156
157 def merge_feature_audit(df: pd.DataFrame, feature_audit: pd.DataFrame) -> pd.DataFrame:
158     audit_cols = [
159         "feature",
160         "subject_level_std",
161         "subject_level_unique_count",
162         "subject_level_missing_count",
163         "is_constant",
164         "is_near_constant",
165     ]
166     return df.merge(feature_audit[audit_cols], on="feature", how="left")
167
168
169 def add_q_value_display_columns(df: pd.DataFrame) -> pd.DataFrame:
170     out = df.copy()
171     for column in ["q_mannwhitney", "q_welch"]:
172         if column in out.columns:
173             out[f"{column}_scientific"] = out[column].map(
174                 lambda value: "" if pd.isna(value) else f"{value:.3e}"
175             )
176     return out
177
178
179 def fdr_bh(p_values: np.ndarray) -> np.ndarray:
180     p = np.asarray(p_values, dtype=float)
181     adjusted = np.full(p.shape, np.nan, dtype=float)
182     valid = np.isfinite(p)

```

```

183     if not valid.any():
184         return adjusted
185     valid_p = p[valid]
186     order = np.argsort(valid_p)
187     ordered_p = valid_p[order]
188     n = len(ordered_p)
189     ordered_q = ordered_p * n / np.arange(1, n + 1)
190     ordered_q = np.minimum.accumulate(ordered_q[::-1])[::-1]
191     ordered_q = np.clip(ordered_q, 0, 1)
192     q = np.empty_like(ordered_q)
193     q[order] = ordered_q
194     adjusted[valid] = q
195     return adjusted
196
197
198 def cohens_d(pd_values: np.ndarray, healthy_values: np.ndarray) -> float:
199     n_pd = len(pd_values)
200     n_healthy = len(healthy_values)
201     if n_pd < 2 or n_healthy < 2:
202         return np.nan
203     var_pd = np.var(pd_values, ddof=1)
204     var_healthy = np.var(healthy_values, ddof=1)
205     pooled = ((n_pd - 1) * var_pd + (n_healthy - 1) * var_healthy) / (
206         n_pd + n_healthy - 2
207     )
208     if pooled <= 0:
209         return 0.0
210     return float((np.mean(pd_values) - np.mean(healthy_values)) / np.sqrt(pooled))
211
212
213 def cliffs_delta(pd_values: np.ndarray, healthy_values: np.ndarray) -> float:
214     comparisons = pd_values[:, None] - healthy_values[None, :]
215     greater = np.sum(comparisons > 0)
216     lesser = np.sum(comparisons < 0)
217     return float((greater - lesser) / comparisons.size)
218
219
220 def run_univariate_statistics(
221     subject_table: pd.DataFrame, features: list[str]
222 ) -> pd.DataFrame:
223     y = subject_table["class"].astype(int).to_numpy()
224     rows: list[dict[str, Any]] = []
225     for feature in features:
226         values = subject_table[feature].to_numpy(dtype=float)
227         pd_values = values[y == POS_LABEL]
228         healthy_values = values[y == NEG_LABEL]
229         try:
230             u_stat, mw_p = mannwhitneyu(
231                 pd_values, healthy_values, alternative="two-sided"
232             )
233         except ValueError:
234             u_stat, mw_p = np.nan, np.nan
235         try:
236             t_stat, t_p = ttest_ind(
237                 pd_values, healthy_values, equal_var=False, nan_policy="omit"
238             )
239         except ValueError:
240             t_stat, t_p = np.nan, np.nan
241         d = cohens_d(pd_values, healthy_values)
242         delta = cliffs_delta(pd_values, healthy_values)
243         rows.append(
244             {
245                 "feature": feature,
246                 "category": categorize_feature(feature),
247                 "mean_pd": float(np.mean(pd_values)),
248                 "mean_healthy": float(np.mean(healthy_values)),
249                 "median_pd": float(np.median(pd_values)),
250                 "median_healthy": float(np.median(healthy_values)),
251                 "mannwhitney_u": float(u_stat),
252                 "p_mannwhitney": float(mw_p),
253                 "welch_t": float(t_stat),
254                 "p_welch": float(t_p),
255                 "cohens_d": d,
256                 "abs_cohens_d": abs(d),
257                 "cliffs_delta": delta,
258                 "abs_cliffs_delta": abs(delta),
259             }
260         )
261     stats = pd.DataFrame(rows)
262     stats["q_mannwhitney"] = fdr_bh(stats["p_mannwhitney"].to_numpy())
263     stats["q_welch"] = fdr_bh(stats["p_welch"].to_numpy())
264     return stats.sort_values(["q_mannwhitney", "p_mannwhitney"], na_position="last")
265
266
267 def stratified_folds(y: np.ndarray) -> StratifiedKFold:
268     return StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
269
270
271 def run_li_stability(subject_table: pd.DataFrame, features: list[str]) -> pd.DataFrame:
272     x = subject_table[features]
273     y = subject_table["class"].astype(int).to_numpy()
274     selected_counts = pd.Series(0, index=features, dtype=int)

```

```

275 abs_coef_sums = pd.Series(0.0, index=features, dtype=float)
276
277 for fold, (train_idx, _) in enumerate(stratified_folds(y).split(x, y), start=1):
278     pipeline = Pipeline(
279         steps=[
280             ("scaler", StandardScaler()),
281             (
282                 "model",
283                 LogisticRegressionCV(
284                     Cs=L1_CS,
285                     cv=3,
286                     penalty="l1",
287                     solver="liblinear",
288                     scoring="balanced_accuracy",
289                     max_iter=5000,
290                     random_state=RANDOM_STATE + fold,
291                     n_jobs=1,
292                 ),
293             ),
294         ]
295     )
296     pipeline.fit(x.iloc[train_idx], y[train_idx])
297     coefs = np.abs(pipeline.named_steps["model"].coef_.ravel())
298     selected = coefs > COEF_EPSILON
299     selected_counts += selected.astype(int)
300     abs_coef_sums += coefs
301
302 out = pd.DataFrame(
303     {
304         "feature": features,
305         "category": [categorize_feature(feature) for feature in features],
306         "selected_folds": selected_counts.to_numpy(),
307         "selection_frequency": selected_counts.to_numpy() / N_SPLITS,
308         "mean_abs_l1_coefficient": abs_coef_sums.to_numpy() / N_SPLITS,
309     }
310 )
311 return out.sort_values(
312     ["selection_frequency", "mean_abs_l1_coefficient"],
313     ascending=[False, False],
314 )
315
316
317 def run_tree_and_permutation(
318     subject_table: pd.DataFrame, features: list[str]
319 ) -> tuple[pd.DataFrame, pd.DataFrame]:
320     x = subject_table[features].to_numpy(dtype=float)
321     y = subject_table["class"].astype(int).to_numpy()
322     tree_fold_importances: list[np.ndarray] = []
323     permutation_fold_importances: list[np.ndarray] = []
324     permutation_fold_stds: list[np.ndarray] = []
325
326     for fold, (train_idx, test_idx) in enumerate(stratified_folds(y).split(x, y), start=1):
327         model = ExtraTreesClassifier(
328             n_estimators=N_ESTIMATORS,
329             random_state=RANDOM_STATE + fold,
330             n_jobs=1,
331         )
332         model.fit(x[train_idx], y[train_idx])
333         tree_fold_importances.append(model.feature_importances_)
334
335         permutation_mean, permutation_std = heldout_permutation_importance(
336             model,
337             x[test_idx],
338             y[test_idx],
339             random_state=RANDOM_STATE + fold,
340         )
341         permutation_fold_importances.append(permutation_mean)
342         permutation_fold_stds.append(permutation_std)
343
344     tree_array = np.vstack(tree_fold_importances)
345     permutation_array = np.vstack(permutation_fold_importances)
346     permutation_std_array = np.vstack(permutation_fold_stds)
347
348     tree_df = pd.DataFrame(
349         {
350             "feature": features,
351             "category": [categorize_feature(feature) for feature in features],
352             "tree_importance_mean": tree_array.mean(axis=0),
353             "tree_importance_std": tree_array.std(axis=0, ddof=1),
354         }
355     ).sort_values("tree_importance_mean", ascending=False)
356
357     permutation_df = pd.DataFrame(
358         {
359             "feature": features,
360             "category": [categorize_feature(feature) for feature in features],
361             "permutation_importance_mean": permutation_array.mean(axis=0),
362             "permutation_importance_std_across_folds": permutation_array.std(
363                 axis=0, ddof=1
364             ),
365             "mean_within_fold_permutation_std": permutation_std_array.mean(axis=0),
366         }

```

```

367     ).sort_values("permutation_importance_mean", ascending=False)
368     return tree_df, permutation_df
369
370
371 def heldout_permutation_importance(
372     model: ExtraTreesClassifier,
373     x_test: np.ndarray,
374     y_test: np.ndarray,
375     random_state: int,
376 ) -> tuple[np.ndarray, np.ndarray]:
377     rng = np.random.default_rng(random_state)
378     baseline = balanced_accuracy_score(y_test, model.predict(x_test))
379     importances = np.zeros((x_test.shape[1], PERMUTATION_REPEATS), dtype=float)
380     for feature_index in range(x_test.shape[1]):
381         for repeat in range(PERMUTATION_REPEATS):
382             x_permuted = x_test.copy()
383             x_permuted[:, feature_index] = rng.permutation(x_permuted[:, feature_index])
384             score = balanced_accuracy_score(y_test, model.predict(x_permuted))
385             importances[feature_index, repeat] = baseline - score
386     ddof = 1 if PERMUTATION_REPEATS > 1 else 0
387     return importances.mean(axis=1), importances.std(axis=1, ddof=ddof)
388
389
390 def make_xgboost_model(fold: int) -> XGBClassifier:
391     return XGBClassifier(
392         n_estimators=XGBOOST_N_ESTIMATORS,
393         max_depth=2,
394         learning_rate=0.05,
395         subsample=0.8,
396         colsample_bytree=0.8,
397         reg_alpha=0.1,
398         reg_lambda=5.0,
399         objective="binary:logistic",
400         eval_metric="logloss",
401         tree_method="hist",
402         random_state=RANDOM_STATE + fold,
403         n_jobs=1,
404     )
405
406
407 def positive_class_shap_values(values: Any, n_features: int) -> np.ndarray:
408     if hasattr(values, "values"):
409         values = values.values
410     if isinstance(values, list):
411         values = values[POS_LABEL] if len(values) > POS_LABEL else values[0]
412     array = np.asarray(values)
413     if array.ndim == 3:
414         if array.shape[2] == n_features:
415             array = array[POS_LABEL] if array.shape[0] > POS_LABEL else array[0]
416         else:
417             array = array[:, :, POS_LABEL] if array.shape[2] > POS_LABEL else array[:, :, 0]
418     if array.ndim != 2 or array.shape[1] != n_features:
419         raise ValueError(f"Unexpected SHAP values shape: {array.shape}")
420     return array
421
422
423 def run_xgboost_and_shap(
424     subject_table: pd.DataFrame, features: list[str]
425 ) -> tuple[pd.DataFrame, pd.DataFrame]:
426     if not HAS_XGBOOST_SHAP:
427         skipped = pd.DataFrame(
428             {
429                 "feature": features,
430                 "category": [categorize_feature(feature) for feature in features],
431                 "xgboost_importance_mean": np.nan,
432                 "xgboost_importance_std": np.nan,
433                 "xgboost_import_error": XGBOOST_SHAP_IMPORT_ERROR,
434             }
435         )
436     shap_skipped = pd.DataFrame(
437         {
438             "feature": features,
439             "category": [categorize_feature(feature) for feature in features],
440             "shap_mean_abs_mean": np.nan,
441             "shap_mean_abs_std": np.nan,
442             "shap_import_error": XGBOOST_SHAP_IMPORT_ERROR,
443         }
444     )
445     return skipped, shap_skipped
446
447 x = subject_table[features]
448 y = subject_table["class"].astype(int).to_numpy()
449 xgb_fold_importances: list[np.ndarray] = []
450 shap_fold_importances: list[np.ndarray] = []
451
452 for fold, (train_idx, test_idx) in enumerate(stratified_folds(y).split(x, y), start=1):
453     model = make_xgboost_model(fold)
454     model.fit(x.iloc[train_idx], y[train_idx])
455     xgb_fold_importances.append(model.feature_importances_)
456
457     explainer = shap.TreeExplainer(model)
458     shap_values = positive_class_shap_values(

```

```

459         explainer.shap_values(x.iloc[test_idx]),
460         len(features),
461     )
462     shap_fold_importances.append(np.abs(shap_values).mean(axis=0))
463
464 xgb_array = np.vstack(xgb_fold_importances)
465 shap_array = np.vstack(shap_fold_importances)
466 xgb = pd.DataFrame(
467     {
468         "feature": features,
469         "category": [categorize_feature(feature) for feature in features],
470         "xgboost_importance_mean": xgb_array.mean(axis=0),
471         "xgboost_importance_std": xgb_array.std(axis=0, ddof=1),
472     }
473 ).sort_values("xgboost_importance_mean", ascending=False)
474 xgb["rank_xgboost_importance"] = rank_desc(xgb["xgboost_importance_mean"])
475
476 shap_importance = pd.DataFrame(
477     {
478         "feature": features,
479         "category": [categorize_feature(feature) for feature in features],
480         "shap_mean_abs_mean": shap_array.mean(axis=0),
481         "shap_mean_abs_std": shap_array.std(axis=0, ddof=1),
482     }
483 ).sort_values("shap_mean_abs_mean", ascending=False)
484 shap_importance["rank_shap_mean_abs"] = rank_desc(
485     shap_importance["shap_mean_abs_mean"]
486 )
487 return xgb, shap_importance
488
489 def categorize_feature(feature: str) -> str:
490     name = feature.lower()
491     if "tqwt" in name:
492         return "TQWT"
493     if any(token in name for token in ["wavelet", "_lt_", "ed_", "ed2_", "entropy"]):
494         return "Wavelet"
495     if "jitter" in name:
496         return "Jitter"
497     if "shimmer" in name or "shim_" in name or "apq" in name:
498         return "Shimmer"
499     if "hnr" in name or "harmonicity" in name or "harmtonoise" in name:
500         return "HNR / Harmonicity"
501     if "intensity" in name:
502         return "Intensity"
503     if (
504         name.startswith("f")
505         and len(name) == 2
506         and name[1].isdigit()
507         or name.startswith("b")
508         and len(name) == 2
509         and name[1].isdigit()
510     ):
511         return "Formant / Bandwidth"
512     if "mfcc" in name:
513         return "MFCC"
514     if "delta_delta" in name or "delta" in name:
515         return "Delta / Delta-delta"
516     if any(token in name for token in ["rpde", "dfa", "ppe", "gne"]):
517         return "Nonlinear"
518     return "Other"
519
520
521 def rank_desc(series: pd.Series) -> pd.Series:
522     return series.rank(ascending=False, method="average", na_option="bottom")
523
524
525 def rank_asc(series: pd.Series) -> pd.Series:
526     return series.rank(ascending=True, method="average", na_option="bottom")
527
528
529 def build_rank_summary(
530     univariate: pd.DataFrame,
531     li: pd.DataFrame,
532     tree: pd.DataFrame,
533     permutation: pd.DataFrame,
534     xgboost_importance: pd.DataFrame | None = None,
535     shap_importance: pd.DataFrame | None = None,
536 ) -> pd.DataFrame:
537     summary = (
538         univariate[
539             [
540                 "feature",
541                 "category",
542                 "p_mannwhitney",
543                 "q_mannwhitney",
544                 "cohens_d",
545                 "abs_cohens_d",
546                 "cliffs_delta",
547                 "abs_cliffs_delta",
548                 "q_mannwhitney_scientific",
549                 "subject_level_std",

```

```

551         "subject_level_unique_count",
552         "is_constant",
553         "is_near_constant",
554     ]
555 ]
556 .merge(
557     l1[
558         [
559             "feature",
560             "selected_folds",
561             "selection_frequency",
562             "mean_abs_l1_coefficient",
563             "l1_selected_any",
564             "l1_selected_near_constant",
565         ]
566     ],
567     on="feature",
568     how="left",
569 )
570 .merge(
571     tree[["feature", "tree_importance_mean", "tree_importance_std"]],
572     on="feature",
573     how="left",
574 )
575 .merge(
576     permutation[
577         [
578             "feature",
579             "permutation_importance_mean",
580             "permutation_importance_std_across_folds",
581             "permutation_importance_positive",
582         ]
583     ],
584     on="feature",
585     how="left",
586 )
587 )
588 if xgboost_importance is not None:
589     summary = summary.merge(
590         xgboost_importance[
591             [
592                 "feature",
593                 "xgboost_importance_mean",
594                 "xgboost_importance_std",
595                 "rank_xgboost_importance",
596             ]
597         ],
598         on="feature",
599         how="left",
600     )
601 if shap_importance is not None:
602     summary = summary.merge(
603         shap_importance[
604             [
605                 "feature",
606                 "shap_mean_abs_mean",
607                 "shap_mean_abs_std",
608                 "rank_shap_mean_abs",
609             ]
610         ],
611         on="feature",
612         how="left",
613     )
614 summary["rank_q_mannwhitney"] = rank_asc(summary["q_mannwhitney"])
615 summary["rank_abs_cohens_d"] = rank_desc(summary["abs_cohens_d"])
616 summary["rank_abs_cliffs_delta"] = rank_desc(summary["abs_cliffs_delta"])
617 summary["rank_selection_frequency"] = rank_desc(summary["selection_frequency"])
618 summary["rank_permutation_importance"] = rank_desc(
619     summary["permutation_importance_mean"]
620 )
621 summary["rank_tree_importance"] = rank_desc(summary["tree_importance_mean"])
622 rank_cols = [
623     "rank_q_mannwhitney",
624     "rank_abs_cohens_d",
625     "rank_abs_cliffs_delta",
626     "rank_selection_frequency",
627     "rank_permutation_importance",
628     "rank_tree_importance",
629 ]
630 summary["mean_rank"] = summary[rank_cols].mean(axis=1)
631 summary["rank_score"] = 1.0 / summary["mean_rank"]
632 summary = summary.sort_values(["mean_rank", "feature"]).reset_index(drop=True)
633 summary["included_in_top10"] = False
634 summary["included_in_top20"] = False
635 summary["included_in_top50"] = False
636 summary.loc[summary.index[:10], "included_in_top10"] = True
637 summary.loc[summary.index[:20], "included_in_top20"] = True
638 summary.loc[summary.index[:50], "included_in_top50"] = True
639 near_top50 = summary.loc[
640     summary["included_in_top50"] & summary["is_near_constant"], "feature"
641 ].tolist()
642 if near_top50:

```

```

643         raise ValueError(f"Near-constant features entered Top 50: {near_top50}")
644     return summary
645
646
647 def save_top20_rank_plot(summary: pd.DataFrame) -> None:
648     top20 = summary.head(20).sort_values("mean_rank", ascending=True)
649     fig, ax = plt.subplots(figsize=(11, 8), dpi=150)
650     ax.barh(top20["feature"], top20["rank_score"], color="#2f80ed")
651     ax.set_xlabel("Rank score (1 / mean rank)")
652     ax.set_ylabel("Feature")
653     ax.set_title("Stage 4 Top 20 Biomarker Candidates")
654     ax.invert_yaxis()
655     fig.tight_layout()
656     fig.savefig(FIGURES_DIR / "stage4_top20_biomarker_rank.png")
657     plt.close(fig)
658
659
660 def save_top20_effect_size_plot(summary: pd.DataFrame) -> None:
661     top20 = summary.head(20).copy()
662     top20["effect_direction"] = np.where(top20["cohens_d"] >= 0, "PD higher", "Healthy higher")
663     colors = top20["effect_direction"].map(
664         {"PD higher": "#d62728", "Healthy higher": "#1f77b4"}
665     )
666     top20 = top20.sort_values("cohens_d")
667     fig, ax = plt.subplots(figsize=(11, 8), dpi=150)
668     ax.barh(top20["feature"], top20["cohens_d"], color=colors.loc[top20.index])
669     ax.axvline(0, color="#333333", linewidth=0.8)
670     ax.set_xlabel("Cohen's d (PD - Healthy)")
671     ax.set_ylabel("Feature")
672     ax.set_title("Stage 4 Top 20 Effect Sizes")
673     fig.tight_layout()
674     fig.savefig(FIGURES_DIR / "stage4_top20_effect_size.png")
675     plt.close(fig)
676
677
678 def save_top50_category_plot(summary: pd.DataFrame) -> None:
679     counts = summary.head(50)["category"].value_counts().sort_values()
680     fig, ax = plt.subplots(figsize=(10, 6), dpi=150)
681     ax.barh(counts.index, counts.values, color="#27ae60")
682     ax.set_xlabel("Count in Top 50")
683     ax.set_ylabel("Feature category")
684     ax.set_title("Stage 4 Feature Category Counts in Top 50")
685     for index, value in enumerate(counts.values):
686         ax.text(value, index, f"{int(value)}", va="center")
687     fig.tight_layout()
688     fig.savefig(FIGURES_DIR / "stage4_feature_category_counts_top50.png")
689     plt.close(fig)
690
691
692 def save_volcano_plot(univariate: pd.DataFrame) -> None:
693     plot_df = univariate.copy()
694     min_positive_q = plot_df.loc[
695         plot_df["q_mannwhitney"] > 0, "q_mannwhitney"
696     ].min()
697     if pd.isna(min_positive_q):
698         min_positive_q = 1e-300
699     plot_df["safe_q"] = plot_df["q_mannwhitney"].replace(0, min_positive_q)
700     plot_df["neg_log10_q"] = -np.log10(plot_df["safe_q"])
701     fig, ax = plt.subplots(figsize=(9, 7), dpi=150)
702     ax.scatter(
703         plot_df["cohens_d"],
704         plot_df["neg_log10_q"],
705         s=18,
706         alpha=0.75,
707         color="#4b5563",
708         edgecolors="none",
709     )
710     ax.axvline(0, color="#333333", linewidth=0.8)
711     ax.set_xlabel("Cohen's d (PD - Healthy)")
712     ax.set_ylabel("-log10(FDR q-value)")
713     ax.set_title("Stage 4 Dataset 1 Volcano Plot")
714     fig.tight_layout()
715     fig.savefig(FIGURES_DIR / "stage4_volcano_plot_dataset1.png")
716     plt.close(fig)
717
718
719 def save_shap_top20_plot(shap_importance: pd.DataFrame) -> None:
720     if shap_importance["shap_mean_abs_mean"].isna().all():
721         return
722     top20 = shap_importance.head(20).sort_values("shap_mean_abs_mean", ascending=True)
723     fig, ax = plt.subplots(figsize=(11, 8), dpi=150)
724     ax.barh(top20["feature"], top20["shap_mean_abs_mean"], color="#7a5195")
725     ax.set_xlabel("Mean absolute SHAP value")
726     ax.set_ylabel("Feature")
727     ax.set_title("Stage 4 Supplemental XGBoost SHAP Top 20")
728     fig.tight_layout()
729     fig.savefig(FIGURES_DIR / "stage4_shap_top20_mean_abs.png")
730     plt.close(fig)
731
732
733 def write_report(
734     audit: dict[str, Any],

```

```

735 summary: pd.DataFrame,
736 univariate: pd.DataFrame,
737 l1: pd.DataFrame,
738 xgboost_importance: pd.DataFrame,
739 shap_importance: pd.DataFrame,
740 ) -> None:
741     dataset2_ref = (
742         f"~{rel(DATASET2_REFERENCE_PATH)}~ exists"
743         if DATASET2_REFERENCE_PATH.exists()
744         else f"~{rel(DATASET2_REFERENCE_PATH)}~ not found"
745     )
746     top20_cols = [
747         "feature",
748         "category",
749         "mean_rank",
750         "rank_score",
751         "q_mannwhitney",
752         "cohens_d",
753         "cliffs_delta",
754         "selection_frequency",
755         "permutation_importance_mean",
756         "tree_importance_mean",
757     ]
758     if {"xgboost_importance_mean", "shap_mean_abs_mean"}.issubset(summary.columns):
759         top20_cols.extend(["xgboost_importance_mean", "shap_mean_abs_mean"])
760     top10 = summary.head(10)[top20_cols].copy()
761     top20 = summary.head(20)[top20_cols].copy()
762     top50 = summary.head(50)[["feature", "category", "mean_rank", "rank_score"]].copy()
763     near_constant_features = set(audit["near_constant_features"])
764     near_constant_in_top50 = sorted(near_constant_features & set(summary.head(50)["feature"]))
765     near_constant_l1_selected = l1[
766         l1["feature"].isin(near_constant_features) & (l1["selected_folds"] > 0)
767     ]
768     nonzero_permutation_count = int(
769         (summary["permutation_importance_mean"] > 0).sum()
770     )
771     lines = [
772         "# Stage 4 Biomarker Candidate Report",
773         "",
774         "Main analysis: Dataset 1 subject-level PD/Healthy comparison.",
775         "",
776         "Dataset 2 is only noted as a reference prepared file and is not used in the main biomarker ranking.",
777         f"Dataset 2 reference status: {dataset2_ref}.",
778         "",
779         "## Subject-level audit",
780         "",
781         f"- Subject count: ~{audit['subject_count']}~",
782         f"- Class distribution: ~{audit['class_distribution']}~",
783         f"- Acoustic feature count: ~{audit['acoustic_feature_count']}~",
784         f"- Missing value count: ~{audit['missing_value_count']}~",
785         f"- Constant feature count: ~{audit['constant_feature_count']}~",
786         f"- Near-constant feature count (~std <= {NEAR_CONSTANT_STD_THRESHOLD}): ~{audit['near_constant_feature_count']}~",
787         f"- Near-constant features in Top 50: ~{len(near_constant_in_top50)}~",
788         f"- Near-constant features selected by L1 at least once: ~{len(near_constant_l1_selected)}~",
789         "- Main features exclude 'id', 'gender', 'class', and 'sex_male'.",
790         "",
791         "## Methods",
792         "",
793         "- Dataset 1 records were aggregated to one row per subject before all statistical tests and models.",
794         "- Univariate screening used Mann-Whitney U, Welch t-test, FDR-BH correction, Cohen's d, and Cliff's delta.",
795         "- L1 stability selection used subject-level StratifiedKFold; scaling and L1 Logistic were fit only inside training
796         ~> folds.",
797         "- ExtraTrees and permutation importance used subject-level StratifiedKFold; permutation importance was computed on
798         ~> held-out folds.",
799         "- XGBoost and SHAP are supplemental evidence only; they are not included in the mean-rank formula and do not change
800         ~> Top 10/20/50 membership.",
801         "- These features are exploratory biomarker candidates for modeling interpretation and require independent
802         ~> validation.",
803         "- Some near-constant features were selected by L1, so L1 stability is treated as one evidence source rather than a
804         ~> standalone final conclusion.",
805         f"- Permutation importance is sparse (~{nonzero_permutation_count}~ of ~{summary.shape[0]}~ features have positive
806         ~> mean importance), which may reflect substitution effects among high-dimensional correlated acoustic features.",
807         "- The final Top 20 are candidate voice biomarker features, not clinical causal mechanisms.",
808         "",
809         "## Top 10 biomarker candidates",
810         "",
811         markdown_table(top10, floatfmt=".5f"),
812         "",
813         "## Top 20 biomarker candidates",
814         "",
815         markdown_table(top20, floatfmt=".5f"),
816         "",
817         "## Top 50 biomarker candidates",
818         "",
819         markdown_table(top50, floatfmt=".5f"),
820         "",
821         "## Supplemental XGBoost/SHAP evidence",
822         "",
823         f"- XGBoost/SHAP available: ~{HAS_XGBOOST_SHAP}~",
824         f"- XGBoost/SHAP import error: ~{XGBOOST_SHAP_IMPORT_ERROR}~",
825         f"- Top SHAP feature: ~{shap_importance.iloc[0]['feature']} if not shap_importance.empty else ''~",

```



```

820         f"- Top XGBoost importance feature: `{xgboost_importance.iloc[0]['feature']}` if not xgboost_importance.empty else
↪         `{}`.",
821         "",
822         "## Output files",
823         "",
824         "- `results/stage4_subject_level_feature_table_dataset1.csv`",
825         "- `results/stage4_feature_audit_dataset1.csv`",
826         "- `results/stage4_univariate_statistics_dataset1.csv`",
827         "- `results/stage4_l1_stability_selection_dataset1.csv`",
828         "- `results/stage4_tree_importance_dataset1.csv`",
829         "- `results/stage4_permutation_importance_dataset1.csv`",
830         "- `results/stage4_xgboost_importance_dataset1.csv`",
831         "- `results/stage4_shap_importance_dataset1.csv`",
832         "- `results/stage4_biomarker_rank_summary_dataset1.csv`",
833         "- `results/stage4_top10_biomarkers_dataset1.csv`",
834         "- `results/stage4_top20_biomarkers_dataset1.csv`",
835         "- `results/stage4_top50_biomarkers_dataset1.csv`",
836         "- `results/figures/stage4_top20_biomarker_rank.png`",
837         "- `results/figures/stage4_top20_effect_size.png`",
838         "- `results/figures/stage4_feature_category_counts_top50.png`",
839         "- `results/figures/stage4_volcano_plot_dataset1.png`",
840         "- `results/figures/stage4_shap_top20_mean_abs.png`",
841         "",
842         "## Guardrails",
843         "",
844         "- The 755 recordings were not treated as independent samples for significance testing.",
845         "- Metadata and sex variables were excluded from the main acoustic feature list.",
846         "- Final ranking combines univariate, stability, permutation, and tree-based evidence rather than relying only on
↪         tree importance.",
847         "- Supplemental XGBoost/SHAP evidence is not used by Stage 5 or Stage 6.",
848         "- The report uses associative interpretation only.",
849         "",
850         f"Univariate features with FDR q < 0.05: `{int((univariate['q_mannwhitney'] < 0.05).sum())}`",
851     ]
852     (RESULTS_DIR / "stage4_biomarker_report.md").write_text(
853         "\n".join(lines), encoding="utf-8"
854     )
855
856
857 def markdown_table(df: pd.DataFrame, floatfmt: str = ".4f") -> str:
858     headers = list(df.columns)
859     rows = []
860     for _, row in df.iterrows():
861         values = []
862         for column in headers:
863             value = row[column]
864             if isinstance(value, float) and not pd.isna(value):
865                 if column.startswith("q_"):
866                     values.append(format(value, ".3e"))
867                 else:
868                     values.append(format(value, floatfmt))
869             elif pd.isna(value):
870                 values.append("")
871             else:
872                 values.append(str(value))
873         rows.append(values)
874     table = [
875         "| " + " | ".join(headers) + " |",
876         "| " + " | ".join(["---"] * len(headers)) + " |",
877     ]
878     table.extend("| " + " | ".join(row) + " |" for row in rows)
879     return "\n".join(table)
880
881
882 def main() -> None:
883     RESULTS_DIR.mkdir(exist_ok=True)
884     FIGURES_DIR.mkdir(exist_ok=True)
885
886     df = load_dataset1()
887     features = acoustic_columns(df)
888     subject_table = build_subject_table(df, features)
889     audit = audit_subject_table(subject_table, features)
890     feature_audit = build_feature_audit_table(subject_table, features)
891
892     univariate = run_univariate_statistics(subject_table, features)
893     univariate = add_q_value_display_columns(
894         merge_feature_audit(univariate, feature_audit)
895     )
896     l1 = merge_feature_audit(run_l1_stability(subject_table, features), feature_audit)
897     l1["l1_selected_any"] = l1["selected_folds"] > 0
898     l1["l1_selected_near_constant"] = l1["l1_selected_any"] & l1["is_near_constant"]
899     tree, permutation = run_tree_and_permutation(subject_table, features)
900     xgboost_importance, shap_importance = run_xgboost_and_shap(subject_table, features)
901     tree = merge_feature_audit(tree, feature_audit)
902     permutation = merge_feature_audit(permutation, feature_audit)
903     xgboost_importance = merge_feature_audit(xgboost_importance, feature_audit)
904     shap_importance = merge_feature_audit(shap_importance, feature_audit)
905     permutation["permutation_importance_positive"] = (
906         permutation["permutation_importance_mean"] > 0
907     )
908     summary = build_rank_summary(
909         univariate, l1, tree, permutation, xgboost_importance, shap_importance

```

```

910     )
911     top20 = summary.head(20).copy()
912     top10 = summary.head(10).copy()
913     top50 = summary.head(50).copy()
914
915     write_csv(
916         subject_table,
917         RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv",
918     )
919     write_csv(
920         feature_audit,
921         RESULTS_DIR / "stage4_feature_audit_dataset1.csv",
922     )
923     write_csv(
924         univariate,
925         RESULTS_DIR / "stage4_univariate_statistics_dataset1.csv",
926     )
927     write_csv(
928         l1,
929         RESULTS_DIR / "stage4_l1_stability_selection_dataset1.csv",
930     )
931     write_csv(
932         tree,
933         RESULTS_DIR / "stage4_tree_importance_dataset1.csv",
934     )
935     write_csv(
936         permutation,
937         RESULTS_DIR / "stage4_permutation_importance_dataset1.csv",
938     )
939     write_csv(
940         xgboost_importance,
941         RESULTS_DIR / "stage4_xgboost_importance_dataset1.csv",
942     )
943     write_csv(
944         shap_importance,
945         RESULTS_DIR / "stage4_shap_importance_dataset1.csv",
946     )
947     write_csv(
948         summary,
949         RESULTS_DIR / "stage4_biomarker_rank_summary_dataset1.csv",
950     )
951     write_csv(
952         top10,
953         RESULTS_DIR / "stage4_top10_biomarkers_dataset1.csv",
954     )
955     write_csv(
956         top20,
957         RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv",
958     )
959     write_csv(
960         top50,
961         RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv",
962     )
963
964     save_top20_rank_plot(summary)
965     save_top20_effect_size_plot(summary)
966     save_top50_category_plot(summary)
967     save_volcano_plot(univariate)
968     save_shap_top20_plot(shap_importance)
969     write_report(audit, summary, univariate, l1, xgboost_importance, shap_importance)
970
971
972 if __name__ == "__main__":
973     main()

```

A.4 代码四：二类潜在语音表型建模

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Any
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import mannwhitneyu
14 from sklearn.cluster import AgglomerativeClustering, KMeans
15 from sklearn.decomposition import PCA
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.metrics import (
18     accuracy_score,
19     adjusted_rand_score,
20     balanced_accuracy_score,
21     calinski_harabasz_score,
22     davies_bouldin_score,
23     f1_score,
24     roc_auc_score,
25     silhouette_score,
26 )
27 from sklearn.mixture import GaussianMixture
28 from sklearn.model_selection import StratifiedKFold
29 from sklearn.pipeline import Pipeline
30 from sklearn.preprocessing import StandardScaler
31 from sklearn.linear_model import LogisticRegression
32 from sklearn.svm import SVC
33
34
35 ROOT = Path(__file__).resolve().parents[1]
36 RESULTS_DIR = ROOT / "results"
37 FIGURES_DIR = RESULTS_DIR / "figures"
38
39 SUBJECT_TABLE_PATH = RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv"
40 TOP20_PATH = RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv"
41 TOP50_PATH = RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv"
42
43 OUTPUT_CSV_ENCODING = "utf-8-sig"
44 RANDOM_STATE = 42
45 PD_LABEL = 1
46 EXPECTED_SUBJECTS = 252
47 EXPECTED_PD_SUBJECTS = 188
48 EXPECTED_ACOUSTIC_FEATURES = 752
49 STABILITY_RUNS = 50
50 STABILITY_SAMPLE_FRACTION = 0.8
51
52 METADATA_COLUMNS = {
53     "id",
54     "gender",
55     "class",
56     "sex_male",
57     "record_count",
58 }
59 LABEL_COLUMN_TOKENS = ("cluster", "label", "prediction", "phenotype")
60
61
62 def rel(path: Path) -> str:
63     return path.relative_to(ROOT).as_posix()
64
65
66 def write_csv(df: pd.DataFrame, path: Path) -> None:
67     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
68
69
70 def require_file(path: Path) -> None:
71     if not path.exists():
72         raise FileNotFoundError(f"Missing required Stage 4 input file: {path}")
73
74
75 def load_inputs() -> tuple[pd.DataFrame, list[str], list[str]]:
76     for path in [SUBJECT_TABLE_PATH, TOP20_PATH, TOP50_PATH]:
77         require_file(path)
78
79     subject_table = pd.read_csv(SUBJECT_TABLE_PATH)
80     top20 = pd.read_csv(TOP20_PATH)
81     top50 = pd.read_csv(TOP50_PATH)
82
83     required_subject_columns = {"id", "class", "sex_male"}
84     missing_subject_columns = sorted(required_subject_columns - set(subject_table.columns))
85     if missing_subject_columns:
86         raise ValueError(f"Subject table missing required columns: {missing_subject_columns}")
87
88     if subject_table.shape[0] != EXPECTED_SUBJECTS:
89         raise ValueError(
90             f"Expected {EXPECTED_SUBJECTS} subject rows, found {subject_table.shape[0]}"
```

```

91     )
92     duplicated_id_count = int(subject_table["id"].duplicated().sum())
93     if duplicated_id_count:
94         raise ValueError(
95             f"Subject table must contain one row per subject; duplicated ids: {duplicated_id_count}"
96         )
97
98     for label, table, expected_rows in [
99         ("Top20", top20, 20),
100         ("Top50", top50, 50),
101     ]:
102         if "feature" not in table.columns:
103             raise ValueError(f"{label} input is missing the required 'feature' column")
104         if table.shape[0] != expected_rows:
105             raise ValueError(f"{label} input expected {expected_rows} rows, found {table.shape[0]}")
106
107         top20_features = top20["feature"].astype(str).tolist()
108         top50_features = top50["feature"].astype(str).tolist()
109         missing_top20 = sorted(set(top20_features) - set(subject_table.columns))
110         missing_top50 = sorted(set(top50_features) - set(subject_table.columns))
111         if missing_top20 or missing_top50:
112             raise ValueError(
113                 "Stage 4 biomarker features missing from subject-level table: "
114                 f"top20={missing_top20}, top50={missing_top50}"
115             )
116
117     return subject_table, top20_features, top50_features
118
119
120 def is_forbidden_feature_column(column: str) -> bool:
121     name = column.lower()
122     return column in METADATA_COLUMNS or any(token in name for token in LABEL_COLUMN_TOKENS)
123
124
125 def acoustic_columns(subject_table: pd.DataFrame) -> list[str]:
126     columns = [
127         column
128         for column in subject_table.select_dtypes(include="number").columns
129         if not is_forbidden_feature_column(column)
130     ]
131     if len(columns) != EXPECTED_ACOUSTIC_FEATURES:
132         raise ValueError(
133             f"Expected {EXPECTED_ACOUSTIC_FEATURES} all-acoustic features after metadata "
134             f"exclusion, found {len(columns)}"
135         )
136     forbidden = sorted(set(columns) & METADATA_COLUMNS)
137     if forbidden:
138         raise ValueError(f"Forbidden metadata columns entered acoustic features: {forbidden}")
139     return columns
140
141
142 def validate_pd_subjects(subject_table: pd.DataFrame) -> pd.DataFrame:
143     before_counts = subject_table["class"].value_counts().sort_index().to_dict()
144     if int(sum(before_counts.values())) != EXPECTED_SUBJECTS:
145         raise ValueError("Class distribution does not sum to the expected subject count")
146
147     pd_subjects = subject_table.loc[subject_table["class"] == PD_LABEL].copy()
148     if pd_subjects.shape[0] != EXPECTED_PD_SUBJECTS:
149         raise ValueError(
150             f"Expected {EXPECTED_PD_SUBJECTS} PD subjects, found {pd_subjects.shape[0]}"
151         )
152     return pd_subjects
153
154
155 def write_input_audit(
156     subject_table: pd.DataFrame,
157     pd_subjects: pd.DataFrame,
158     all_features: list[str],
159     top20_features: list[str],
160     top50_features: list[str],
161 ) -> None:
162     excluded_columns = [
163         column if column in subject_table.columns else f"{column} (not present)"
164         for column in ["id", "gender", "class", "sex_male", "record_count"]
165     ]
166     audit = pd.DataFrame(
167         [
168             {
169                 "input_file": rel(SUBJECT_TABLE_PATH),
170                 "total_subjects_before_filter": int(subject_table.shape[0]),
171                 "pd_subjects_after_filter": int(pd_subjects.shape[0]),
172                 "class_distribution_before_filter": str(
173                     {
174                         int(k): int(v)
175                         for k, v in subject_table["class"]
176                             .value_counts()
177                             .sort_index()
178                             .items()
179                     }
180                 ),
181                 "class_distribution_after_filter": str(
182                     {

```

```

183             int(k): int(v)
184             for k, v in pd_subjects["class"].value_counts().sort_index().items()
185         }
186     ),
187     "missing_value_count": int(subject_table.isna().sum().sum()),
188     "duplicated_id_count": int(subject_table["id"].duplicated().sum()),
189     "acoustic_feature_count": int(len(all_features)),
190     "excluded_columns": "; ".join(excluded_columns),
191     "feature_sets_used": "top20_biomarkers; top50_biomarkers; all_acoustic",
192     "top20_feature_count": int(len(top20_features)),
193     "top50_feature_count": int(len(top50_features)),
194     "all_acoustic_feature_count": int(len(all_features)),
195 }
196 ]
197 )
198 write_csv(audit, RESULTS_DIR / "stage5_pd_subject_clustering_input_audit.csv")
199
200
201 def categorize_feature(feature: str) -> str:
202     name = feature.lower()
203     if "tqwt" in name:
204         return "TQWT"
205     if any(token in name for token in ["wavelet", "_lt_", "ed_", "ed2_", "entropy"]):
206         return "Wavelet"
207     if "jitter" in name:
208         return "Jitter"
209     if "shimmer" in name or "shim_" in name or "apq" in name:
210         return "Shimmer"
211     if "hnr" in name or "harmonicity" in name or "harmtonoise" in name:
212         return "HNR / Harmonicity"
213     if "intensity" in name:
214         return "Intensity"
215     if (
216         name.startswith("f")
217         and len(name) == 2
218         and name[1].isdigit()
219         or name.startswith("b")
220         and len(name) == 2
221         and name[1].isdigit()
222     ):
223         return "Formant / Bandwidth"
224     if "mfcc" in name:
225         return "MFCC"
226     if "delta_delta" in name or "delta" in name:
227         return "Delta / Delta-delta"
228     if any(token in name for token in ["rpde", "dfa", "ppe", "gne"]):
229         return "Nonlinear"
230     return "Other"
231
232
233 def build_representation(
234     data: pd.DataFrame, representation: str
235 ) -> tuple[np.ndarray, int, float]:
236     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
237     if representation == "scaled_only":
238         return scaled, scaled.shape[1], np.nan
239
240     if representation == "pca_90":
241         pca = PCA(n_components=0.90, random_state=RANDOM_STATE)
242     elif representation.startswith("pca_"):
243         pca = PCA(n_components=int(representation.split("_", maxsplit=1)[1]), random_state=RANDOM_STATE)
244     else:
245         raise ValueError(f"Unknown representation: {representation}")
246
247     projected = pca.fit_transform(scaled)
248     return projected, projected.shape[1], float(np.sum(pca.explained_variance_ratio_))
249
250
251 def cluster_labels(
252     matrix: np.ndarray, method: str, random_state: int = RANDOM_STATE
253 ) -> np.ndarray:
254     if method == "KMeans":
255         return KMeans(
256             n_clusters=2,
257             n_init=50,
258             random_state=random_state,
259         ).fit_predict(matrix)
260     if method == "GaussianMixture":
261         return GaussianMixture(
262             n_components=2,
263             covariance_type="full",
264             reg_covar=1e-6,
265             random_state=random_state,
266         ).fit_predict(matrix)
267     if method == "AgglomerativeClustering":
268         return AgglomerativeClustering(n_clusters=2, linkage="ward").fit_predict(matrix)
269     raise ValueError(f"Unknown clustering method: {method}")
270
271
272 def clustering_quality(matrix: np.ndarray, labels: np.ndarray) -> tuple[float, float, float, str]:
273     unique_labels = np.unique(labels)
274     if unique_labels.size < 2:

```

```

275         return np.nan, np.nan, np.nan, "single cluster produced"
276     try:
277         silhouette = float(silhouette_score(matrix, labels))
278         calinski = float(calinski_harabasz_score(matrix, labels))
279         davis = float(davies_bouldin_score(matrix, labels))
280     except ValueError as exc:
281         return np.nan, np.nan, np.nan, f"metric calculation failed: {exc}"
282     return silhouette, calinski, davis, ""
283
284
285 def stability_analysis(
286     data: pd.DataFrame,
287     representation: str,
288     method: str,
289     full_labels: np.ndarray,
290 ) -> tuple[float, float]:
291     rng = np.random.default_rng(RANDOM_STATE)
292     n_subjects = data.shape[0]
293     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
294     scores: list[float] = []
295
296     for run_index in range(STABILITY_RUNS):
297         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
298         sample_data = data.iloc[sample_index]
299         sample_matrix, _, _ = build_representation(sample_data, representation)
300         sample_labels = cluster_labels(
301             sample_matrix,
302             method,
303             random_state=RANDOM_STATE + run_index + 1,
304         )
305         scores.append(float(adjusted_rand_score(full_labels[sample_index], sample_labels)))
306
307     return float(np.mean(scores)), float(np.std(scores, ddof=1))
308
309
310 def run_clustering_candidates(
311     pd_subjects: pd.DataFrame, feature_sets: dict[str, list[str]]
312 ) -> tuple[pd.DataFrame, dict[tuple[str, str, str], dict[str, Any]]]:
313     representations = {
314         "top20_biomarkers": ["scaled_only", "pca_5", "pca_10"],
315         "top50_biomarkers": ["pca_10", "pca_90"],
316         "all_acoustic": ["pca_20", "pca_90"],
317     }
318     methods = ["KMeans", "GaussianMixture", "AgglomerativeClustering"]
319
320     rows: list[dict[str, Any]] = []
321     candidate_artifacts: dict[tuple[str, str, str], dict[str, Any]] = {}
322
323     for feature_set, features in feature_sets.items():
324         data = pd_subjects[features]
325         for representation in representations[feature_set]:
326             matrix, n_components, explained_variance = build_representation(data, representation)
327             for method in methods:
328                 labels = cluster_labels(matrix, method)
329                 counts = pd.Series(labels).value_counts().reindex([0, 1], fill_value=0)
330                 cluster_0_size = int(counts.loc[0])
331                 cluster_1_size = int(counts.loc[1])
332                 smaller_cluster_size = int(min(cluster_0_size, cluster_1_size))
333                 smaller_cluster_ratio = float(smaller_cluster_size / len(labels))
334
335                 silhouette, calinski, davis, metric_warning = clustering_quality(matrix, labels)
336                 stability_mean, stability_std = stability_analysis(
337                     data,
338                     representation,
339                     method,
340                     labels,
341                 )
342
343                 warnings = []
344                 if metric_warning:
345                     warnings.append(metric_warning)
346                 if smaller_cluster_ratio < 0.10:
347                     warnings.append("smaller_cluster_ratio < 0.10; not eligible for recommendation")
348
349                 key = (feature_set, representation, method)
350                 rows.append(
351                     {
352                         "feature_set": feature_set,
353                         "representation": representation,
354                         "clustering_method": method,
355                         "n_subjects": int(len(labels)),
356                         "n_features_original": int(len(features)),
357                         "n_components_after_pca": int(n_components),
358                         "pca_explained_variance_sum": explained_variance,
359                         "cluster_0_size": cluster_0_size,
360                         "cluster_1_size": cluster_1_size,
361                         "smaller_cluster_size": smaller_cluster_size,
362                         "smaller_cluster_ratio": smaller_cluster_ratio,
363                         "silhouette": silhouette,
364                         "calinski_harabasz": calinski,
365                         "davies_bouldin": davis,
366                         "stability_ari_mean": stability_mean,

```

```

367         "stability_ari_std": stability_std,
368         "stability_n_runs": STABILITY_RUNS,
369         "recommended_flag": False,
370         "exclusion_or_warning": "; ".join(warnings),
371     }
372 )
373     candidate_artifacts[key] = {
374         "labels": labels,
375         "matrix": matrix,
376         "features": features,
377         "n_components_after_pca": n_components,
378         "pca_explained_variance_sum": explained_variance,
379     }
380
381 metrics = pd.DataFrame(rows)
382 return metrics, candidate_artifacts
383
384
385 def minmax(series: pd.Series, higher_is_better: bool = True) -> pd.Series:
386     values = series.astype(float)
387     valid = values.replace([np.inf, -np.inf], np.nan)
388     min_value = valid.min(skipna=True)
389     max_value = valid.max(skipna=True)
390     if pd.isna(min_value) or pd.isna(max_value) or max_value == min_value:
391         return pd.Series(0.5, index=series.index)
392     scaled = (valid - min_value) / (max_value - min_value)
393     if not higher_is_better:
394         scaled = 1 - scaled
395     return scaled.fillna(0.0)
396
397
398 def choose_recommended(metrics: pd.DataFrame) -> tuple[pd.DataFrame, tuple[str, str, str]]:
399     out = metrics.copy()
400     eligible = out["smaller_cluster_ratio"] >= 0.10
401     if not eligible.any():
402         raise ValueError("No clustering candidate has smaller_cluster_ratio >= 0.10")
403
404     balance = 1 - (0.5 - out["smaller_cluster_ratio"]).abs() / 0.5
405     score = (
406         0.25 * minmax(out["silhouette"], higher_is_better=True)
407         + 0.15 * minmax(out["calinski_harabasz"], higher_is_better=True)
408         + 0.15 * minmax(out["davies_bouldin"], higher_is_better=False)
409         + 0.30 * minmax(out["stability_ari_mean"], higher_is_better=True)
410         + 0.15 * balance
411     )
412     score += out["feature_set"].map(
413         {
414             "top20_biomarkers": 0.08,
415             "top50_biomarkers": 0.04,
416             "all_acoustic": -0.02,
417         }
418     )
419     score = score.where(eligible, -np.inf)
420     out["selection_score"] = score
421
422     best_index = int(out["selection_score"].idxmax())
423     best_score = float(out.loc[best_index, "selection_score"])
424
425     top20_close = out[
426         (out["feature_set"] == "top20_biomarkers")
427         & eligible
428         & (out["selection_score"] >= best_score - 0.03)
429     ]
430     if not top20_close.empty:
431         best_index = int(top20_close["selection_score"].idxmax())
432     else:
433         interpretable_close = out[
434             (out["feature_set"].isin(["top20_biomarkers", "top50_biomarkers"]))
435             & eligible
436             & (out["selection_score"] >= best_score - 0.08)
437         ]
438         if not interpretable_close.empty and out.loc[best_index, "feature_set"] == "all_acoustic":
439             best_index = int(interpretable_close["selection_score"].idxmax())
440
441     out["recommended_flag"] = False
442     out.loc[best_index, "recommended_flag"] = True
443     key = (
444         str(out.loc[best_index, "feature_set"]),
445         str(out.loc[best_index, "representation"]),
446         str(out.loc[best_index, "clustering_method"]),
447     )
448     return out, key
449
450
451 def pca_2d_coordinates(data: pd.DataFrame) -> np.ndarray:
452     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
453     return PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(scaled)
454
455
456 def write_cluster_assignments(
457     pd_subjects: pd.DataFrame,
458     recommended_key: tuple[str, str, str],

```

```

459     recommended_artifact: dict[str, Any],
460 ) -> pd.DataFrame:
461     feature_set, representation, method = recommended_key
462     labels = np.asarray(recommended_artifact["labels"], dtype=int)
463     counts = pd.Series(labels).value_counts().to_dict()
464     coords = pca_2d_coordinates(pd_subjects[recommended_artifact["features"]])
465     assignments = pd.DataFrame(
466         {
467             "id": pd_subjects["id"].to_numpy(),
468             "class": pd_subjects["class"].to_numpy(),
469             "sex_male": pd_subjects["sex_male"].to_numpy(),
470             "cluster": labels,
471             "cluster_size": [int(counts[label]) for label in labels],
472             "feature_set": feature_set,
473             "representation": representation,
474             "clustering_method": method,
475             "pca_1": coords[:, 0],
476             "pca_2": coords[:, 1],
477         }
478     )
479     write_csv(assignments, RESULTS_DIR / "stage5_cluster_assignments_dataset1_pd.csv")
480     return assignments
481
482
483 def fdr_bh(p_values: np.ndarray) -> np.ndarray:
484     p = np.asarray(p_values, dtype=float)
485     adjusted = np.full(p.shape, np.nan, dtype=float)
486     valid = np.isfinite(p)
487     if not valid.any():
488         return adjusted
489     valid_p = p[valid]
490     order = np.argsort(valid_p)
491     ordered_p = valid_p[order]
492     n = len(ordered_p)
493     ordered_q = ordered_p * n / np.arange(1, n + 1)
494     ordered_q = np.minimum.accumulate(ordered_q[::-1])[::-1]
495     ordered_q = np.clip(ordered_q, 0, 1)
496     q = np.empty_like(ordered_q)
497     q[order] = ordered_q
498     adjusted[valid] = q
499     return adjusted
500
501
502 def cohens_d(values_1: np.ndarray, values_0: np.ndarray) -> float:
503     n_1 = len(values_1)
504     n_0 = len(values_0)
505     if n_1 < 2 or n_0 < 2:
506         return np.nan
507     var_1 = np.var(values_1, ddof=1)
508     var_0 = np.var(values_0, ddof=1)
509     pooled = ((n_1 - 1) * var_1 + (n_0 - 1) * var_0) / (n_1 + n_0 - 2)
510     if pooled <= 0:
511         return 0.0
512     return float((np.mean(values_1) - np.mean(values_0)) / np.sqrt(pooled))
513
514
515 def cluster_feature_differences(
516     pd_subjects: pd.DataFrame,
517     labels: np.ndarray,
518     features: list[str],
519 ) -> tuple[pd.DataFrame, pd.DataFrame]:
520     raw = pd_subjects[features]
521     scaled = pd.DataFrame(
522         StandardScaler().fit_transform(raw.to_numpy(dtype=float)),
523         columns=features,
524         index=raw.index,
525     )
526     rows: list[dict[str, Any]] = []
527     cluster_0_mask = labels == 0
528     cluster_1_mask = labels == 1
529
530     for feature in features:
531         raw_0 = raw.loc[cluster_0_mask, feature].to_numpy(dtype=float)
532         raw_1 = raw.loc[cluster_1_mask, feature].to_numpy(dtype=float)
533         z_0 = scaled.loc[cluster_0_mask, feature].to_numpy(dtype=float)
534         z_1 = scaled.loc[cluster_1_mask, feature].to_numpy(dtype=float)
535
536         try:
537             u_stat, p_value = mannwhitneyu(raw_0, raw_1, alternative="two-sided")
538         except ValueError:
539             u_stat, p_value = np.nan, np.nan
540
541         mean_difference = float(np.mean(raw_1) - np.mean(raw_0))
542         z_difference = float(np.mean(z_1) - np.mean(z_0))
543         d_value = cohens_d(raw_1, raw_0)
544
545         rows.append(
546             {
547                 "feature": feature,
548                 "category": categorize_feature(feature),
549                 "cluster_0_mean": float(np.mean(raw_0)),
550                 "cluster_1_mean": float(np.mean(raw_1)),

```



```

551         "mean_difference_cluster1_minus_cluster0": mean_difference,
552         "cluster_0_zmean": float(np.mean(z_0)),
553         "cluster_1_zmean": float(np.mean(z_1)),
554         "zmean_difference_cluster1_minus_cluster0": z_difference,
555         "cohens_d_cluster1_minus_cluster0": d_value,
556         "mannwhitney_u": float(u_stat),
557         "p_mannwhitney": float(p_value),
558         "abs_cohens_d": float(abs(d_value)) if np.isfinite(d_value) else np.nan,
559         "abs_zmean_difference": float(abs(z_difference)),
560         "direction": (
561             "higher_in_cluster_1"
562             if z_difference > 0
563             else "higher_in_cluster_0"
564             if z_difference < 0
565             else "no_difference"
566         ),
567     }
568 )
569
570 differences = pd.DataFrame(rows)
571 differences["q_mannwhitney"] = fdr_bh(differences["p_mannwhitney"].to_numpy())
572 ordered_columns = [
573     "feature",
574     "category",
575     "cluster_0_mean",
576     "cluster_1_mean",
577     "mean_difference_cluster1_minus_cluster0",
578     "cluster_0_zmean",
579     "cluster_1_zmean",
580     "zmean_difference_cluster1_minus_cluster0",
581     "cohens_d_cluster1_minus_cluster0",
582     "mannwhitney_u",
583     "p_mannwhitney",
584     "q_mannwhitney",
585     "abs_cohens_d",
586     "abs_zmean_difference",
587     "direction",
588 ]
589 differences = differences[ordered_columns]
590 differences = differences.sort_values(
591     ["abs_zmean_difference", "abs_cohens_d", "q_mannwhitney"],
592     ascending=[False, False, True],
593 ).reset_index(drop=True)
594 top_features = differences.head(20).copy()
595
596 write_csv(differences, RESULTS_DIR / "stage5_cluster_feature_differences.csv")
597 write_csv(top_features, RESULTS_DIR / "stage5_cluster_interpretation_top_features.csv")
598 return differences, top_features
599
600
601 def plot_pca_clusters(
602     pd_subjects: pd.DataFrame,
603     features: list[str],
604     labels: np.ndarray,
605     output_path: Path,
606     title: str,
607 ) -> None:
608     coords = pca_2d_coordinates(pd_subjects[features])
609     fig, ax = plt.subplots(figsize=(8, 6))
610     for cluster in sorted(np.unique(labels)):
611         mask = labels == cluster
612         ax.scatter(
613             coords[mask, 0],
614             coords[mask, 1],
615             s=42,
616             alpha=0.82,
617             label=f"Cluster {cluster}",
618         )
619     ax.set_title(title)
620     ax.set_xlabel("PCA 1")
621     ax.set_ylabel("PCA 2")
622     ax.legend(title="Main cluster label")
623     fig.tight_layout()
624     fig.savefig(output_path, dpi=220)
625     plt.close(fig)
626
627
628 def plot_cluster_sizes(metrics: pd.DataFrame) -> None:
629     plot_data = metrics.copy()
630     plot_data["candidate"] = (
631         plot_data["feature_set"]
632         + "\n"
633         + plot_data["representation"]
634         + "\n"
635         + plot_data["clustering_method"]
636     )
637     colors = np.where(plot_data["recommended_flag"], "#d55e00", "#4c78a8")
638     fig, ax = plt.subplots(figsize=(12, 7))
639     ax.bar(np.arange(plot_data.shape[0]), plot_data["smaller_cluster_ratio"], color=colors)
640     ax.axhline(0.10, color="#cc3311", linestyle="--", linewidth=1.2, label="0.10 threshold")
641     ax.set_xticks(np.arange(plot_data.shape[0]))
642     ax.set_xticklabels(plot_data["candidate"], rotation=75, ha="right", fontsize=7)

```

```

643     ax.set_ylabel("Smaller cluster ratio")
644     ax.set_title("Stage 5 Candidate Cluster Size Balance")
645     ax.legend()
646     fig.tight_layout()
647     fig.savefig(FIGURES_DIR / "stage5_cluster_size_comparison.png", dpi=220)
648     plt.close(fig)
649
650
651 def plot_feature_difference_top20(top_features: pd.DataFrame) -> None:
652     plot_data = top_features.sort_values("cohens_d_cluster1_minus_cluster0")
653     colors = np.where(
654         plot_data["cohens_d_cluster1_minus_cluster0"] >= 0,
655         "#2f80ed",
656         "#e07a5f",
657     )
658     fig, ax = plt.subplots(figsize=(9, 7))
659     ax.barh(
660         plot_data["feature"],
661         plot_data["cohens_d_cluster1_minus_cluster0"],
662         color=colors,
663     )
664     ax.axvline(0, color="#333333", linewidth=0.8)
665     ax.set_xlabel("Cohen's d (cluster 1 minus cluster 0)")
666     ax.set_ylabel("Feature")
667     ax.set_title("Stage 5 Top 20 Cluster-Distinguishing Voice Features")
668     fig.tight_layout()
669     fig.savefig(FIGURES_DIR / "stage5_cluster_feature_difference_top20.png", dpi=220)
670     plt.close(fig)
671
672
673 def run_cluster_label_classifier(
674     pd_subjects: pd.DataFrame,
675     features: list[str],
676     labels: np.ndarray,
677 ) -> pd.DataFrame:
678     x = pd_subjects[features].to_numpy(dtype=float)
679     y = labels.astype(int)
680     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
681     models: dict[str, Any] = {
682         "Logistic Regression": Pipeline(
683             [
684                 ("scaler", StandardScaler()),
685                 (
686                     "model",
687                     LogisticRegression(
688                         max_iter=5000,
689                         class_weight="balanced",
690                         random_state=RANDOM_STATE,
691                     ),
692                 ),
693             ]
694         ),
695         "SVM-RBF": Pipeline(
696             [
697                 ("scaler", StandardScaler()),
698                 (
699                     "model",
700                     SVC(
701                         kernel="rbf",
702                         probability=True,
703                         class_weight="balanced",
704                         random_state=RANDOM_STATE,
705                     ),
706                 ),
707             ]
708         ),
709         "Random Forest": RandomForestClassifier(
710             n_estimators=500,
711             random_state=RANDOM_STATE,
712             class_weight="balanced",
713         ),
714     }
715
716     rows: list[dict[str, Any]] = []
717     for model_name, model in models.items():
718         fold_rows: list[dict[str, Any]] = []
719         for fold_index, (train_index, test_index) in enumerate(cv.split(x, y), start=1):
720             model.fit(x[train_index], y[train_index])
721             predictions = model.predict(x[test_index])
722             try:
723                 probabilities = model.predict_proba(x[test_index])[:, 1]
724                 auc = float(roc_auc_score(y[test_index], probabilities))
725             except (AttributeError, ValueError):
726                 auc = np.nan
727
728             fold_row = {
729                 "model": model_name,
730                 "fold": fold_index,
731                 "accuracy": float(accuracy_score(y[test_index], predictions)),
732                 "balanced_accuracy": float(
733                     balanced_accuracy_score(y[test_index], predictions)
734                 ),

```

```

735         "f1_macro": float(f1_score(y[test_index], predictions, average="macro")),
736         "f1_weighted": float(
737             f1_score(y[test_index], predictions, average="weighted")
738         ),
739         "roc_auc": auc,
740     }
741     rows.append(fold_row)
742     fold_rows.append(fold_row)
743
744     fold_metrics = pd.DataFrame(fold_rows)
745     for summary_name, reducer in [("mean", np.nanmean), ("std", np.nanstd)]:
746         rows.append(
747             {
748                 "model": model_name,
749                 "fold": summary_name,
750                 "accuracy": float(reducer(fold_metrics["accuracy"])),
751                 "balanced_accuracy": float(
752                     reducer(fold_metrics["balanced_accuracy"])
753                 ),
754                 "f1_macro": float(reducer(fold_metrics["f1_macro"])),
755                 "f1_weighted": float(reducer(fold_metrics["f1_weighted"])),
756                 "roc_auc": float(reducer(fold_metrics["roc_auc"])),
757             }
758         )
759
760     classifier_metrics = pd.DataFrame(rows)
761     write_csv(
762         classifier_metrics,
763         RESULTS_DIR / "stage5_cluster_label_classifier_metrics.csv",
764     )
765     return classifier_metrics
766
767
768 def markdown_table(df: pd.DataFrame, columns: list[str], max_rows: int | None = None) -> str:
769     table = df.loc[:, columns].copy()
770     table.columns = [str(column) for column in table.columns]
771     if max_rows is not None:
772         table = table.head(max_rows)
773     for column in table.columns:
774         if pd.api.types.is_float_dtype(table[column]):
775             table[column] = table[column].map(
776                 lambda value: "" if pd.isna(value) else f"{value:.4f}"
777             )
778     header = "| " + " | ".join(table.columns) + " |"
779     separator = "| " + " | ".join(["---" * len(table.columns)]) + " |"
780     rows = [
781         "| " + " | ".join(str(value) for value in row) + " |"
782         for row in table.to_numpy()
783     ]
784     return "\n".join([header, separator, *rows])
785
786
787 def report_selection_reason(metrics: pd.DataFrame, recommended: pd.Series) -> str:
788     high_metric = metrics.sort_values("selection_score", ascending=False).head(3)
789     return (
790         f"主推荐方案为 {recommended['feature_set']} / {recommended['representation']} / "
791         f"{recommended['clustering_method']}`。该方案最小簇比例为 "
792         f"{recommended['smaller_cluster_ratio']:.3f}，稳定性 ARI 均值为 "
793         f"{recommended['stability_ari_mean']:.3f}，在聚类质量、簇均衡性、稳定性与解释性之间取得"
794         f"较好平衡。Top20/Top50 候选优先于 all_acoustic，因为其特征来源于 Stage 4 候选生物标志物，"
795         f"更适合论文解释；all_acoustic 结果作为参考，避免把高维噪声依赖方案作为默认主结论。\\n\\n"
796         f"选择时没有只看单一指标。若某些候选方案单项 silhouette 或 Calinski-Harabasz 较高，"
797         f"但簇规模过度失衡、稳定性较弱、或依赖更高维声学空间，则不作为主推荐方案。\\n\\n"
798         f"Selection score 前三候选: \\n\\n"
799         + markdown_table(
800             high_metric,
801             [
802                 "feature_set",
803                 "representation",
804                 "clustering_method",
805                 "selection_score",
806                 "smaller_cluster_ratio",
807                 "silhouette",
808                 "stability_ari_mean",
809             ],
810         )
811     )
812
813
814 def write_report(
815     subject_table: pd.DataFrame,
816     pd_subjects: pd.DataFrame,
817     metrics: pd.DataFrame,
818     assignments: pd.DataFrame,
819     top_features: pd.DataFrame,
820     classifier_metrics: pd.DataFrame,
821 ) -> None:
822     recommended = metrics.loc[metrics["recommended_flag"]].iloc[0]
823     cluster_counts = assignments["cluster"].value_counts().sort_index()
824     cluster_ratio = (cluster_counts / cluster_counts.sum()).round(4)
825     sex_table = pd.crosstab(assignments["cluster"], assignments["sex_male"], margins=True)
826     category_counts = top_features["category"].value_counts().reset_index()

```

```

827 category_counts.columns = ["category", "top_feature_count"]
828 classifier_summary = classifier_metrics[classifier_metrics["fold"].isin(["mean", "std"])]
829
830 required_boundary_text = (
831     " 由于附件数据未提供运动型与非运动型的真实临床标签，本文不直接构建监督分类模型，"
832     " 而是在帕金森病受试者内部进行无监督二类聚类，以获得基于语音特征的潜在分型。"
833     " 随后训练的分类器预测的是聚类标签，而非临床确诊标签。因此，该模型只能作为"
834     " 运动型/非运动型辅助诊断的探索性参考，仍需真实临床症状标签进一步验证。"
835 )
836
837 report = [
838     "# Stage 5 Dataset 1 PD Two-Cluster Voice Phenotyping",
839     "",
840     "## 1. Stage 5 目标与边界",
841     "本阶段只在 Dataset 1 的 PD 受试者内部进行 k=2 无监督聚类，目标是得到基于语音特征的二类潜在表型，并训练一个预测聚类标签的"
842     "↪ cluster-label classifier。",
843     "所有结论均为探索性辅助分型，不是临床确诊结果。",
844     "",
845     "## 2. 为什么不能做真实运动型/非运动型监督诊断",
846     required_boundary_text,
847     "",
848     "## 3. 输入文件与样本审计",
849     f"- 输入 subject-level 文件: `{rel(SUBJECT_TABLE_PATH)}`",
850     f"- 总受试者数: `{subject_table.shape[0]}`",
851     f"- PD 受试者数: `{pd_subjects.shape[0]}`",
852     f"- class 分布: `{subject_table['class'].value_counts().sort_index().to_dict()}`",
853     f"- 缺失值总数: `{int(subject_table.isna().sum().sum())}`",
854     f"- duplicated id 数: `{int(subject_table['id'].duplicated().sum())}`",
855     "",
856     "## 4. 特征集与预处理",
857     "使用 Stage 4 的 Top20、Top50 与 all_acoustic 三套特征集。所有候选方案先执行 StandardScaler; PCA 表示均在标准化之后生成。"
858     "↪ ",
859     "`id`, `gender`, `class`, `sex_male`, `record_count` 及任何标签或后续生成列均不进入聚类输入。`sex_male` 仅用于事后描述。"
860     "↪ ",
861     "",
862     "## 5. 聚类候选方案完整指标表",
863     markdown_table(
864         metrics,
865         [
866             "feature_set",
867             "representation",
868             "clustering_method",
869             "cluster_0_size",
870             "cluster_1_size",
871             "smaller_cluster_ratio",
872             "silhouette",
873             "calinski_harabasz",
874             "davies_bouldin",
875             "stability_ari_mean",
876             "recommended_flag",
877         ],
878     ),
879     "",
880     "## 6. 稳定性分析结果",
881     f"每个候选方案执行 `{STABILITY_RUNS}` 次 80% 受试者重采样，重新标准化、PCA 与聚类，并在可比较样本上计算 Adjusted Rand"
882     "↪ Index。",
883     "稳定性均值和标准差已写入 `results/stage5_clustering_metrics.csv`。",
884     "",
885     "## 7. 主推荐方案选择理由",
886     report_selection_reason(metrics, recommended),
887     "",
888     "## 8. 主方案两簇人数与比例",
889     markdown_table(
890         pd.DataFrame(
891             {
892                 "cluster": cluster_counts.index,
893                 "n_subjects": cluster_counts.values,
894                 "ratio": cluster_ratio.values,
895             }
896         ),
897         ["cluster", "n_subjects", "ratio"],
898     ),
899     "",
900     "## 9. 两簇 Top 区分语音特征",
901     markdown_table(
902         top_features,
903         [
904             "feature",
905             "category",
906             "zmean_difference_cluster1_minus_cluster0",
907             "cohens_d_cluster1_minus_cluster0",
908             "q_mannwhitney",
909             "direction",
910         ],
911     ),
912     "",
913     "## 10. 特征类别分布与解释",
914     markdown_table(category_counts, ["category", "top_feature_count"]),
915     "Top 区分特征类别分布用于描述两个潜在语音表型的声学差异来源，不构成临床病因解释。",
916     "",
917     "## 11. 性别分布的事后描述",
918     markdown_table(sex_table.reset_index(), list(sex_table.reset_index().columns)),

```

```

915     "`sex_male` 未进入聚类或 cluster-label classifier, 只作为事后描述, 不能作因果解释。",
916     "",
917     "## 12. cluster-label classifier 结果",
918     markdown_table(
919         classifier_summary,
920         [
921             "model",
922             "fold",
923             "accuracy",
924             "balanced_accuracy",
925             "f1_macro",
926             "f1_weighted",
927             "roc_auc",
928         ],
929     ),
930     "该分类器学习的是无监督聚类标签, 用于将潜在语音表型分型规则模型化; 它不是训练自真实临床运动型/非运动型标签,
    ↳ 因此不能解释为真实临床症状分类性能。",
931     "",
932     "## 13. 局限性",
933     "- 聚类标签来自语音特征内部结构, 没有外部临床症状标签验证。",
934     "- Mann-Whitney U 与 FDR 是聚类后的描述性比较, 不是外部验证假设检验, 也不是临床显著性证据。",
935     "- cluster 编号本身没有临床含义; 任何 potential motor-like voice phenotype 或 potential nonmotor-like voice phenotype
    ↳ 命名都只是解释性命名。",
936     "- all_acoustic 高维结果可能受冗余与噪声影响, 因此不优先作为论文主解释方案。",
937     "",
938     "## 14. 论文建议表述",
939     "建议写作: 本文在 PD 受试者内部基于 Stage 4 筛选的语音特征进行 k=2 无监督聚类, 得到两个基于语音特征的潜在表型簇,
    ↳ 并进一步训练 cluster-label classifier 以复现该探索性分型规则。",
940     "不建议写作: 将本阶段结果表述为真实临床亚型诊断性能。",
941     "",
942     "## 输出文件",
943     "- `results/stage5_pd_subject_clustering_input_audit.csv`",
944     "- `results/stage5_clustering_metrics.csv`",
945     "- `results/stage5_cluster_assignments_dataset1_pd.csv`",
946     "- `results/stage5_cluster_feature_differences.csv`",
947     "- `results/stage5_cluster_interpretation_top_features.csv`",
948     "- `results/stage5_cluster_label_classifier_metrics.csv`",
949     "- `results/stage5_two_cluster_report.md`",
950     "- `results/figures/stage5_pca_k2_clusters_top20.png`",
951     "- `results/figures/stage5_pca_k2_clusters_top50.png`",
952     "- `results/figures/stage5_cluster_size_comparison.png`",
953     "- `results/figures/stage5_cluster_feature_difference_top20.png`",
954 ]
955 (RESULTS_DIR / "stage5_two_cluster_report.md").write_text(
956     "\n".join(report),
957     encoding="utf-8",
958 )
959
960
961 def main() -> None:
962     RESULTS_DIR.mkdir(exist_ok=True)
963     FIGURES_DIR.mkdir(exist_ok=True)
964
965     subject_table, top20_features, top50_features = load_inputs()
966     all_features = acoustic_columns(subject_table)
967     pd_subjects = validate_pd_subjects(subject_table)
968     write_input_audit(
969         subject_table,
970         pd_subjects,
971         all_features,
972         top20_features,
973         top50_features,
974     )
975
976     feature_sets = {
977         "top20_biomarkers": top20_features,
978         "top50_biomarkers": top50_features,
979         "all_acoustic": all_features,
980     }
981     metrics, artifacts = run_clustering_candidates(pd_subjects, feature_sets)
982     metrics, recommended_key = choose_recommended(metrics)
983     write_csv(metrics, RESULTS_DIR / "stage5_clustering_metrics.csv")
984
985     recommended_artifact = artifacts[recommended_key]
986     labels = np.asarray(recommended_artifact["labels"], dtype=int)
987     assignments = write_cluster_assignments(
988         pd_subjects,
989         recommended_key,
990         recommended_artifact,
991     )
992
993     differences, top_features = cluster_feature_differences(
994         pd_subjects,
995         labels,
996         recommended_artifact["features"],
997     )
998     classifier_metrics = run_cluster_label_classifier(
999         pd_subjects,
1000         recommended_artifact["features"],
1001         labels,
1002     )
1003

```

```

1004     plot_pca_clusters(
1005         pd_subjects,
1006         top20_features,
1007         labels,
1008         FIGURES_DIR / "stage5_pca_k2_clusters_top20.png",
1009         "Stage 5 Main Cluster Labels Projected on Top20 Biomarker PCA",
1010     )
1011     plot_pca_clusters(
1012         pd_subjects,
1013         top50_features,
1014         labels,
1015         FIGURES_DIR / "stage5_pca_k2_clusters_top50.png",
1016         "Stage 5 Main Cluster Labels Projected on Top50 Biomarker PCA",
1017     )
1018     plot_cluster_sizes(metrics)
1019     plot_feature_difference_top20(top_features)
1020
1021     write_report(
1022         subject_table,
1023         pd_subjects,
1024         metrics,
1025         assignments,
1026         top_features,
1027         classifier_metrics,
1028     )
1029
1030
1031 if __name__ == "__main__":
1032     main()

```

A.5 代码五：六类潜在语音表型建模

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Any
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import kruskal
14 from sklearn.cluster import AgglomerativeClustering, KMeans
15 from sklearn.decomposition import PCA
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.linear_model import LogisticRegression
18 from sklearn.metrics import (
19     accuracy_score,
20     adjusted_rand_score,
21     balanced_accuracy_score,
22     calinski_harabasz_score,
23     confusion_matrix,
24     davies_bouldin_score,
25     f1_score,
26     normalized_mutual_info_score,
27     precision_score,
28     recall_score,
29     roc_auc_score,
30     silhouette_score,
31 )
32 from sklearn.mixture import GaussianMixture
33 from sklearn.model_selection import StratifiedKFold
34 from sklearn.pipeline import Pipeline
35 from sklearn.preprocessing import StandardScaler
36 from sklearn.svm import SVC
37
38
39 ROOT = Path(__file__).resolve().parents[1]
40 RESULTS_DIR = ROOT / "results"
41 FIGURES_DIR = RESULTS_DIR / "figures"
42
43 SUBJECT_TABLE_PATH = RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv"
44 TOP20_PATH = RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv"
45 TOP50_PATH = RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv"
46 RANK_SUMMARY_PATH = RESULTS_DIR / "stage4_biomarker_rank_summary_dataset1.csv"
47 STAGE5_ASSIGNMENTS_PATH = RESULTS_DIR / "stage5_cluster_assignments_dataset1_pd.csv"
48 STAGE5_DIFFERENCES_PATH = RESULTS_DIR / "stage5_cluster_feature_differences.csv"
49
50 OUTPUT_CSV_ENCODING = "utf-8-sig"
51 RANDOM_STATE = 42
52 EXPECTED_SUBJECTS = 252
53 EXPECTED_PD_SUBJECTS = 188
54 EXPECTED_ACOUSTIC_FEATURES = 752
55 PD_LABEL = 1
56 MAIN_K = 6
57 STABILITY_RUNS = 50
58 STABILITY_SAMPLE_FRACTION = 0.8
59
60 METADATA_COLUMNS = {"id", "gender", "class", "sex_male", "record_count"}
61 LABEL_COLUMN_TOKENS = ("cluster", "label", "prediction", "phenotype")
62 METHODS = ["KMeans", "GaussianMixture", "AgglomerativeClustering"]
63 FEATURE_SET_REPRESENTATIONS = {
64     "top20_biomarkers": ["scaled_only", "pca_5", "pca_10"],
65     "top50_biomarkers": ["pca_10", "pca_90"],
66     "all_acoustic": ["pca_20", "pca_90"],
67 }
68 SAFE_LABEL_SUFFIXES = ["A", "B", "C", "D", "E", "F"]
69
70
71 def rel(path: Path) -> str:
72     return path.relative_to(ROOT).as_posix()
73
74
75 def write_csv(df: pd.DataFrame, path: Path) -> None:
76     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
77
78
79 def require_file(path: Path) -> None:
80     if not path.exists():
81         raise FileNotFoundError(f"Missing required input file: {path}")
82
83
84 def is_forbidden_feature_column(column: str) -> bool:
85     name = column.lower()
86     return column in METADATA_COLUMNS or any(token in name for token in LABEL_COLUMN_TOKENS)
87
88
89 def categorize_feature(feature: str) -> str:
90     name = feature.lower()
```

```

91     if "tqwt" in name:
92         return "TQWT"
93     if any(token in name for token in ["wavelet", "_lt_", "ed_", "ed2_", "entropy"]):
94         return "Wavelet"
95     if "jitter" in name:
96         return "Jitter"
97     if "shimmer" in name or "shim_" in name or "apq" in name:
98         return "Shimmer"
99     if "hnr" in name or "harmonicity" in name or "harmtonoise" in name:
100         return "HNR / Harmonicity"
101     if "intensity" in name:
102         return "Intensity"
103     if (
104         name.startswith("f")
105         and len(name) == 2
106         and name[1].isdigit()
107         or name.startswith("b")
108         and len(name) == 2
109         and name[1].isdigit()
110     ):
111         return "Formant / Bandwidth"
112     if "mfcc" in name:
113         return "MFCC"
114     if "delta_delta" in name or "delta" in name:
115         return "Delta / Delta-delta"
116     if any(token in name for token in ["rpde", "dfa", "ppe", "gne"]):
117         return "Nonlinear"
118     return "Other"
119
120
121 def load_inputs() -> tuple[pd.DataFrame, list[str], list[str], pd.DataFrame, pd.DataFrame, pd.DataFrame]:
122     required_paths = [
123         SUBJECT_TABLE_PATH,
124         TOP20_PATH,
125         TOP50_PATH,
126         RANK_SUMMARY_PATH,
127         STAGE5_ASSIGNMENTS_PATH,
128         STAGE5_DIFFERENCES_PATH,
129     ]
130     for path in required_paths:
131         require_file(path)
132
133     subject_table = pd.read_csv(SUBJECT_TABLE_PATH)
134     top20 = pd.read_csv(TOP20_PATH)
135     top50 = pd.read_csv(TOP50_PATH)
136     rank_summary = pd.read_csv(RANK_SUMMARY_PATH)
137     stage5_assignments = pd.read_csv(STAGE5_ASSIGNMENTS_PATH)
138     stage5_differences = pd.read_csv(STAGE5_DIFFERENCES_PATH)
139
140     required_subject_columns = {"id", "class", "sex_male"}
141     missing = sorted(required_subject_columns - set(subject_table.columns))
142     if missing:
143         raise ValueError(f"Subject-level table missing required columns: {missing}")
144     if subject_table.shape[0] != EXPECTED_SUBJECTS:
145         raise ValueError(f"Expected {EXPECTED_SUBJECTS} subjects, found {subject_table.shape[0]}")
146     duplicated_id_count = int(subject_table["id"].duplicated().sum())
147     if duplicated_id_count:
148         raise ValueError(f"Subject-level table must have one row per subject; duplicated ids={duplicated_id_count}")
149
150     for label, table, expected_rows in [("Top20", top20, 20), ("Top50", top50, 50)]:
151         if "feature" not in table.columns:
152             raise ValueError(f"{label} file is missing the required 'feature' column")
153         if table.shape[0] != expected_rows:
154             raise ValueError(f"{label} expected {expected_rows} rows, found {table.shape[0]}")
155
156     required_stage5_columns = {"id", "cluster"}
157     missing_stage5 = sorted(required_stage5_columns - set(stage5_assignments.columns))
158     if missing_stage5:
159         raise ValueError(f"Stage 5 assignments missing required columns: {missing_stage5}")
160
161     top20_features = top20["feature"].astype(str).tolist()
162     top50_features = top50["feature"].astype(str).tolist()
163     missing_top20 = sorted(set(top20_features) - set(subject_table.columns))
164     missing_top50 = sorted(set(top50_features) - set(subject_table.columns))
165     if missing_top20 or missing_top50:
166         raise ValueError(
167             "Stage 4 biomarker features missing from subject-level table: "
168             f"top20={missing_top20}, top50={missing_top50}"
169         )
170
171     return subject_table, top20_features, top50_features, rank_summary, stage5_assignments, stage5_differences
172
173
174 def acoustic_columns(subject_table: pd.DataFrame) -> list[str]:
175     columns = [
176         column
177         for column in subject_table.select_dtypes(include="number").columns
178         if not is_forbidden_feature_column(column)
179     ]
180     if len(columns) != EXPECTED_ACOUSTIC_FEATURES:
181         raise ValueError(f"Expected {EXPECTED_ACOUSTIC_FEATURES} acoustic features, found {len(columns)}")
182     return columns

```



```

183
184
185 def validate_pd_subjects(subject_table: pd.DataFrame, stage5_assignments: pd.DataFrame) -> pd.DataFrame:
186     pd_subjects = subject_table.loc[subject_table["class"] == PD_LABEL].copy()
187     if pd_subjects.shape[0] != EXPECTED_PD_SUBJECTS:
188         raise ValueError(f"Expected {EXPECTED_PD_SUBJECTS} PD subjects, found {pd_subjects.shape[0]}")
189     overlap = set(pd_subjects["id"]).intersection(set(stage5_assignments["id"]))
190     if len(overlap) != EXPECTED_PD_SUBJECTS:
191         raise ValueError(f"Stage 5 assignments overlap {len(overlap)} PD subjects, expected {EXPECTED_PD_SUBJECTS}")
192     return pd_subjects
193
194
195 def write_input_audit(
196     subject_table: pd.DataFrame,
197     pd_subjects: pd.DataFrame,
198     all_features: list[str],
199     top20_features: list[str],
200     top50_features: list[str],
201     stage5_assignments: pd.DataFrame,
202 ) -> None:
203     excluded_columns = [
204         column if column in subject_table.columns else f"{column} (not present)"
205         for column in ["id", "gender", "class", "sex_male", "record_count"]
206     ]
207     overlap = len(set(pd_subjects["id"]).intersection(set(stage5_assignments["id"])))
208     audit = pd.DataFrame(
209         [
210             {
211                 "input_file": rel(SUBJECT_TABLE_PATH),
212                 "total_subjects_before_filter": int(subject_table.shape[0]),
213                 "pd_subjects_after_filter": int(pd_subjects.shape[0]),
214                 "class_distribution_before_filter": str(
215                     {int(k): int(v) for k, v in subject_table["class"].value_counts().sort_index().items()}
216                 ),
217                 "class_distribution_after_filter": str(
218                     {int(k): int(v) for k, v in pd_subjects["class"].value_counts().sort_index().items()}
219                 ),
220                 "missing_value_count": int(subject_table.isna().sum().sum()),
221                 "duplicated_id_count": int(subject_table["id"].duplicated().sum()),
222                 "acoustic_feature_count": int(len(all_features)),
223                 "excluded_columns": "; ".join(excluded_columns + ["stage5_cluster", "stage6_cluster"]),
224                 "feature_sets_used": "top20_biomarkers; top50_biomarkers; all_acoustic",
225                 "top20_feature_count": int(len(top20_features)),
226                 "top50_feature_count": int(len(top50_features)),
227                 "all_acoustic_feature_count": int(len(all_features)),
228                 "stage5_assignment_file_used": rel(STAGE5_ASSIGNMENTS_PATH),
229                 "stage5_label_overlap_subject_count": int(overlap),
230             }
231         ]
232     )
233     write_csv(audit, RESULTS_DIR / "stage6_six_cluster_input_audit.csv")
234
235
236 def build_representation(data: pd.DataFrame, representation: str) -> tuple[np.ndarray, int, float]:
237     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
238     if representation == "scaled_only":
239         return scaled, scaled.shape[1], np.nan
240     if representation == "pca_90":
241         pca = PCA(n_components=0.90, random_state=RANDOM_STATE)
242     elif representation.startswith("pca_"):
243         pca = PCA(n_components=int(representation.split("_", maxsplit=1)[1]), random_state=RANDOM_STATE)
244     else:
245         raise ValueError(f"Unknown representation: {representation}")
246     projected = pca.fit_transform(scaled)
247     return projected, projected.shape[1], float(np.sum(pca.explained_variance_ratio_))
248
249
250 def cluster_labels(matrix: np.ndarray, method: str, k: int, random_state: int = RANDOM_STATE) -> np.ndarray:
251     if method == "KMeans":
252         return KMeans(n_clusters=k, n_init=100, random_state=random_state).fit_predict(matrix)
253     if method == "GaussianMixture":
254         return GaussianMixture(
255             n_components=k,
256             covariance_type="full",
257             reg_covar=1e-6,
258             random_state=random_state,
259         ).fit_predict(matrix)
260     if method == "AgglomerativeClustering":
261         return AgglomerativeClustering(n_clusters=k, linkage="ward").fit_predict(matrix)
262     raise ValueError(f"Unknown clustering method: {method}")
263
264
265 def clustering_quality(matrix: np.ndarray, labels: np.ndarray) -> tuple[float, float, float, str]:
266     if np.unique(labels).size < 2:
267         return np.nan, np.nan, np.nan, "single cluster produced"
268     try:
269         return (
270             float(silhouette_score(matrix, labels)),
271             float(calinski_harabasz_score(matrix, labels)),
272             float(davies_bouldin_score(matrix, labels)),
273             "",
274         )

```

```

275     except ValueError as exc:
276         return np.nan, np.nan, np.nan, f"metric calculation failed: {exc}"
277
278
279 def stability_analysis(
280     data: pd.DataFrame,
281     representation: str,
282     method: str,
283     k: int,
284     full_labels: np.ndarray,
285     n_runs: int = STABILITY_RUNS,
286 ) -> tuple[float, float]:
287     rng = np.random.default_rng(RANDOM_STATE + k)
288     n_subjects = data.shape[0]
289     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
290     scores: list[float] = []
291     for run_index in range(n_runs):
292         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
293         sample_matrix, _, _ = build_representation(data.iloc[sample_index], representation)
294         sample_labels = cluster_labels(sample_matrix, method, k, random_state=RANDOM_STATE + run_index + 1)
295         scores.append(float(adjusted_rand_score(full_labels[sample_index], sample_labels)))
296     return float(np.mean(scores)), float(np.std(scores, ddof=1))
297
298
299 def cluster_size_stats(labels: np.ndarray, k: int) -> dict[str, Any]:
300     counts = pd.Series(labels).value_counts().reindex(range(k), fill_value=0).astype(int)
301     min_size = int(counts.min())
302     max_size = int(counts.max())
303     min_ratio = float(min_size / len(labels))
304     max_ratio = float(max_size / len(labels))
305     return {
306         "cluster_sizes": ";".join(f"{cluster}:{int(size)}" for cluster, size in counts.items()),
307         "min_cluster_size": min_size,
308         "max_cluster_size": max_size,
309         "min_cluster_ratio": min_ratio,
310         "max_cluster_ratio": max_ratio,
311         "size_imbalance_ratio": float(max_size / min_size) if min_size > 0 else np.inf,
312     }
313
314
315 def run_clustering_candidates(
316     pd_subjects: pd.DataFrame,
317     feature_sets: dict[str, list[str]],
318     k: int = MAIN_K,
319 ) -> tuple[pd.DataFrame, dict[tuple[str, str, str, int], dict[str, Any]]]:
320     rows: list[dict[str, Any]] = []
321     artifacts: dict[tuple[str, str, str, int], dict[str, Any]] = {}
322
323     for feature_set, features in feature_sets.items():
324         data = pd_subjects[features]
325         for representation in FEATURE_SET_REPRESENTATIONS[feature_set]:
326             matrix, n_components, explained_variance = build_representation(data, representation)
327             for method in METHODS:
328                 labels = cluster_labels(matrix, method, k)
329                 size_stats = cluster_size_stats(labels, k)
330                 silhouette, calinski, davies, metric_warning = clustering_quality(matrix, labels)
331                 stability_mean, stability_std = stability_analysis(data, representation, method, k, labels)
332
333                 warnings = []
334                 if metric_warning:
335                     warnings.append(metric_warning)
336                 if size_stats["min_cluster_size"] < 8:
337                     warnings.append("min_cluster_size < 8; not eligible for recommendation")
338                 if size_stats["min_cluster_ratio"] < 0.05:
339                     warnings.append("min_cluster_ratio < 0.05; not eligible for recommendation")
340
341                 key = (feature_set, representation, method, k)
342                 rows.append(
343                     {
344                         "feature_set": feature_set,
345                         "representation": representation,
346                         "clustering_method": method,
347                         "k": int(k),
348                         "n_subjects": int(len(labels)),
349                         "n_features_original": int(len(features)),
350                         "n_components_after_pca": int(n_components),
351                         "pca_explained_variance_sum": explained_variance,
352                         **size_stats,
353                         "silhouette": silhouette,
354                         "calinski_harabasz": calinski,
355                         "davies_bouldin": davies,
356                         "stability_ari_mean": stability_mean,
357                         "stability_ari_std": stability_std,
358                         "stability_n_runs": STABILITY_RUNS,
359                         "recommended_flag": False,
360                         "exclusion_or_warning": "; ".join(warnings),
361                     }
362                 )
363                 artifacts[key] = {
364                     "labels": labels,
365                     "matrix": matrix,
366                     "features": features,

```

```

367         "n_components_after_pca": n_components,
368         "pca_explained_variance_sum": explained_variance,
369     }
370     return pd.DataFrame(rows), artifacts
371
372
373 def minmax(series: pd.Series, higher_is_better: bool = True) -> pd.Series:
374     valid = series.astype(float).replace([np.inf, -np.inf], np.nan)
375     min_value = valid.min(skipna=True)
376     max_value = valid.max(skipna=True)
377     if pd.isna(min_value) or pd.isna(max_value) or max_value == min_value:
378         return pd.Series(0.5, index=series.index)
379     scaled = (valid - min_value) / (max_value - min_value)
380     if not higher_is_better:
381         scaled = 1 - scaled
382     return scaled.fillna(0.0)
383
384
385 def stage5_alignment_score(labels: np.ndarray, stage5_labels: np.ndarray) -> float:
386     nmi = normalized_mutual_info_score(stage5_labels, labels)
387     return float(nmi)
388
389
390 def choose_recommended(
391     metrics: pd.DataFrame,
392     artifacts: dict[tuple[str, str, str, int], dict[str, Any]],
393     stage5_labels: np.ndarray,
394 ) -> tuple[pd.DataFrame, tuple[str, str, str, int]]:
395     out = metrics.copy()
396     alignments = []
397     for _, row in out.iterrows():
398         key = (row["feature_set"], row["representation"], row["clustering_method"], int(row["k"]))
399         alignments.append(stage5_alignment_score(artifacts[key]["labels"], stage5_labels))
400     out["stage5_nmi"] = alignments
401
402     eligible = (out["min_cluster_size"] >= 8) & (out["min_cluster_ratio"] >= 0.05)
403     if not eligible.any():
404         raise ValueError("No k=6 candidate is eligible after min cluster constraints")
405
406     balance = 1 - (out["max_cluster_ratio"] - out["min_cluster_ratio"]).clip(lower=0)
407     score = (
408         0.20 * minmax(out["silhouette"], True)
409         + 0.12 * minmax(out["calinski_harabasz"], True)
410         + 0.13 * minmax(out["davies_bouldin"], False)
411         + 0.30 * minmax(out["stability_ari_mean"], True)
412         + 0.15 * balance
413         + 0.10 * minmax(out["stage5_nmi"], True)
414     )
415     score += out["feature_set"].map(
416         {"top20_biomarkers": 0.04, "top50_biomarkers": 0.08, "all_acoustic": -0.03}
417     )
418     score = score.where(eligible, -np.inf)
419     out["selection_score"] = score
420     best_index = int(out["selection_score"].idxmax())
421     best_score = float(out.loc[best_index, "selection_score"])
422
423     interpretable_close = out[
424         (out["feature_set"].isin(["top20_biomarkers", "top50_biomarkers"]))
425         & eligible
426         & (out["selection_score"] >= best_score - 0.08)
427     ]
428     if not interpretable_close.empty and out.loc[best_index, "feature_set"] == "all_acoustic":
429         best_index = int(interpretable_close["selection_score"].idxmax())
430
431     top50_close = out[
432         (out["feature_set"] == "top50_biomarkers")
433         & eligible
434         & (out["selection_score"] >= float(out.loc[best_index, "selection_score"]) - 0.03)
435     ]
436     if not top50_close.empty and out.loc[best_index, "feature_set"] == "top20_biomarkers":
437         best_index = int(top50_close["selection_score"].idxmax())
438
439     out["recommended_flag"] = False
440     out.loc[best_index, "recommended_flag"] = True
441     key = (
442         str(out.loc[best_index, "feature_set"]),
443         str(out.loc[best_index, "representation"]),
444         str(out.loc[best_index, "clustering_method"]),
445         int(out.loc[best_index, "k"]),
446     )
447     return out, key
448
449
450 def run_k_scan(
451     pd_subjects: pd.DataFrame,
452     features: list[str],
453     feature_set: str,
454     representation: str,
455 ) -> pd.DataFrame:
456     rows: list[dict[str, Any]] = []
457     data = pd_subjects[features]
458     for k in range(2, 9):

```

```

459     matrix, n_components, explained_variance = build_representation(data, representation)
460     for method in METHODS:
461         labels = cluster_labels(matrix, method, k)
462         size_stats = cluster_size_stats(labels, k)
463         silhouette, calinski, davies, metric_warning = clustering_quality(matrix, labels)
464         stability_mean, stability_std = stability_analysis(data, representation, method, k, labels)
465         rows.append(
466             {
467                 "feature_set": feature_set,
468                 "representation": representation,
469                 "clustering_method": method,
470                 "k": int(k),
471                 "n_subjects": int(len(labels)),
472                 "n_features_original": int(len(features)),
473                 "n_components_after_pca": int(n_components),
474                 "pca_explained_variance_sum": explained_variance,
475                 **size_stats,
476                 "silhouette": silhouette,
477                 "calinski_harabasz": calinski,
478                 "davies_bouldin": davies,
479                 "stability_ari_mean": stability_mean,
480                 "stability_ari_std": stability_std,
481                 "stability_n_runs": STABILITY_RUNS,
482                 "exclusion_or_warning": metric_warning,
483             }
484         )
485     k_scan = pd.DataFrame(rows)
486     write_csv(k_scan, RESULTS_DIR / "stage6_k_scan_metrics.csv")
487     return k_scan
488
489 def consensus_kmeans_summary(data: pd.DataFrame, representation: str, labels: np.ndarray) -> dict[str, float]:
490     n_subjects = data.shape[0]
491     coassociation = np.zeros((n_subjects, n_subjects), dtype=float)
492     coobserved = np.zeros((n_subjects, n_subjects), dtype=float)
493     rng = np.random.default_rng(RANDOM_STATE + 600)
494     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
495     for run_index in range(STABILITY_RUNS):
496         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
497         sample_matrix, _, _ = build_representation(data.iloc[sample_index], representation)
498         sample_labels = cluster_labels(sample_matrix, "KMeans", MAIN_K, random_state=RANDOM_STATE + run_index + 1)
499         for i_pos, i_subject in enumerate(sample_index):
500             same = sample_labels == labels[sample_index[i_pos]]
501             coassociation[i_subject, sample_index[same]] += 1
502             coobserved[i_subject, sample_index[same]] += 1
503     with np.errstate(divide="ignore", invalid="ignore"):
504         consensus = np.divide(coassociation, coobserved, out=np.full_like(coassociation, np.nan), where=coobserved > 0)
505     same_mask = labels[:, None] == labels[None, :]
506     diff_mask = labels[:, None] != labels[None, :]
507     return {
508         "consensus_within_cluster_mean": float(np.nanmean(consensus[same_mask])),
509         "consensus_between_cluster_mean": float(np.nanmean(consensus[diff_mask])),
510         "consensus_separation": float(np.nanmean(consensus[same_mask]) - np.nanmean(consensus[diff_mask])),
511     }
512
513 def pca_2d_coordinates(data: pd.DataFrame) -> np.ndarray:
514     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
515     return PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(scaled)
516
517 def aligned_stage5(stage5_assignments: pd.DataFrame, pd_subjects: pd.DataFrame) -> pd.Series:
518     stage5 = stage5_assignments[["id", "cluster"]].rename(columns={"cluster": "stage5_cluster"})
519     merged = pd_subjects[["id"]].merge(stage5, on="id", how="left")
520     if merged["stage5_cluster"].isna().any():
521         raise ValueError("Missing Stage 5 cluster labels after aligning to PD subject order")
522     return merged["stage5_cluster"].astype(int)
523
524 def write_assignments(
525     pd_subjects: pd.DataFrame,
526     stage5_labels: pd.Series,
527     recommended_key: tuple[str, str, str, int],
528     artifact: dict[str, Any],
529 ) -> pd.DataFrame:
530     feature_set, representation, method, _ = recommended_key
531     labels = np.asarray(artifact["labels"], dtype=int)
532     counts = pd.Series(labels).value_counts().to_dict()
533     coords = pca_2d_coordinates(pd_subjects[artifact["features"]])
534     assignments = pd.DataFrame(
535         {
536             "id": pd_subjects["id"].to_numpy(),
537             "class": pd_subjects["class"].to_numpy(),
538             "sex_male": pd_subjects["sex_male"].to_numpy(),
539             "stage5_cluster": stage5_labels.to_numpy(),
540             "stage6_cluster": labels,
541             "stage6_cluster_size": [int(counts[label]) for label in labels],
542             "stage6_cluster_ratio": [float(counts[label] / len(labels)) for label in labels],
543             "feature_set": feature_set,
544             "representation": representation,
545             "clustering_method": method,
546             "pca_1": coords[:, 0],
547         }
548     )

```

```

551         "pca_2": coords[:, 1],
552     }
553 )
554 write_csv(assignments, RESULTS_DIR / "stage6_six_cluster_assignments_dataset1_pd.csv")
555 return assignments
556
557
558 def fdr_bh(p_values: np.ndarray) -> np.ndarray:
559     p = np.asarray(p_values, dtype=float)
560     adjusted = np.full(p.shape, np.nan, dtype=float)
561     valid = np.isfinite(p)
562     if not valid.any():
563         return adjusted
564     valid_p = p[valid]
565     order = np.argsort(valid_p)
566     ordered_p = valid_p[order]
567     n = len(ordered_p)
568     ordered_q = ordered_p * n / np.arange(1, n + 1)
569     ordered_q = np.minimum.accumulate(ordered_q[::-1])[:-1]
570     ordered_q = np.clip(ordered_q, 0, 1)
571     q = np.empty_like(ordered_q)
572     q[order] = ordered_q
573     adjusted[valid] = q
574     return adjusted
575
576
577 def cohens_d(values_1: np.ndarray, values_0: np.ndarray) -> float:
578     n_1 = len(values_1)
579     n_0 = len(values_0)
580     if n_1 < 2 or n_0 < 2:
581         return np.nan
582     var_1 = np.var(values_1, ddof=1)
583     var_0 = np.var(values_0, ddof=1)
584     pooled = ((n_1 - 1) * var_1 + (n_0 - 1) * var_0) / (n_1 + n_0 - 2)
585     if pooled <= 0:
586         return 0.0
587     return float((np.mean(values_1) - np.mean(values_0)) / np.sqrt(pooled))
588
589
590 def build_feature_profiles(
591     pd_subjects: pd.DataFrame,
592     labels: np.ndarray,
593     features: list[str],
594 ) -> tuple[pd.DataFrame, pd.DataFrame]:
595     raw = pd_subjects[features]
596     scaled = pd.DataFrame(
597         StandardScaler().fit_transform(raw.to_numpy(dtype=float)),
598         columns=features,
599         index=raw.index,
600     )
601     rows: list[dict[str, Any]] = []
602     for feature in features:
603         row: dict[str, Any] = {"feature": feature, "category": categorize_feature(feature)}
604         cluster_values = []
605         for cluster in range(MAIN_K):
606             values = raw.loc[labels == cluster, feature].to_numpy(dtype=float)
607             z_values = scaled.loc[labels == cluster, feature].to_numpy(dtype=float)
608             row[f"cluster_{cluster}_mean"] = float(np.mean(values))
609             row[f"cluster_{cluster}_zmean"] = float(np.mean(z_values))
610             cluster_values.append(values)
611         zmeans = np.array([row[f"cluster_{cluster}_zmean"] for cluster in range(MAIN_K)], dtype=float)
612         try:
613             h_stat, p_value = kruskal(*cluster_values)
614         except ValueError:
615             h_stat, p_value = np.nan, np.nan
616         row["max_abs_zmean_difference"] = float(np.max(zmeans) - np.min(zmeans))
617         row["kruskal_h"] = float(h_stat)
618         row["p_kruskal"] = float(p_value)
619         rows.append(row)
620
621     profiles = pd.DataFrame(rows)
622     profiles["q_kruskal"] = fdr_bh(profiles["p_kruskal"].to_numpy())
623     ordered_columns = (
624         ["feature", "category"]
625         + [f"cluster_{cluster}_mean" for cluster in range(MAIN_K)]
626         + [f"cluster_{cluster}_zmean" for cluster in range(MAIN_K)]
627         + ["max_abs_zmean_difference", "kruskal_h", "p_kruskal", "q_kruskal"]
628     )
629     profiles = profiles[ordered_columns].sort_values(
630         ["max_abs_zmean_difference", "q_kruskal"],
631         ascending=[False, True],
632     ).reset_index(drop=True)
633
634     top_rows: list[dict[str, Any]] = []
635     for cluster in range(MAIN_K):
636         other_mask = labels != cluster
637         cluster_mask = labels == cluster
638         for _, row in profiles.iterrows():
639             feature = str(row["feature"])
640             zmean = float(row[f"cluster_{cluster}_zmean"])
641             other_zmean = float(np.mean([row[f"cluster_{other}_zmean"] for other in range(MAIN_K) if other != cluster]))
642             contrast = zmean - other_zmean

```

```

643         d_value = cohens_d(
644             raw.loc[cluster_mask, feature].to_numpy(dtype=float),
645             raw.loc[other_mask, feature].to_numpy(dtype=float),
646         )
647         top_rows.append(
648             {
649                 "stage6_cluster": cluster,
650                 "feature": feature,
651                 "category": row["category"],
652                 "cluster_zmean": zmean,
653                 "other_clusters_mean_zmean": other_zmean,
654                 "one_vs_rest_zmean_difference": contrast,
655                 "one_vs_rest_cohens_d": d_value,
656                 "abs_one_vs_rest_zmean_difference": abs(contrast),
657                 "abs_one_vs_rest_cohens_d": abs(d_value) if np.isfinite(d_value) else np.nan,
658                 "q_kruskal": row["q_kruskal"],
659                 "direction": "positive" if contrast >= 0 else "negative",
660             }
661         )
662     top_by_cluster = (
663         pd.DataFrame(top_rows)
664         .sort_values(
665             ["stage6_cluster", "abs_one_vs_rest_zmean_difference", "abs_one_vs_rest_cohens_d", "q_kruskal"],
666             ascending=[True, False, False, True],
667         )
668         .groupby("stage6_cluster", group_keys=False)
669         .head(10)
670         .reset_index(drop=True)
671     )
672
673     write_csv(profiles, RESULTS_DIR / "stage6_six_cluster_feature_profiles.csv")
674     write_csv(top_by_cluster, RESULTS_DIR / "stage6_six_cluster_top_features_by_cluster.csv")
675     return profiles, top_by_cluster
676
677
678 def category_phrase(category: str) -> str:
679     mapping = {
680         "Delta / Delta-delta": "dynamic_delta",
681         "TQWT": "tqwt",
682         "MFCC": "mfcc",
683         "Shimmer": "shimmer",
684         "Jitter": "jitter",
685         "HNR / Harmonicity": "harmonicity",
686         "Nonlinear": "nonlinear",
687         "Intensity": "intensity",
688         "Formant / Bandwidth": "formant_bandwidth",
689         "Wavelet": "wavelet",
690         "Other": "mixed_acoustic",
691     }
692     return mapping.get(category, "mixed_acoustic")
693
694
695 def build_cluster_profiles(
696     assignments: pd.DataFrame,
697     top_by_cluster: pd.DataFrame,
698 ) -> pd.DataFrame:
699     rows: list[dict[str, Any]] = []
700     for cluster in range(MAIN_K):
701         cluster_assignments = assignments.loc[assignments["stage6_cluster"] == cluster]
702         cluster_features = top_by_cluster.loc[top_by_cluster["stage6_cluster"] == cluster]
703         category_counts = cluster_features["category"].value_counts()
704         dominant_categories = "; ".join(f"{cat}:{int(count)}" for cat, count in category_counts.items())
705         positives = cluster_features.loc[cluster_features["one_vs_rest_zmean_difference"] >= 0].head(5)
706         negatives = cluster_features.loc[cluster_features["one_vs_rest_zmean_difference"] < 0].head(5)
707         if positives.empty:
708             positives = cluster_features.sort_values("one_vs_rest_zmean_difference", ascending=False).head(5)
709         if negatives.empty:
710             negatives = cluster_features.sort_values("one_vs_rest_zmean_difference", ascending=True).head(5)
711         dominant_category = category_counts.index[0] if not category_counts.empty else "Other"
712         direction = "high" if positives["one_vs_rest_zmean_difference"].mean() >=
713             ↪ abs(negatives["one_vs_rest_zmean_difference"].mean()) else "low"
714         label =
715             ↪ f"latent_voice_phenotype_{SAFE_LABEL_SUFFIXES[cluster]}_{category_phrase(str(dominant_category))}_{direction}"
716         rows.append(
717             {
718                 "stage6_cluster": cluster,
719                 "n_subjects": int(cluster_assignments.shape[0]),
720                 "ratio": float(cluster_assignments.shape[0] / assignments.shape[0]),
721                 "dominant_feature_categories": dominant_categories,
722                 "top_positive_features": "; ".join(positives["feature"].astype(str).tolist()),
723                 "top_negative_features": "; ".join(negatives["feature"].astype(str).tolist()),
724                 "stage5_cluster_distribution": str(
725                     {int(k): int(v) for k, v in cluster_assignments["stage5_cluster"].value_counts().sort_index().items()}
726                 ),
727                 "sex_male_distribution": str(
728                     {int(k): int(v) for k, v in cluster_assignments["sex_male"].value_counts().sort_index().items()}
729                 ),
730                 "tentative_voice_phenotype_label": label,
731                 "interpretation_note": "Exploratory acoustic phenotype label; not a clinical symptom diagnosis.",
732             }
733         )
734     profiles = pd.DataFrame(rows)

```

```

733 write_csv(profiles, RESULTS_DIR / "stage6_six_cluster_profiles.csv")
734 return profiles
735
736
737 def stage5_crosswalk(assignments: pd.DataFrame) -> pd.DataFrame:
738     crosstab = pd.crosstab(assignments["stage5_cluster"], assignments["stage6_cluster"])
739     nmi = float(normalized_mutual_info_score(assignments["stage5_cluster"], assignments["stage6_cluster"]))
740     ari = float(adjusted_rand_score(assignments["stage5_cluster"], assignments["stage6_cluster"]))
741     majority_map = {}
742     for stage6_cluster in sorted(assignments["stage6_cluster"].unique()):
743         subset = assignments.loc[assignments["stage6_cluster"] == stage6_cluster, "stage5_cluster"]
744         majority_map[int(stage6_cluster)] = int(subset.value_counts().idxmax())
745     collapsed = assignments["stage6_cluster"].map(majority_map)
746     collapsed_ari = float(adjusted_rand_score(assignments["stage5_cluster"], collapsed))
747
748     rows: list[dict[str, Any]] = []
749     for stage5_cluster in crosstab.index:
750         for stage6_cluster in crosstab.columns:
751             count = int(crosstab.loc[stage5_cluster, stage6_cluster])
752             row_total = int(crosstab.loc[stage5_cluster].sum())
753             col_total = int(crosstab[stage6_cluster].sum())
754             rows.append(
755                 {
756                     "stage5_cluster": int(stage5_cluster),
757                     "stage6_cluster": int(stage6_cluster),
758                     "count": count,
759                     "row_percentage": float(count / row_total) if row_total else 0.0,
760                     "column_percentage": float(count / col_total) if col_total else 0.0,
761                     "adjusted_rand_index_stage5_vs_stage6": ari,
762                     "normalized_mutual_info_stage5_vs_stage6": nmi,
763                     "collapsed_stage6_to_stage5_majority_ari": collapsed_ari,
764                 }
765             )
766     crosswalk = pd.DataFrame(rows)
767     write_csv(crosswalk, RESULTS_DIR / "stage6_stage5_crosswalk.csv")
768     return crosswalk
769
770
771 def run_classifier(pd_subjects: pd.DataFrame, features: list[str], labels: np.ndarray) -> tuple[pd.DataFrame, pd.DataFrame]:
772     x = pd_subjects[features].to_numpy(dtype=float)
773     y = labels.astype(int)
774     min_cluster_size = int(pd.Series(y).value_counts().min())
775     n_splits = min(5, min_cluster_size)
776     if n_splits < 2:
777         raise ValueError("six-cluster-label classifier requires at least 2 samples per class")
778
779     cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=RANDOM_STATE)
780     models: dict[str, Any] = {
781         "Multinomial Logistic Regression": Pipeline(
782             [
783                 ("scaler", StandardScaler()),
784                 (
785                     "model",
786                     LogisticRegression(
787                         max_iter=5000,
788                         class_weight="balanced",
789                         solver="lbfgs",
790                         random_state=RANDOM_STATE,
791                     ),
792                 ),
793             ]
794         ),
795         "SVM-RBF": Pipeline(
796             [
797                 ("scaler", StandardScaler()),
798                 (
799                     "model",
800                     SVC(
801                         kernel="rbf",
802                         probability=True,
803                         class_weight="balanced",
804                         random_state=RANDOM_STATE,
805                     ),
806                 ),
807             ]
808         ),
809         "Random Forest": RandomForestClassifier(
810             n_estimators=500,
811             class_weight="balanced",
812             random_state=RANDOM_STATE,
813         ),
814     }
815
816     metric_rows: list[dict[str, Any]] = []
817     cm_rows: list[dict[str, Any]] = []
818     labels_order = list(range(MAIN_K))
819
820     for model_name, model in models.items():
821         fold_rows: list[dict[str, Any]] = []
822         for fold_index, (train_index, test_index) in enumerate(cv.split(x, y), start=1):
823             model.fit(x[train_index], y[train_index])
824             predictions = model.predict(x[test_index])

```

```

825         try:
826             probabilities = model.predict_proba(x[test_index])
827             if probabilities.shape[1] == MAIN_K and set(np.unique(y[test_index])) == set(labels_order):
828                 auc = float(roc_auc_score(y[test_index], probabilities, multi_class="ovr", labels=labels_order))
829             else:
830                 auc = np.nan
831         except (AttributeError, ValueError):
832             auc = np.nan
833         row = {
834             "model": model_name,
835             "fold": fold_index,
836             "n_splits": n_splits,
837             "accuracy": float(accuracy_score(y[test_index], predictions)),
838             "balanced_accuracy": float(balanced_accuracy_score(y[test_index], predictions)),
839             "f1_macro": float(f1_score(y[test_index], predictions, average="macro", zero_division=0)),
840             "f1_weighted": float(f1_score(y[test_index], predictions, average="weighted", zero_division=0)),
841             "precision_macro": float(precision_score(y[test_index], predictions, average="macro", zero_division=0)),
842             "recall_macro": float(recall_score(y[test_index], predictions, average="macro", zero_division=0)),
843             "multiclass_roc_auc_ovr": auc,
844         }
845         metric_rows.append(row)
846         fold_rows.append(row)
847         matrix = confusion_matrix(y[test_index], predictions, labels=labels_order)
848         for true_label in labels_order:
849             for predicted_label in labels_order:
850                 cm_rows.append(
851                     {
852                         "model": model_name,
853                         "fold": fold_index,
854                         "true_stage6_cluster": true_label,
855                         "predicted_stage6_cluster": predicted_label,
856                         "count": int(matrix[true_label, predicted_label]),
857                     }
858                 )
859         fold_metrics = pd.DataFrame(fold_rows)
860         for summary_name, reducer in [("mean", np.nanmean), ("std", np.nanstd)]:
861             metric_rows.append(
862                 {
863                     "model": model_name,
864                     "fold": summary_name,
865                     "n_splits": n_splits,
866                     "accuracy": float(reducer(fold_metrics["accuracy"])),
867                     "balanced_accuracy": float(reducer(fold_metrics["balanced_accuracy"])),
868                     "f1_macro": float(reducer(fold_metrics["f1_macro"])),
869                     "f1_weighted": float(reducer(fold_metrics["f1_weighted"])),
870                     "precision_macro": float(reducer(fold_metrics["precision_macro"])),
871                     "recall_macro": float(reducer(fold_metrics["recall_macro"])),
872                     "multiclass_roc_auc_ovr": float(reducer(fold_metrics["multiclass_roc_auc_ovr"])),
873                 }
874             )
875         metrics = pd.DataFrame(metric_rows)
876         confusion = pd.DataFrame(cm_rows)
877         write_csv(metrics, RESULTS_DIR / "stage6_six_cluster_label_classifier_metrics.csv")
878         write_csv(confusion, RESULTS_DIR / "stage6_six_cluster_label_classifier_confusion_matrices.csv")
879         return metrics, confusion
880
881
882
883 def markdown_table(df: pd.DataFrame, columns: list[str], max_rows: int | None = None) -> str:
884     table = df.loc[:, columns].copy()
885     table.columns = [str(column) for column in table.columns]
886     if max_rows is not None:
887         table = table.head(max_rows)
888     for column in table.columns:
889         if pd.api.types.is_float_dtype(table[column]):
890             table[column] = table[column].map(lambda value: "" if pd.isna(value) else f"{value:.4f}")
891     header = "| " + " | ".join(table.columns) + " |"
892     separator = "| " + " | ".join(["---"] * len(table.columns)) + " |"
893     rows = ["| " + " | ".join(str(value) for value in row) + " |" for row in table.to_numpy()]
894     return "\n".join([header, separator, *rows])
895
896
897 def plot_pca_clusters(assignments: pd.DataFrame) -> None:
898     fig, ax = plt.subplots(figsize=(8, 6))
899     for cluster in sorted(assignments["stage6_cluster"].unique()):
900         subset = assignments.loc[assignments["stage6_cluster"] == cluster]
901         ax.scatter(subset["pca_1"], subset["pca_2"], s=44, alpha=0.85, label=f"Cluster {cluster}")
902     ax.set_title("Stage 6 Main k=6 Latent Voice Phenotype Clusters")
903     ax.set_xlabel("PCA 1")
904     ax.set_ylabel("PCA 2")
905     ax.legend(title="Stage 6 cluster", ncol=2)
906     fig.tight_layout()
907     fig.savefig(FIGURES_DIR / "stage6_pca_k6_clusters_main.png", dpi=220)
908     plt.close(fig)
909
910
911 def plot_cluster_size(assignments: pd.DataFrame) -> None:
912     counts = assignments["stage6_cluster"].value_counts().sort_index()
913     ratios = counts / counts.sum()
914     fig, ax = plt.subplots(figsize=(8, 5))
915     bars = ax.bar(counts.index.astype(str), counts.values, color="#4c78a8")
916     for bar, ratio in zip(bars, ratios):

```



```

917         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), f"{ratio:.1%}", ha="center", va="bottom")
918     ax.set_title("Stage 6 Cluster Size Distribution")
919     ax.set_xlabel("Stage 6 cluster")
920     ax.set_ylabel("Number of PD subjects")
921     fig.tight_layout()
922     fig.savefig(FIGURES_DIR / "stage6_cluster_size_distribution.png", dpi=220)
923     plt.close(fig)
924
925
926 def plot_candidate_metrics(metrics: pd.DataFrame) -> None:
927     plot_data = metrics.copy()
928     plot_data["candidate"] = (
929         plot_data["feature_set"] + "\n" + plot_data["representation"] + "\n" + plot_data["clustering_method"]
930     )
931     fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)
932     for ax, metric, title in [
933         (axes[0], "silhouette", "Silhouette"),
934         (axes[1], "davies_bouldin", "Davies-Bouldin"),
935         (axes[2], "stability_ari_mean", "Stability ARI Mean"),
936     ]:
937         colors = np.where(plot_data["recommended_flag"], "#d55e00", "#4c78a8")
938         ax.bar(np.arange(plot_data.shape[0]), plot_data[metric], color=colors)
939         ax.set_ylabel(title)
940         ax.set_title(f"Stage 6 Candidate {title}")
941     axes[-1].set_xticks(np.arange(plot_data.shape[0]))
942     axes[-1].set_xticklabels(plot_data["candidate"], rotation=75, ha="right", fontsize=7)
943     fig.tight_layout()
944     fig.savefig(FIGURES_DIR / "stage6_candidate_metrics_comparison.png", dpi=220)
945     plt.close(fig)
946
947
948 def plot_feature_heatmap(feature_profiles: pd.DataFrame) -> None:
949     top = feature_profiles.head(20)
950     matrix = top[[f"cluster_{cluster}_zmean" for cluster in range(MAIN_K)]].to_numpy(dtype=float)
951     fig, ax = plt.subplots(figsize=(9, 8))
952     image = ax.imshow(matrix, aspect="auto", cmap="coolwarm")
953     ax.set_yticks(np.arange(top.shape[0]))
954     ax.set_yticklabels(top["feature"], fontsize=7)
955     ax.set_xticks(np.arange(MAIN_K))
956     ax.set_xticklabels([f"Cluster {cluster}" for cluster in range(MAIN_K)])
957     ax.set_title("Stage 6 Cluster z-Mean Heatmap for Top Features")
958     ax.set_xlabel("Stage 6 cluster")
959     ax.set_ylabel("Feature")
960     fig.colorbar(image, ax=ax, label="Cluster z-mean")
961     fig.tight_layout()
962     fig.savefig(FIGURES_DIR / "stage6_cluster_feature_heatmap.png", dpi=220)
963     plt.close(fig)
964
965
966 def plot_crosswalk_heatmap(crosswalk: pd.DataFrame) -> None:
967     pivot = crosswalk.pivot(index="stage5_cluster", columns="stage6_cluster", values="count").fillna(0)
968     fig, ax = plt.subplots(figsize=(8, 4))
969     image = ax.imshow(pivot.to_numpy(dtype=float), aspect="auto", cmap="Blues")
970     ax.set_xticks(np.arange(pivot.shape[1]))
971     ax.set_xticklabels([f"Stage 6 {column}" for column in pivot.columns])
972     ax.set_yticks(np.arange(pivot.shape[0]))
973     ax.set_yticklabels([f"Stage 5 {index}" for index in pivot.index])
974     for i in range(pivot.shape[0]):
975         for j in range(pivot.shape[1]):
976             ax.text(j, i, int(pivot.iloc[i, j]), ha="center", va="center", color="#111111")
977     ax.set_title("Stage 5 Two-Cluster vs Stage 6 Six-Cluster Crosswalk")
978     ax.set_xlabel("Stage 6 cluster")
979     ax.set_ylabel("Stage 5 cluster")
980     fig.colorbar(image, ax=ax, label="Subject count")
981     fig.tight_layout()
982     fig.savefig(FIGURES_DIR / "stage6_stage5_crosswalk_heatmap.png", dpi=220)
983     plt.close(fig)
984
985
986 def plot_top_features_by_cluster(top_by_cluster: pd.DataFrame) -> None:
987     fig, axes = plt.subplots(3, 2, figsize=(14, 12))
988     axes = axes.flatten()
989     for cluster, ax in enumerate(axes):
990         data = top_by_cluster.loc[top_by_cluster["stage6_cluster"] == cluster].copy()
991         data = data.sort_values("one_vs_rest_zmean_difference")
992         colors = np.where(data["one_vs_rest_zmean_difference"] >= 0, "#2f80ed", "#e07a5f")
993         ax.barh(data["feature"], data["one_vs_rest_zmean_difference"], color=colors)
994         ax.axvline(0, color="#333333", linewidth=0.8)
995         ax.set_title(f"Cluster {cluster}")
996         ax.set_xlabel("One-vs-rest z-mean difference")
997         ax.tick_params(axis="y", labelsiz=7)
998     fig.suptitle("Stage 6 Top Distinguishing Features by Cluster", y=0.995)
999     fig.tight_layout()
1000     fig.savefig(FIGURES_DIR / "stage6_top_features_by_cluster.png", dpi=220)
1001     plt.close(fig)
1002
1003
1004 def report_selection_reason(metrics: pd.DataFrame, recommended: pd.Series) -> str:
1005     top = metrics.sort_values("selection_score", ascending=False).head(5)
1006     text = (
1007         f"主推荐方案为 `{recommended['feature_set']}` / `{recommended['representation']}` / "
1008         f"`{recommended['clustering_method']}`。该方案的最小簇人数为 `{int(recommended['min_cluster_size'])}`，"

```

```

1009         f" 最小簇比例为 `{recommended['min_cluster_ratio']:.3f}`，稳定性 ARI 均值为 "
1010         f"`{recommended['stability_ari_mean']:.3f}`。选择时同时考虑了簇大小、稳定性、聚类指标、"
1011         " 特征集可解释性、是否依赖高维噪声，以及与 Stage 5 二类语音表型的关系。\\n\\n"
1012         " 没有只按单项指标选择方案。若某些候选 silhouette 或 Calinski-Harabasz 较高，但存在小簇、"
1013         " 不稳定或高维噪声依赖，则不作为论文主解释方案。\\n\\n"
1014     )
1015     return text + markdown_table(
1016         top,
1017         [
1018             "feature_set",
1019             "representation",
1020             "clustering_method",
1021             "selection_score",
1022             "min_cluster_size",
1023             "min_cluster_ratio",
1024             "silhouette",
1025             "stability_ari_mean",
1026             "stage5_nmi",
1027             "recommended_flag",
1028         ],
1029     )
1030
1031
1032 def write_report(
1033     subject_table: pd.DataFrame,
1034     pd_subjects: pd.DataFrame,
1035     metrics: pd.DataFrame,
1036     k_scan: pd.DataFrame,
1037     assignments: pd.DataFrame,
1038     feature_profiles: pd.DataFrame,
1039     top_by_cluster: pd.DataFrame,
1040     cluster_profiles: pd.DataFrame,
1041     crosswalk: pd.DataFrame,
1042     classifier_metrics: pd.DataFrame,
1043     consensus: dict[str, float],
1044 ) -> None:
1045     recommended = metrics.loc[metrics["recommended_flag"]].iloc[0]
1046     classifier_summary = classifier_metrics[classifier_metrics["fold"].isin(["mean", "std"])]
1047     cluster_size_table = (
1048         assignments.groupby("stage6_cluster")
1049         .size()
1050         .reset_index(name="n_subjects")
1051     )
1052     cluster_size_table["ratio"] = cluster_size_table["n_subjects"] / assignments.shape[0]
1053     sex_table = pd.crosstab(assignments["stage6_cluster"], assignments["sex_male"], margins=True)
1054     category_counts = top_by_cluster["category"].value_counts().reset_index()
1055     category_counts.columns = ["category", "top_feature_count"]
1056     boundary_text = (
1057         " 由于附件数据未提供静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆、睡眠障碍的真实临床标签，"
1058         " 本文不直接构建六类症状监督分类模型，而是在帕金森病受试者内部进行 k=6 无监督聚类，"
1059         " 以获得基于语音特征的六类潜在表型。随后训练的分类器预测的是聚类标签，而非临床确诊标签。"
1060         " 因此，该模型只能作为六类帕金森病症状辅助诊断的探索性语音表型参考，仍需真实临床症状标签进一步验证。"
1061     )
1062     report = [
1063         "# Stage 6 Dataset 1 PD Six-Cluster Voice Phenotyping",
1064         "",
1065         "## 1. Stage 6 目标与边界",
1066         "本阶段在 PD 受试者内部进行 k=6 无监督聚类，输出基于语音特征的六类潜在表型，并训练 six-cluster-label classifier  

1067         ↳ 预测聚类标签。",
1068         " 六个 cluster 编号没有临床症状含义。",
1069         "",
1070         "## 2. 为什么不能做真实六类临床症状监督诊断",
1071         boundary_text,
1072         "",
1073         "## 3. 输入文件与样本审计",
1074         f"- Subject-level input: `{rel(SUBJECT_TABLE_PATH)}`",
1075         f"- Total subjects: `{subject_table.shape[0]}`",
1076         f"- PD subjects: `{pd_subjects.shape[0]}`",
1077         f"- Class distribution: `{subject_table['class'].value_counts().sort_index().to_dict()}`",
1078         f"- Missing values: `{int(subject_table.isna().sum().sum())}`",
1079         f"- Duplicated id count: `{int(subject_table['id'].duplicated().sum())}`",
1080         f"- Stage 5 assignment file: `{rel(STAGE5_ASSIGNMENTS_PATH)}`",
1081         "",
1082         "## 4. 特征集与预处理",
1083         "使用 Top20、Top50 和 all_acoustic 三套特征集，所有表示均先 StandardScaler，再按需要 PCA。`id`，`gender`，`class`，  

1084         ↳ `sex_male`，`record_count`，Stage 5 cluster 和 Stage 6 cluster 不进入聚类或分类器输入。",
1085         "",
1086         "## 5. k=6 聚类候选方案完整指标表",
1087         markdown_table(
1088             metrics,
1089             [
1090                 "feature_set",
1091                 "representation",
1092                 "clustering_method",
1093                 "cluster_sizes",
1094                 "min_cluster_size",
1095                 "min_cluster_ratio",
1096                 "silhouette",
1097                 "davies_bouldin",
1098                 "stability_ari_mean",
1099                 "recommended_flag",
1100             ],
1101         ),
1102     ]

```

```

1099     ),
1100     "",
1101     "### 6. k=2 到 k=8 参考扫描结果",
1102     "k 扫描只用于说明 k=6 的相对聚类质量和稳定性; k=6 是由题目六类设定驱动的探索性语音表型划分, 不是由真实六类临床标签监督得到。",
1103     ↪ " ",
1104     markdown_table(
1105         k_scan,
1106         [
1107             "k",
1108             "clustering_method",
1109             "min_cluster_size",
1110             "silhouette",
1111             "davies_bouldin",
1112             "stability_ari_mean",
1113         ],
1114         max_rows=21,
1115     ),
1116     "",
1117     "### 7. 稳定性分析结果",
1118     f" 每个 k=6 候选和 k 扫描方案均执行 `{STABILITY_RUNS}` 次 80% PD 受试者重采样, 重新标准化、PCA 与聚类, 并计算 ARI。",
1119     f"主方案 Consensus KMeans 参考: within-cluster mean `{consensus['consensus_within_cluster_mean']:.3f}`,",
1120     ↪ " between-cluster mean `{consensus['consensus_between_cluster_mean']:.3f}`, separation",
1121     ↪ "`{consensus['consensus_separation']:.3f}`。",
1122     "",
1123     "### 8. 主推荐方案选择理由",
1124     report_selection_reason(metrics, recommended),
1125     "",
1126     "### 9. 六个簇的人数与比例",
1127     markdown_table(cluster_size_table, ["stage6_cluster", "n_subjects", "ratio"]),
1128     "",
1129     "### 10. 六个簇的声学特征画像",
1130     markdown_table(
1131         cluster_profiles,
1132         [
1133             "stage6_cluster",
1134             "n_subjects",
1135             "ratio",
1136             "dominant_feature_categories",
1137             "tentative_voice_phenotype_label",
1138         ],
1139     ),
1140     "",
1141     "### 11. 每个簇的 Top 区分语音特征",
1142     markdown_table(
1143         top_by_cluster,
1144         [
1145             "stage6_cluster",
1146             "feature",
1147             "category",
1148             "one_vs_rest_zmean_difference",
1149             "one_vs_rest_cohens_d",
1150             "q_kruskal",
1151         ],
1152         max_rows=60,
1153     ),
1154     "",
1155     "### 12. 特征类别分布与解释",
1156     markdown_table(category_counts, ["category", "top_feature_count"]),
1157     " 这些类别分布用于描述六类潜在语音表型的声学差异来源, 不是临床病因解释。",
1158     "",
1159     "### 13. Stage 5 二类表型与 Stage 6 六类表型关系",
1160     markdown_table(
1161         crosswalk,
1162         [
1163             "stage5_cluster",
1164             "stage6_cluster",
1165             "count",
1166             "row_percentage",
1167             "column_percentage",
1168             "normalized_mutual_info_stage5_vs_stage6",
1169             "collapsed_stage6_to_stage5_majority_ari",
1170         ],
1171     ),
1172     " 该交叉表用于观察六类潜在表型是否可视为二类潜在语音表型的细分; 不能解释为真实临床层级诊断。",
1173     "",
1174     "### 14. 性别分布的事后描述",
1175     markdown_table(sex_table.reset_index(), list(sex_table.reset_index().columns)),
1176     " `sex_male` 未进入 Stage 6 聚类或 six-cluster-label classifier, 只用于事后描述, 不能作为因果解释。",
1177     "",
1178     "### 15. six-cluster-label classifier 结果",
1179     markdown_table(
1180         classifier_summary,
1181         [
1182             "model",
1183             "fold",
1184             "accuracy",
1185             "balanced_accuracy",
1186             "f1_macro",
1187             "f1_weighted",
1188             "precision_macro",
1189             "recall_macro",
1190             "multiclass_roc_auc_ovr",

```

```

1188     ],
1189 ),
1190 "该分类器学习的是无监督六类聚类标签，用于将潜在语音表型分型规则模型化；它不是训练自真实静止性震颤、运动迟缓、肌肉僵直、疼痛、
    ↳ 痴呆或睡眠障碍临床标签，因此不能解释为真实六类症状诊断性能。",
1191 "",
1192 "## 16. 局限性",
1193 "- 六类标签来自语音特征内部结构，没有外部临床症状标签验证。",
1194 "- Kruskal-Wallis 和 FDR 是聚类后的描述性比较，不是外部临床验证假设检验。",
1195 "- cluster 编号和 tentative label 均为声学解释命名，不对应具体临床症状。",
1196 "- k=6 由题目设定驱动，k 扫描只提供参考。",
1197 "",
1198 "## 17. 论文建议表述",
1199 "建议写作：本文在 PD 受试者内部基于 Stage 4 筛选的声学特征进行 k=6 无监督聚类，得到六个基于语音特征的潜在表型，并训练
    ↳ six-cluster-label classifier 复现该探索性分型规则。",
1200 "不建议写作：将本阶段结果表述为真实六类临床症状诊断性能。",
1201 "",
1202 "## 输出文件",
1203 "- `results/stage6_six_cluster_input_audit.csv`",
1204 "- `results/stage6_six_cluster_metrics.csv`",
1205 "- `results/stage6_k_scan_metrics.csv`",
1206 "- `results/stage6_six_cluster_assignments_dataset1_pd.csv`",
1207 "- `results/stage6_six_cluster_feature_profiles.csv`",
1208 "- `results/stage6_six_cluster_top_features_by_cluster.csv`",
1209 "- `results/stage6_six_cluster_profiles.csv`",
1210 "- `results/stage6_stage5_crosswalk.csv`",
1211 "- `results/stage6_six_cluster_label_classifier_metrics.csv`",
1212 "- `results/stage6_six_cluster_label_classifier_confusion_matrices.csv`",
1213 ]
1214 (RESULTS_DIR / "stage6_six_cluster_report.md").write_text("\n".join(report), encoding="utf-8")
1215
1216
1217 def main() -> None:
1218     RESULTS_DIR.mkdir(exist_ok=True)
1219     FIGURES_DIR.mkdir(exist_ok=True)
1220
1221     subject_table, top20_features, top50_features, _, stage5_assignments, _ = load_inputs()
1222     all_features = acoustic_columns(subject_table)
1223     pd_subjects = validate_pd_subjects(subject_table, stage5_assignments)
1224     write_input_audit(subject_table, pd_subjects, all_features, top20_features, top50_features, stage5_assignments)
1225
1226     feature_sets = {
1227         "top20_biomarkers": top20_features,
1228         "top50_biomarkers": top50_features,
1229         "all_acoustic": all_features,
1230     }
1231     stage5_labels = aligned_stage5(stage5_assignments, pd_subjects)
1232     metrics, artifacts = run_clustering_candidates(pd_subjects, feature_sets, MAIN_K)
1233     metrics, recommended_key = choose_recommended(metrics, artifacts, stage5_labels.to_numpy())
1234     write_csv(metrics, RESULTS_DIR / "stage6_six_cluster_metrics.csv")
1235
1236     recommended_artifact = artifacts[recommended_key]
1237     recommended_labels = np.asarray(recommended_artifact["labels"], dtype=int)
1238     assignments = write_assignments(pd_subjects, stage5_labels, recommended_key, recommended_artifact)
1239     k_scan = run_k_scan(
1240         pd_subjects,
1241         recommended_artifact["features"],
1242         recommended_key[0],
1243         recommended_key[1],
1244     )
1245     consensus = consensus_kmeans_summary(
1246         pd_subjects[recommended_artifact["features"]],
1247         recommended_key[1],
1248         recommended_labels,
1249     )
1250
1251     feature_profiles, top_by_cluster = build_feature_profiles(
1252         pd_subjects,
1253         recommended_labels,
1254         recommended_artifact["features"],
1255     )
1256     cluster_profiles = build_cluster_profiles(assignments, top_by_cluster)
1257     crosswalk = stage5_crosswalk(assignments)
1258     classifier_metrics, _ = run_classifier(
1259         pd_subjects,
1260         recommended_artifact["features"],
1261         recommended_labels,
1262     )
1263
1264     plot_pca_clusters(assignments)
1265     plot_cluster_size(assignments)
1266     plot_candidate_metrics(metrics)
1267     plot_feature_heatmap(feature_profiles)
1268     plot_crosswalk_heatmap(crosswalk)
1269     plot_top_features_by_cluster(top_by_cluster)
1270     write_report(
1271         subject_table,
1272         pd_subjects,
1273         metrics,
1274         k_scan,
1275         assignments,
1276         feature_profiles,
1277         top_by_cluster,

```

```
1278         cluster_profiles,
1279         crosswalk,
1280         classifier_metrics,
1281         consensus,
1282     )
1283
1284
1285 if __name__ == "__main__":
1286     main()
```

A.6 代码六：PD 内部分型特征扩展与权重敏感性分析

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from pathlib import Path
5 from typing import Any
6
7 import matplotlib
8
9 matplotlib.use("Agg")
10
11 import matplotlib.pyplot as plt
12 import numpy as np
13 import pandas as pd
14 from sklearn.cluster import AgglomerativeClustering, KMeans
15 from sklearn.decomposition import PCA, SparsePCA
16 from sklearn.metrics import (
17     adjusted_rand_score,
18     calinski_harabasz_score,
19     davies_bouldin_score,
20     silhouette_score,
21 )
22 from sklearn.mixture import GaussianMixture
23 from sklearn.preprocessing import StandardScaler
24
25
26 ROOT = Path(__file__).resolve().parents[1]
27 RESULTS_DIR = ROOT / "results"
28 FIGURES_DIR = RESULTS_DIR / "figures"
29
30 SUBJECT_TABLE_PATH = RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv"
31 TOP20_PATH = RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv"
32 TOP50_PATH = RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv"
33 STAGE5_METRICS_PATH = RESULTS_DIR / "stage5_clustering_metrics.csv"
34 STAGE6_METRICS_PATH = RESULTS_DIR / "stage6_six_cluster_metrics.csv"
35 STAGE5_ASSIGNMENTS_PATH = RESULTS_DIR / "stage5_cluster_assignments_dataset1_pd.csv"
36 STAGE6_ASSIGNMENTS_PATH = RESULTS_DIR / "stage6_six_cluster_assignments_dataset1_pd.csv"
37
38 OUTPUT_CSV_ENCODING = "utf-8-sig"
39 RANDOM_STATE = 42
40 PD_LABEL = 1
41 EXPECTED_SUBJECTS = 252
42 EXPECTED_PD_SUBJECTS = 188
43 EXPECTED_ACOUSTIC_FEATURES = 752
44 STABILITY_RUNS = 50
45 STABILITY_SAMPLE_FRACTION = 0.8
46 WEIGHT_SENSITIVITY_RUNS = 10_000
47
48 METADATA_COLUMNS = {"id", "gender", "class", "sex_male", "record_count"}
49 LABEL_COLUMN_TOKENS = ("cluster", "label", "prediction", "phenotype")
50 METHODS = ["KMeans", "GaussianMixture", "AgglomerativeClustering"]
51 OBJECTIVE_METRICS = [
52     "silhouette_norm",
53     "calinski_harabasz_norm",
54     "davies_bouldin_norm",
55     "stability_ari_norm",
56     "balance_norm",
57 ]
58
59
60 @dataclass(frozen=True)
61 class FeatureScheme:
62     name: str
63     source_features: list[str]
64     is_sparse_pca: bool = False
65     sparse_components: int | None = None
66     selection_rule: str = ""
67
68
69 def rel(path: Path) -> str:
70     return path.relative_to(ROOT).as_posix()
71
72
73 def write_csv(df: pd.DataFrame, path: Path) -> None:
74     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
75
76
77 def require_inputs() -> None:
78     for path in [
79         SUBJECT_TABLE_PATH,
80         TOP20_PATH,
81         TOP50_PATH,
82         STAGE5_METRICS_PATH,
83         STAGE6_METRICS_PATH,
84         STAGE5_ASSIGNMENTS_PATH,
85         STAGE6_ASSIGNMENTS_PATH,
86     ]:
87         if not path.exists():
88             raise FileNotFoundError(f"Missing required input: {path}")
89
90

```

```

91 def is_forbidden_feature_column(column: str) -> bool:
92     name = column.lower()
93     return column in METADATA_COLUMNS or any(token in name for token in LABEL_COLUMN_TOKENS)
94
95
96 def acoustic_columns(subject_table: pd.DataFrame) -> list[str]:
97     columns = [
98         column
99         for column in subject_table.select_dtypes(include="number").columns
100         if not is_forbidden_feature_column(column)
101     ]
102     if len(columns) != EXPECTED_ACOUSTIC_FEATURES:
103         raise ValueError(
104             f"Expected {EXPECTED_ACOUSTIC_FEATURES} acoustic features after metadata "
105             f"exclusion, found {len(columns)}"
106         )
107     forbidden = sorted(column for column in columns if is_forbidden_feature_column(column))
108     if forbidden:
109         raise ValueError(f"Forbidden columns entered acoustic feature list: {forbidden}")
110     return columns
111
112
113 def load_inputs() -> tuple[pd.DataFrame, pd.DataFrame, list[str], list[str], list[str]]:
114     require_inputs()
115     subject_table = pd.read_csv(SUBJECT_TABLE_PATH)
116     top20_table = pd.read_csv(TOP20_PATH)
117     top50_table = pd.read_csv(TOP50_PATH)
118
119     required_columns = {"id", "class", "sex_male", "record_count"}
120     missing = sorted(required_columns - set(subject_table.columns))
121     if missing:
122         raise ValueError(f"Subject table missing required columns: {missing}")
123     if subject_table.shape[0] != EXPECTED_SUBJECTS:
124         raise ValueError(f"Expected {EXPECTED_SUBJECTS} subjects, found {subject_table.shape[0]}")
125     if int(subject_table["id"].duplicated().sum()):
126         raise ValueError("Subject table must contain one row per subject")
127     if int(subject_table.isna().sum().sum()):
128         raise ValueError("Subject table contains missing values; Stage 5b/6b expects sealed Stage 4 table")
129
130     for label, table, expected_rows in [(("Top20", top20_table, 20), ("Top50", top50_table, 50))]:
131         if "feature" not in table.columns:
132             raise ValueError(f"{label} file missing required 'feature' column")
133         if table.shape[0] != expected_rows:
134             raise ValueError(f"{label} expected {expected_rows} rows, found {table.shape[0]}")
135
136     pd_subjects = subject_table.loc[subject_table["class"] == PD_LABEL].copy()
137     if pd_subjects.shape[0] != EXPECTED_PD_SUBJECTS:
138         raise ValueError(f"Expected {EXPECTED_PD_SUBJECTS} PD subjects, found {pd_subjects.shape[0]}")
139     if set(pd_subjects["class"].unique()) != {PD_LABEL}:
140         raise ValueError("Healthy samples entered PD-only table")
141
142     all_features = acoustic_columns(subject_table)
143     top20_features = top20_table["feature"].astype(str).tolist()
144     top50_features = top50_table["feature"].astype(str).tolist()
145     for label, features in [(("Top20", top20_features), ("Top50", top50_features))]:
146         missing_features = sorted(set(features) - set(all_features))
147         if missing_features:
148             raise ValueError(f"{label} features not present in acoustic feature list: {missing_features}")
149
150     return subject_table, pd_subjects, all_features, top20_features, top50_features
151
152
153 def pd_iqr_mad(pd_subjects: pd.DataFrame, features: list[str]) -> tuple[pd.Series, pd.Series]:
154     data = pd_subjects[features]
155     iqr = (data.quantile(0.75) - data.quantile(0.25)).astype(float)
156     mad = (data - data.median()).abs().median().astype(float)
157     return iqr, mad
158
159
160 def connected_components_from_corr(corr: np.ndarray, threshold: float) -> list[list[int]]:
161     seen = np.zeros(corr.shape[0], dtype=bool)
162     components: list[list[int]] = []
163     for index in range(corr.shape[0]):
164         if seen[index]:
165             continue
166         stack = [index]
167         seen[index] = True
168         component: list[int] = []
169         while stack:
170             current = stack.pop()
171             component.append(current)
172             neighbors = np.where((corr[current] > threshold) & (~seen))[0]
173             for neighbor in neighbors:
174                 seen[int(neighbor)] = True
175                 stack.append(int(neighbor))
176             components.append(component)
177     return components
178
179
180 def corr_reduce_features(
181     pd_subjects: pd.DataFrame,
182     features: list[str],

```

```

183     threshold: float,
184     feature_set_name: str,
185     iqr: pd.Series,
186     mad: pd.Series,
187 ) -> tuple[list[str], pd.DataFrame]:
188     corr = pd_subjects[features].corr().abs().to_numpy()
189     components = connected_components_from_corr(corr, threshold)
190     retained_rows: list[dict[str, Any]] = []
191     retained_features: list[str] = []
192
193     for component_id, component in enumerate(components, start=1):
194         component_features = [features[index] for index in component]
195         retained = sorted(
196             component_features,
197             key=lambda feature: (float(iqr[feature]), float(mad[feature])), feature),
198             reverse=True,
199         )[0]
200         retained_features.append(retained)
201         retained_rows.append(
202             {
203                 "feature_set": feature_set_name,
204                 "feature": retained,
205                 "source_feature_count": len(features),
206                 "threshold_abs_r": threshold,
207                 "component_id": component_id,
208                 "component_size": len(component_features),
209                 "component_features": ";".join(sorted(component_features)),
210                 "pd_iqr": float(iqr[retained]),
211                 "pd_mad": float(mad[retained]),
212                 "selection_rule": "PD-only correlation components; retain largest PD IQR, tie-break by MAD then feature
                ↪ name",
213             }
214         )
215
216     return retained_features, pd.DataFrame(retained_rows)
217
218
219 def top_dispersion_features(
220     features: list[str],
221     scores: pd.Series,
222     n_features: int,
223     feature_set_name: str,
224     iqr: pd.Series,
225     mad: pd.Series,
226 ) -> tuple[list[str], pd.DataFrame]:
227     ordered = sorted(features, key=lambda feature: (float(scores[feature])), feature), reverse=True)
228     retained = ordered[:n_features]
229     rows = [
230         {
231             "feature_set": feature_set_name,
232             "feature": feature,
233             "rank": rank,
234             "pd_iqr": float(iqr[feature]),
235             "pd_mad": float(mad[feature]),
236             "selection_rule": f"PD-only top {n_features} by {'MAD' if 'mad' in feature_set_name else 'IQR'}",
237         }
238         for rank, feature in enumerate(retained, start=1)
239     ]
240     return retained, pd.DataFrame(rows)
241
242
243 def build_feature_schemes(
244     pd_subjects: pd.DataFrame,
245     all_features: list[str],
246     top20_features: list[str],
247     top50_features: list[str],
248 ) -> tuple[dict[str, FeatureScheme], pd.DataFrame, pd.DataFrame]:
249     iqr, mad = pd_iqr_mad(pd_subjects, all_features)
250     retained_tables: list[pd.DataFrame] = []
251
252     corr_all_090, details = corr_reduce_features(
253         pd_subjects, all_features, 0.90, "pd_corr_reduced_all_090", iqr, mad
254     )
255     retained_tables.append(details)
256     corr_all_095, details = corr_reduce_features(
257         pd_subjects, all_features, 0.95, "pd_corr_reduced_all_095", iqr, mad
258     )
259     retained_tables.append(details)
260     corr_top50_090, details = corr_reduce_features(
261         pd_subjects, top50_features, 0.90, "pd_corr_reduced_top50_090", iqr, mad
262     )
263     retained_tables.append(details)
264
265     high_iqr_50, details = top_dispersion_features(
266         all_features, iqr, 50, "pd_high_iqr_top50", iqr, mad
267     )
268     retained_tables.append(details)
269     high_iqr_100, details = top_dispersion_features(
270         all_features, iqr, 100, "pd_high_iqr_top100", iqr, mad
271     )
272     retained_tables.append(details)
273     high_mad_50, details = top_dispersion_features(

```



```

274     all_features, mad, 50, "pd_high_mad_top50", iqr, mad
275 )
276 retained_tables.append(details)
277 high_mad_100, details = top_dispersion_features(
278     all_features, mad, 100, "pd_high_mad_top100", iqr, mad
279 )
280 retained_tables.append(details)
281
282 schemes = {
283     "top20_biomarkers": FeatureScheme(
284         "top20_biomarkers",
285         top20_features,
286         selection_rule=" 封面第 4 阶段 Top20; 仅作参考; 不加评分奖励",
287     ),
288     "top50_biomarkers": FeatureScheme(
289         "top50_biomarkers",
290         top50_features,
291         selection_rule=" 封面第 4 阶段 Top50; 仅作参考; 不加评分奖励",
292     ),
293     "all_acoustic": FeatureScheme(
294         "all_acoustic",
295         all_features,
296         selection_rule=" 全部 752 个声学特征; 聚类前必须先做 PCA",
297     ),
298     "pd_corr_reduced_all_090": FeatureScheme(
299         "pd_corr_reduced_all_090",
300         corr_all_090,
301         selection_rule=" 基于 PD-only 样本, 从全部声学特征出发按 |r| > 0.90 做相关性去冗余",
302     ),
303     "pd_corr_reduced_all_095": FeatureScheme(
304         "pd_corr_reduced_all_095",
305         corr_all_095,
306         selection_rule=" 基于 PD-only 样本, 从全部声学特征出发按 |r| > 0.95 做相关性去冗余",
307     ),
308     "pd_corr_reduced_top50_090": FeatureScheme(
309         "pd_corr_reduced_top50_090",
310         corr_top50_090,
311         selection_rule=" 基于 PD-only 样本, 对第 4 阶段 Top50 按 |r| > 0.90 做相关性去冗余",
312     ),
313     "pd_high_iqr_top50": FeatureScheme(
314         "pd_high_iqr_top50",
315         high_iqr_50,
316         selection_rule=" 基于 PD-only 样本, 在 752 个声学特征中保留 IQR 最大的 50 个",
317     ),
318     "pd_high_iqr_top100": FeatureScheme(
319         "pd_high_iqr_top100",
320         high_iqr_100,
321         selection_rule=" 基于 PD-only 样本, 在 752 个声学特征中保留 IQR 最大的 100 个",
322     ),
323     "pd_high_mad_top50": FeatureScheme(
324         "pd_high_mad_top50",
325         high_mad_50,
326         selection_rule=" 基于 PD-only 样本, 在 752 个声学特征中保留 MAD 最大的 50 个",
327     ),
328     "pd_high_mad_top100": FeatureScheme(
329         "pd_high_mad_top100",
330         high_mad_100,
331         selection_rule=" 基于 PD-only 样本, 在 752 个声学特征中保留 MAD 最大的 100 个",
332     ),
333     "pd_sparse_pca_10": FeatureScheme(
334         "pd_sparse_pca_10",
335         all_features,
336         is_sparse_pca=True,
337         sparse_components=10,
338         selection_rule=" 对 752 个 PD-only 声学特征先标准化, 再提取 10 维 SparsePCA 表示",
339     ),
340     "pd_sparse_pca_20": FeatureScheme(
341         "pd_sparse_pca_20",
342         all_features,
343         is_sparse_pca=True,
344         sparse_components=20,
345         selection_rule=" 对 752 个 PD-only 声学特征先标准化, 再提取 20 维 SparsePCA 表示",
346     ),
347 }
348
349 audit_rows = []
350 for scheme in schemes.values():
351     if scheme.is_sparse_pca:
352         representations = [f"sparse_pca_{scheme.sparse_components}"]
353         n_components = scheme.sparse_components
354     elif len(scheme.source_features) <= 50:
355         representations = ["scaled_only", "pca_5", "pca_10"]
356         n_components = np.nan
357     else:
358         representations = ["pca_10", "pca_20", "pca_90"]
359         n_components = np.nan
360     audit_rows.append(
361         {
362             "feature_set": scheme.name,
363             "n_subjects_used_for_selection": EXPECTED_PD_SUBJECTS,
364             "selection_population": "Dataset 1 PD only (class == 1)",
365             "healthy_subjects_used": 0,

```

```

366         "record_level_samples_used": 0,
367         "n_features_original": len(scheme.source_features),
368         "is_sparse_pca_representation": bool(scheme.is_sparse_pca),
369         "sparse_pca_components": n_components,
370         "representations": ";".join(representations),
371         "selection_rule": scheme.selection_rule,
372         "forbidden_columns_excluded": ";".join(
373             ["id", "gender", "class", "sex_male", "record_count", "Stage 5 cluster", "Stage 6 cluster"]
374         ),
375     }
376 )
377
378 retained_detail = pd.concat(retained_tables, ignore_index=True)
379 return schemes, pd.DataFrame(audit_rows), retained_detail
380
381
382 def representations_for_scheme(scheme: FeatureScheme) -> list[str]:
383     if scheme.is_sparse_pca:
384         return [f"sparse_pca_{scheme.sparse_components}"]
385     if len(scheme.source_features) <= 50:
386         return ["scaled_only", "pca_5", "pca_10"]
387     return ["pca_10", "pca_20", "pca_90"]
388
389 def build_representation(
390     data: pd.DataFrame,
391     scheme: FeatureScheme,
392     representation: str,
393 ) -> tuple[np.ndarray, int, float]:
394     scaled = StandardScaler().fit_transform(data[scheme.source_features].to_numpy(dtype=float))
395     if scheme.is_sparse_pca:
396         n_components = int(scheme.sparse_components or 0)
397         model = SparsePCA(
398             n_components=n_components,
399             random_state=RANDOM_STATE,
400             ridge_alpha=0.01,
401             max_iter=200,
402             tol=1e-4,
403             method="cd",
404             n_jobs=-1,
405         )
406         projected = model.fit_transform(scaled)
407         return projected, projected.shape[1], np.nan
408     if representation == "scaled_only":
409         return scaled, scaled.shape[1], np.nan
410     if representation == "pca_90":
411         pca = PCA(n_components=0.90, random_state=RANDOM_STATE)
412     elif representation.startswith("pca-"):
413         pca = PCA(n_components=int(representation.split("_", maxsplit=1)[1]), random_state=RANDOM_STATE)
414     else:
415         raise ValueError(f"Unknown representation: {representation}")
416     projected = pca.fit_transform(scaled)
417     return projected, projected.shape[1], float(np.sum(pca.explained_variance_ratio_))
418
419
420 def cluster_labels(matrix: np.ndarray, method: str, k: int, random_state: int = RANDOM_STATE) -> np.ndarray:
421     if method == "KMeans":
422         return KMeans(n_clusters=k, n_init=50, random_state=random_state).fit_predict(matrix)
423     if method == "GaussianMixture":
424         return GaussianMixture(
425             n_components=k,
426             covariance_type="full",
427             reg_covar=1e-6,
428             random_state=random_state,
429         ).fit_predict(matrix)
430     if method == "AgglomerativeClustering":
431         return AgglomerativeClustering(n_clusters=k, linkage="ward").fit_predict(matrix)
432     raise ValueError(f"Unknown clustering method: {method}")
433
434
435 def cluster_size_stats(labels: np.ndarray, k: int) -> dict[str, Any]:
436     counts = pd.Series(labels).value_counts().reindex(range(k), fill_value=0).astype(int)
437     min_size = int(counts.min())
438     max_size = int(counts.max())
439     min_ratio = float(min_size / len(labels))
440     max_ratio = float(max_size / len(labels))
441     smaller_ratio = float(min_size / len(labels))
442     return {
443         "cluster_sizes": ";".join(f"{cluster}:{int(size)}" for cluster, size in counts.items()),
444         "min_cluster_size": min_size,
445         "max_cluster_size": max_size,
446         "min_cluster_ratio": min_ratio,
447         "max_cluster_ratio": max_ratio,
448         "size_imbalance_ratio": float(max_size / min_size) if min_size > 0 else np.inf,
449         "smaller_cluster_ratio": smaller_ratio,
450     }
451
452
453 def clustering_quality(matrix: np.ndarray, labels: np.ndarray) -> tuple[float, float, float, str]:
454     if np.unique(labels).size < 2:
455         return np.nan, np.nan, np.nan, "single cluster produced"
456     try:

```

```

458         return (
459             float(silhouette_score(matrix, labels)),
460             float(calinski_harabasz_score(matrix, labels)),
461             float(davies_bouldin_score(matrix, labels)),
462             "",
463         )
464     except ValueError as exc:
465         return np.nan, np.nan, np.nan, f"metric calculation failed: {exc}"
466
467
468 def stability_analysis(
469     pd_subjects: pd.DataFrame,
470     scheme: FeatureScheme,
471     representation: str,
472     method: str,
473     k: int,
474     full_labels: np.ndarray,
475     full_matrix: np.ndarray,
476 ) -> tuple[float, float]:
477     rng = np.random.default_rng(RANDOM_STATE + k)
478     n_subjects = pd_subjects.shape[0]
479     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
480     scores: list[float] = []
481     for run_index in range(STABILITY_RUNS):
482         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
483         if scheme.is_sparse_pca:
484             sample_matrix = full_matrix[sample_index]
485         else:
486             sample_data = pd_subjects.iloc[sample_index]
487             sample_matrix, _, _ = build_representation(sample_data, scheme, representation)
488             sample_labels = cluster_labels(sample_matrix, method, k, RANDOM_STATE + run_index + 1)
489             scores.append(float(adjusted_rand_score(full_labels[sample_index], sample_labels)))
490     return float(np.mean(scores)), float(np.std(scores, ddof=1))
491
492
493 def run_clustering_stage(
494     pd_subjects: pd.DataFrame,
495     schemes: dict[str, FeatureScheme],
496     k: int,
497     stage: str,
498 ) -> pd.DataFrame:
499     rows: list[dict[str, Any]] = []
500     for scheme in schemes.values():
501         for representation in representations_for_scheme(scheme):
502             matrix, n_components, explained_variance = build_representation(pd_subjects, scheme, representation)
503             for method in METHODS:
504                 labels = cluster_labels(matrix, method, k)
505                 size_stats = cluster_size_stats(labels, k)
506                 silhouette, calinski, davies, metric_warning = clustering_quality(matrix, labels)
507                 stability_mean, stability_std = stability_analysis(
508                     pd_subjects, scheme, representation, method, k, labels, matrix
509                 )
510                 warnings = []
511                 if metric_warning:
512                     warnings.append(metric_warning)
513                 if stage == "stage5b" and size_stats["smaller_cluster_ratio"] < 0.10:
514                     warnings.append("smaller_cluster_ratio < 0.10; not eligible for recommendation")
515                 if stage == "stage6b":
516                     if size_stats["min_cluster_size"] < 8:
517                         warnings.append("min_cluster_size < 8; not eligible for recommendation")
518                     if size_stats["min_cluster_ratio"] < 0.05:
519                         warnings.append("min_cluster_ratio < 0.05; not eligible for recommendation")
520
521                 rows.append(
522                     {
523                         "feature_set": scheme.name,
524                         "representation": representation,
525                         "method": method,
526                         "k": int(k),
527                         "candidate_id": f"{scheme.name}|{representation}|{method}|k{k}",
528                         "n_subjects": int(len(labels)),
529                         "n_features_original": int(len(scheme.source_features)),
530                         "n_components_after_transform": int(n_components),
531                         "pca_explained_variance_sum": explained_variance,
532                         **size_stats,
533                         "silhouette": silhouette,
534                         "calinski_harabasz": calinski,
535                         "davies_bouldin": davies,
536                         "stability_ari_mean": stability_mean,
537                         "stability_ari_std": stability_std,
538                         "stability_n_runs": STABILITY_RUNS,
539                         "eligible_for_recommendation": len(warnings) == 0,
540                         "exclusion_or_warning": "; ".join(warnings),
541                     }
542                 )
543     return pd.DataFrame(rows)
544
545
546 def minmax(series: pd.Series, higher_is_better: bool = True) -> pd.Series:
547     valid = series.astype(float).replace([np.inf, -np.inf], np.nan)
548     min_value = valid.min(skipna=True)
549     max_value = valid.max(skipna=True)

```

```

550     if pd.isna(min_value) or pd.isna(max_value) or max_value == min_value:
551         return pd.Series(0.5, index=series.index)
552     scaled = (valid - min_value) / (max_value - min_value)
553     if not higher_is_better:
554         scaled = 1 - scaled
555     return scaled.fillna(0.0)
556
557
558 def add_normalized_metrics(metrics: pd.DataFrame, k: int, stage: str) -> pd.DataFrame:
559     out = metrics.copy()
560     if stage == "stage5b":
561         eligible = out["smaller_cluster_ratio"] >= 0.10
562     else:
563         eligible = (out["min_cluster_size"] >= 8) & (out["min_cluster_ratio"] >= 0.05)
564     out["eligible_for_recommendation"] = eligible
565     out["silhouette_norm"] = np.nan
566     out["calinski_harabasz_norm"] = np.nan
567     out["davies_bouldin_norm"] = np.nan
568     out["stability_ari_norm"] = np.nan
569     out["balance_norm"] = np.nan
570
571     eligible_metrics = out.loc[eligible]
572     if eligible_metrics.empty:
573         raise ValueError(f"No eligible candidates for {stage}")
574
575     out.loc[eligible, "silhouette_norm"] = minmax(eligible_metrics["silhouette"], True)
576     out.loc[eligible, "calinski_harabasz_norm"] = minmax(
577         eligible_metrics["calinski_harabasz"], True
578     )
579     out.loc[eligible, "davies_bouldin_norm"] = minmax(eligible_metrics["davies_bouldin"], False)
580     out.loc[eligible, "stability_ari_norm"] = minmax(eligible_metrics["stability_ari_mean"], True)
581     ideal_min_ratio = 1 / k
582     out.loc[eligible, "balance_norm"] = (
583         eligible_metrics["min_cluster_ratio"].astype(float) / ideal_min_ratio
584     ).clip(upper=1.0)
585     return out
586
587
588 def weight_sensitivity(
589     metrics: pd.DataFrame,
590     stage: str,
591 ) -> tuple[pd.DataFrame, pd.DataFrame]:
592     eligible = metrics.loc[metrics["eligible_for_recommendation"]].copy().reset_index(drop=True)
593     if eligible.empty:
594         raise ValueError(f"No eligible candidates available for {stage} sensitivity analysis")
595
596     values = eligible[OBJECTIVE_METRICS].to_numpy(dtype=float)
597     rng = np.random.default_rng(RANDOM_STATE + (5 if stage == "stage5b" else 6))
598     weights = rng.dirichlet(np.ones(len(OBJECTIVE_METRICS)), size=WEIGHT_SENSITIVITY_RUNS)
599     scores = values @ weights.T
600     ranks = np.argsort(-scores, axis=0)
601     top1_indices = ranks[0, :]
602     top3_indices = ranks[:3, :]
603
604     sensitivity_rows = []
605     for run_index in range(WEIGHT_SENSITIVITY_RUNS):
606         winner = eligible.iloc[int(top1_indices[run_index])]
607         sensitivity_rows.append(
608             {
609                 "run": run_index + 1,
610                 "weight_silhouette": float(weights[run_index, 0]),
611                 "weight_calinski_harabasz": float(weights[run_index, 1]),
612                 "weight_davies_bouldin": float(weights[run_index, 2]),
613                 "weight_stability_ari": float(weights[run_index, 3]),
614                 "weight_balance": float(weights[run_index, 4]),
615                 "top1_candidate_id": winner["candidate_id"],
616                 "top1_feature_set": winner["feature_set"],
617                 "top1_representation": winner["representation"],
618                 "top1_method": winner["method"],
619                 "top1_score": float(scores[int(top1_indices[run_index]), run_index]),
620             }
621         )
622
623     top1_counts = np.bincount(top1_indices, minlength=eligible.shape[0])
624     top3_counts = np.bincount(top3_indices.reshape(-1), minlength=eligible.shape[0])
625     rank_frequency = eligible[
626         [
627             "candidate_id",
628             "feature_set",
629             "representation",
630             "method",
631             "k",
632             "n_features_original",
633             "n_components_after_transform",
634             "cluster_sizes",
635             "min_cluster_size",
636             "min_cluster_ratio",
637             "smaller_cluster_ratio",
638             "silhouette",
639             "calinski_harabasz",
640             "davies_bouldin",
641             "stability_ari_mean",

```

```

642         *OBJECTIVE_METRICS,
643     ]
644 ].copy()
645 rank_frequency["top1_count"] = top1_counts.astype(int)
646 rank_frequency["top1_frequency"] = top1_counts / WEIGHT_SENSITIVITY_RUNS
647 rank_frequency["top3_count"] = top3_counts.astype(int)
648 rank_frequency["top3_frequency"] = top3_counts / WEIGHT_SENSITIVITY_RUNS
649 rank_frequency = rank_frequency.sort_values(
650     ["top1_frequency", "top3_frequency", "stability_ari_mean"],
651     ascending=[False, False, False],
652 ).reset_index(drop=True)
653
654 return pd.DataFrame(sensitivity_rows), rank_frequency
655
656
657 def recommendation_label(rank_frequency: pd.DataFrame) -> str:
658     top_frequency = float(rank_frequency.iloc[0]["top1_frequency"])
659     if top_frequency >= 0.60:
660         return " 稳健主推荐方案"
661     if top_frequency >= 0.30:
662         return " 相对推荐方案"
663     return " 多个可接受方案"
664
665
666 def find_original_candidate(stage: str, rank_frequency: pd.DataFrame) -> pd.DataFrame:
667     metrics_path = STAGE5_METRICS_PATH if stage == "stage5b" else STAGE6_METRICS_PATH
668     old = pd.read_csv(metrics_path)
669     old_method_col = "clustering_method" if "clustering_method" in old.columns else "method"
670     rec = old.loc[old["recommended_flag"].astype(str).str.lower().eq("true")]
671     if rec.empty:
672         return pd.DataFrame()
673     row = rec.iloc[0]
674     k = 2 if stage == "stage5b" else 6
675     representation = str(row["representation"])
676     method = str(row[old_method_col])
677     feature_set = str(row["feature_set"])
678     candidate_id = f"{feature_set}|{representation}|{method}|k{k}"
679     return rank_frequency.loc[rank_frequency["candidate_id"] == candidate_id].copy()
680
681
682 def markdown_table(df: pd.DataFrame, columns: list[str], max_rows: int | None = None) -> str:
683     table = df.loc[:, columns].copy()
684     if max_rows is not None:
685         table = table.head(max_rows)
686     for column in table.columns:
687         if pd.api.types.is_float_dtype(table[column]):
688             table[column] = table[column].map(lambda value: "" if pd.isna(value) else f"{value:.4f}")
689     header = "| " + " | ".join(str(column) for column in table.columns) + " |"
690     separator = "| " + " | ".join(["---"] * len(table.columns)) + " |"
691     rows = ["| " + " | ".join(str(value) for value in row) + " |" for row in table.to_numpy()]
692     return "\n".join([header, separator, *rows])
693
694
695 def markdown_table_cn(
696     df: pd.DataFrame,
697     columns: list[str],
698     labels: dict[str, str],
699     max_rows: int | None = None,
700 ) -> str:
701     table = df.loc[:, columns].copy()
702     table = table.rename(columns=labels)
703     if max_rows is not None:
704         table = table.head(max_rows)
705     for column in table.columns:
706         if pd.api.types.is_float_dtype(table[column]):
707             table[column] = table[column].map(lambda value: "" if pd.isna(value) else f"{value:.4f}")
708     header = "| " + " | ".join(str(column) for column in table.columns) + " |"
709     separator = "| " + " | ".join(["---"] * len(table.columns)) + " |"
710     rows = ["| " + " | ".join(str(value) for value in row) + " |" for row in table.to_numpy()]
711     return "\n".join([header, separator, *rows])
712
713
714 def plot_weight_sensitivity(rank_frequency: pd.DataFrame, stage: str, output_path: Path) -> None:
715     plot_data = rank_frequency.head(12).copy()
716     plot_data["candidate"] = (
717         plot_data["feature_set"] + "\n" + plot_data["representation"] + "\n" + plot_data["method"]
718     )
719     x = np.arange(plot_data.shape[0])
720     fig, ax = plt.subplots(figsize=(13, 6))
721     width = 0.38
722     ax.bar(x - width / 2, plot_data["top1_frequency"], width, label="Top-1 frequency", color="#4c78a8")
723     ax.bar(x + width / 2, plot_data["top3_frequency"], width, label="Top-3 frequency", color="#f58518")
724     ax.axhline(0.60, color="#b22222", linestyle="--", linewidth=1, label="Robust threshold")
725     ax.axhline(0.30, color="#777777", linestyle=":", linewidth=1, label="Relative threshold")
726     ax.set_title(f"{stage.upper()} Weight Sensitivity Top Candidates")
727     ax.set_ylabel("Frequency across 10,000 Dirichlet weights")
728     ax.set_xticks(x)
729     ax.set_xticklabels(plot_data["candidate"], rotation=70, ha="right", fontsize=8)
730     ax.legend()
731     fig.tight_layout()
732     fig.savefig(output_path, dpi=220)
733     plt.close(fig)

```

```

734
735
736 def plot_metric_comparison(metrics: pd.DataFrame, stage: str, output_path: Path) -> None:
737     plot_data = metrics.loc[metrics["eligible_for_recommendation"]].copy()
738     summary = (
739         plot_data.groupby("feature_set", as_index=False)
740         .agg(
741             silhouette=("silhouette", "mean"),
742             stability_ari_mean=("stability_ari_mean", "mean"),
743             balance_norm=("balance_norm", "mean"),
744         )
745         .sort_values("stability_ari_mean", ascending=False)
746     )
747     x = np.arange(summary.shape[0])
748     fig, axes = plt.subplots(3, 1, figsize=(13, 10), sharex=True)
749     for ax, metric, title, color in [
750         (axes[0], "silhouette", "Mean silhouette", "#4c78a8"),
751         (axes[1], "stability_ari_mean", "Mean stability ARI", "#54a24b"),
752         (axes[2], "balance_norm", "Mean balance norm", "#f58518"),
753     ]:
754         ax.bar(x, summary[metric], color=color)
755         ax.set_ylabel(title)
756         ax.set_title(f"{stage.upper()} Eligible Feature Set {title}")
757     axes[-1].set_xticks(x)
758     axes[-1].set_xticklabels(summary["feature_set"], rotation=65, ha="right", fontsize=8)
759     fig.tight_layout()
760     fig.savefig(output_path, dpi=220)
761     plt.close(fig)
762
763
764 def plot_original_vs_pdonly(
765     stage5_rank: pd.DataFrame,
766     stage6_rank: pd.DataFrame,
767     output_path: Path,
768 ) -> None:
769     rows = []
770     for stage, stage_key, rank in [
771         ("Stage 5b", "stage5b", stage5_rank),
772         ("Stage 6b", "stage6b", stage6_rank),
773     ]:
774         original = find_original_candidate(stage_key, rank)
775         if not original.empty:
776             rows.append(
777                 {
778                     "stage": stage,
779                     "candidate_type": "Original Stage main scheme",
780                     "frequency": float(original.iloc[0]["top1_frequency"]),
781                     "label": original.iloc[0]["feature_set"],
782                 }
783             )
784         rows.append(
785             {
786                 "stage": stage,
787                 "candidate_type": "Best PD-only no-bonus scheme",
788                 "frequency": float(rank.iloc[0]["top1_frequency"]),
789                 "label": rank.iloc[0]["feature_set"],
790             }
791         )
792     plot_data = pd.DataFrame(rows)
793     fig, ax = plt.subplots(figsize=(9, 5))
794     labels = plot_data["stage"] + "\n" + plot_data["candidate_type"]
795     colors = ["#4c78a8" if "Original" in value else "#f58518" for value in plot_data["candidate_type"]]
796     ax.bar(np.arange(plot_data.shape[0]), plot_data["frequency"], color=colors)
797     ax.axhline(0.60, color="#b22222", linestyle="--", linewidth=1)
798     ax.axhline(0.30, color="#777777", linestyle=":", linewidth=1)
799     ax.set_xticks(np.arange(plot_data.shape[0]))
800     ax.set_xticklabels(labels, fontsize=8)
801     ax.set_ylabel("Top-1 frequency")
802     ax.set_title("Original Stage 5/6 Main Schemes vs PD-only No-bonus Sensitivity")
803     for index, row in plot_data.iterrows():
804         ax.text(index, row["frequency"], f"{row['frequency']:.1%}\n{row['label']}", ha="center", va="bottom", fontsize=8)
805     fig.tight_layout()
806     fig.savefig(output_path, dpi=220)
807     plt.close(fig)
808
809
810 def robustness_sentence(stage: str, rank_frequency: pd.DataFrame) -> str:
811     original = find_original_candidate(stage, rank_frequency)
812     stage_label = "第 5 阶段" if stage == "stage5b" else "第 6 阶段"
813     if original.empty:
814         return f"{stage_label} 原主方案未能在 5b/6b 候选集中匹配，不能判定其在无奖励敏感性分析下仍稳健。"
815     row = original.iloc[0]
816     frequency = float(row["top1_frequency"])
817     if frequency >= 0.60:
818         result = "仍可视作稳健"
819     elif frequency >= 0.30:
820         result = "只能视为相对稳健，应同时报告其他可接受方案"
821     else:
822         result = "不再稳健，不应继续单独称为最佳方案"
823     return (
824         f"{stage_label} 原主方案 {row['feature_set']} / {row['representation']} / {row['method']}` "
825         f"在无奖励权重敏感性分析中的第一名频率为 {frequency:.1%}，{result}。"

```

```

826     )
827
828
829 def write_report(
830     stage5_metrics: pd.DataFrame,
831     stage6_metrics: pd.DataFrame,
832     stage5_rank: pd.DataFrame,
833     stage6_rank: pd.DataFrame,
834     audit: pd.DataFrame,
835 ) -> None:
836     stage5_label = recommendation_label(stage5_rank)
837     stage6_label = recommendation_label(stage6_rank)
838     report = [
839         "## 第 5b/6b 阶段 PD-only 特征扩展与权重敏感性报告",
840         "",
841         "### 1. 方法学动机",
842         "Healthy/PD 判别特征是为区分病例与健康对照而排序的, 可能主要反映疾病有无、性别或全局声学差异; PD 内部分型需要捕捉 188 名 PD
843         ↳ 受试者内部的异质性。因此第 5b/6b 阶段将特征筛选限定在 PD-only 样本内, 并保留第 4 阶段 Top20/Top50 仅作为对照。",
844         "",
845         "## 2. 新增 PD-only 特征方案",
846         markdown_table_cn(
847             audit,
848             [
849                 "feature_set",
850                 "n_features_original",
851                 "is_sparse_pca_representation",
852                 "sparse_pca_components",
853                 "representations",
854                 "selection_rule",
855             ],
856             {
857                 "feature_set": " 特征方案",
858                 "n_features_original": " 原始特征数",
859                 "is_sparse_pca_representation": " 是否为 SparsePCA 表示",
860                 "sparse_pca_components": "SparsePCA 维数",
861                 "representations": " 聚类表示",
862                 "selection_rule": " 筛选规则",
863             },
864         ),
865         "",
866         "### 3. 取消人为奖励",
867         "第 5b/6b 阶段不对 Top20、Top50 或第 4 阶段 biomarker 特征集加解释性奖励; 第 6b 阶段不使用第 5 阶段 NMI 参与主推荐评分;
868         ↳ 图像好看程度不进入任何评分。解释性只在论文讨论中单独说明。",
869         "",
870         "## 4. 第 5b 阶段全部候选方案客观指标",
871         markdown_table_cn(
872             stage5_metrics,
873             [
874                 "feature_set",
875                 "representation",
876                 "method",
877                 "cluster_sizes",
878                 "smaller_cluster_ratio",
879                 "silhouette",
880                 "calinski_harabasz",
881                 "davies_bouldin",
882                 "stability_ari_mean",
883                 "eligible_for_recommendation",
884             ],
885             {
886                 "feature_set": " 特征方案",
887                 "representation": " 表示方式",
888                 "method": " 聚类方法",
889                 "cluster_sizes": " 簇人数",
890                 "smaller_cluster_ratio": " 较小簇比例",
891                 "silhouette": " 轮廓系数",
892                 "calinski_harabasz": "Calinski-Harabasz",
893                 "davies_bouldin": "Davies-Bouldin",
894                 "stability_ari_mean": " 稳定性 ARI 均值",
895                 "eligible_for_recommendation": " 是否可推荐",
896             },
897         ),
898         "",
899         "### 5. 第 6b 阶段全部候选方案客观指标",
900         markdown_table_cn(
901             stage6_metrics,
902             [
903                 "feature_set",
904                 "representation",
905                 "method",
906                 "cluster_sizes",
907                 "min_cluster_size",
908                 "min_cluster_ratio",
909                 "silhouette",
910                 "calinski_harabasz",
911                 "davies_bouldin",
912                 "stability_ari_mean",
913                 "eligible_for_recommendation",
914             ],
915             {
916                 "feature_set": " 特征方案",
917                 "representation": " 表示方式",

```

```

916         "method": " 聚类方法",
917         "cluster_sizes": " 簇人数",
918         "min_cluster_size": " 最小簇人数",
919         "min_cluster_ratio": " 最小簇比例",
920         "silhouette": " 轮廓系数",
921         "calinski_harabasz": "Calinski-Harabasz",
922         "davies_bouldin": "Davies-Bouldin",
923         "stability_ari_mean": " 稳定性 ARI 均值",
924         "eligible_for_recommendation": " 是否可推荐",
925     },
926 ),
927 "",
928 """ 6. 权重敏感性分析方法",
929 f"对通过硬约束过滤的候选方案，先对 silhouette、Calinski-Harabasz、反向 Davies-Bouldin、stability ARI 和 balance
↪ 五个指标做 min-max 归一化。随后从 Dirichlet(1,1,1,1,1) 随机生成 {WEIGHT_SENSITIVITY_RUNS} 组非负且和为 1 的权重，
↪ 逐次计算综合分并记录第一名和前三名频率。稳定性分析使用 50 次 80% PD 受试者重采样；SparsePCA 方案在已拟合的 SparsePCA
↪ 表示空间内重聚类以避免重复拟合高耗时表示。",
930 "",
931 """ 7. 第 5b 阶段第一名与前三名频率",
932 f"判定: {stage5_label}。",
933 markdown_table_cn(
934     stage5_rank,
935     [
936         "feature_set",
937         "representation",
938         "method",
939         "top1_frequency",
940         "top3_frequency",
941         "silhouette",
942         "davies_bouldin",
943         "stability_ari_mean",
944         "smaller_cluster_ratio",
945     ],
946     {
947         "feature_set": " 特征方案",
948         "representation": " 表示方式",
949         "method": " 聚类方法",
950         "top1_frequency": " 第一名频率",
951         "top3_frequency": " 前三名频率",
952         "silhouette": " 轮廓系数",
953         "davies_bouldin": "Davies-Bouldin",
954         "stability_ari_mean": " 稳定性 ARI 均值",
955         "smaller_cluster_ratio": " 较小簇比例",
956     },
957     max_rows=20,
958 ),
959 "",
960 """ 8. 第 6b 阶段第一名与前三名频率",
961 f"判定: {stage6_label}。",
962 markdown_table_cn(
963     stage6_rank,
964     [
965         "feature_set",
966         "representation",
967         "method",
968         "top1_frequency",
969         "top3_frequency",
970         "min_cluster_size",
971         "min_cluster_ratio",
972         "silhouette",
973         "davies_bouldin",
974         "stability_ari_mean",
975     ],
976     {
977         "feature_set": " 特征方案",
978         "representation": " 表示方式",
979         "method": " 聚类方法",
980         "top1_frequency": " 第一名频率",
981         "top3_frequency": " 前三名频率",
982         "min_cluster_size": " 最小簇人数",
983         "min_cluster_ratio": " 最小簇比例",
984         "silhouette": " 轮廓系数",
985         "davies_bouldin": "Davies-Bouldin",
986         "stability_ari_mean": " 稳定性 ARI 均值",
987     },
988     max_rows=20,
989 ),
990 "",
991 """ 9. 原第 5/6 阶段主方案稳健性",
992 robustness_sentence("stage5b", stage5_rank),
993 robustness_sentence("stage6b", stage6_rank),
994 "",
995 """ 10. 论文表述建议",
996 "若原方案第一名频率低于 60%，论文中不应继续写作唯一“最佳方案”，应改为“在若干客观指标和权重设定下表现较好的候选方案”或
↪ “多个可接受候选方案”。权重敏感性不是外部临床标签验证，只能说明聚类推荐对主观权重的稳健性。",
997 "",
998 """ 11. 最终建议",
999 f"第 5b 阶段: {stage5_label}，首位候选为 `{stage5_rank.iloc[0]['feature_set']}` /
↪ `{stage5_rank.iloc[0]['representation']}` / `{stage5_rank.iloc[0]['method']}`，第一名频率
↪ `{float(stage5_rank.iloc[0]['top1_frequency']):.1%}`。",

```



```

1000     f"第 6b 阶段: {stage6_label}, 首位候选为 `{stage6_rank.iloc[0]['feature_set']}` /
    ↳ {stage6_rank.iloc[0]['representation']} / {stage6_rank.iloc[0]['method']}`, 第一名频率
    ↳ {float(stage6_rank.iloc[0]['top1_frequency']):.1%}。",
1001     " 所有结论仅为 PD 受试者内部语音特征聚类的稳健性补充, 不写作真实临床亚型或六症状诊断。",
1002 ]
1003 (RESULTS_DIR / "stage5b_stage6b_feature_and_weight_sensitivity_report.md").write_text(
1004     "\n".join(report),
1005     encoding="utf-8",
1006 )
1007
1008
1009 def write_revision_suggestions(stage5_rank: pd.DataFrame, stage6_rank: pd.DataFrame) -> None:
1010     stage5_original = find_original_candidate("stage5b", stage5_rank)
1011     stage6_original = find_original_candidate("stage6b", stage6_rank)
1012
1013     def recommendation(stage_name: str, original: pd.DataFrame, rank: pd.DataFrame) -> list[str]:
1014         if original.empty:
1015             return [
1016                 f"- {stage_name}: 原主方案未匹配到 5b/6b 候选, 建议改为报告多个候选方案。",
1017             ]
1018         freq = float(original.iloc[0]["top1_frequency"])
1019         top_freq = float(rank.iloc[0]["top1_frequency"])
1020         if freq >= 0.60:
1021             decision = " 可保留原主方案, 但需说明其经过无奖励权重敏感性分析。"
1022         elif freq >= 0.30:
1023             decision = " 建议保留为相对推荐方案, 并同步报告前 2-3 个可接受方案。"
1024         else:
1025             decision = " 不建议继续称原方案为最佳, 应改为候选方案或多方案报告。"
1026         return [
1027             f"- {stage_name}: 原主方案第一名频率 {freq:.1%}; 当前最高候选第一名频率 {top_freq:.1%}。{decision}",
1028         ]
1029
1030     lines = [
1031         "## 第 5b/6b 阶段论文修改建议",
1032         "",
1033         "### 1. 是否仍推荐原主方案",
1034         *recommendation(" 第 (4) 问 / 第 5 阶段", stage5_original, stage5_rank),
1035         *recommendation(" 第 (5) 问 / 第 6 阶段", stage6_original, stage6_rank),
1036         "",
1037         "### 2. 是否应弱化“最佳方案”",
1038         "- 若对应阶段最高第一名频率低于 60%, 将“最佳方案”改为“相对推荐候选方案”或“多个可接受候选方案”。",
1039         "- 不把 Top20/Top50 的可解释性写成评分优势, 只能在讨论部分解释其论文可读性。",
1040         "",
1041         "### 3. 需要改弱的句子类型",
1042         "- 将“该方案为最优/最佳聚类方案”改为“该方案在本研究设定的客观指标下表现较好”。",
1043         "- 将“证明存在真实亚型”改为“提示 PD 受试者语音特征中存在可探索的内部异质性”。",
1044         "- 将“六类症状诊断模型”改为“k=6 潜在语音表型聚类模型”。",
1045         "- 避免把语音聚类簇写成已由外部临床标签确认的运动相关亚型或六症状诊断类别。",
1046         "",
1047         "### 4. 新增表格和图",
1048         "- 表: 第 5b/6b 阶段候选方案客观指标表。",
1049         "- 表: 第 5b/6b 阶段权重敏感性第一名和前三名频率表。",
1050         "- 图: `stage5b_weight_sensitivity_top_candidates.png` 和 `stage6b_weight_sensitivity_top_candidates.png`。",
1051         "- 图: `stage5b_feature_set_metric_comparison.png` 和 `stage6b_feature_set_metric_comparison.png`。",
1052         "- 图: `stage5b_stage6b_original_vs_pdonly_comparison.png`。",
1053         "",
1054         "### 5. 放置位置",
1055         "- 正文保留简短方法说明、首位候选频率和最终推荐判断。",
1056         "- 完整 96 个候选方案指标、相关性去冗余保留清单和 10000 组权重敏感性明细建议放附录。",
1057     ]
1058     (RESULTS_DIR / "stage5b_stage6b_paper_revision_suggestions.md").write_text(
1059         "\n".join(lines),
1060         encoding="utf-8",
1061     )
1062
1063
1064 def verify_outputs(stage5_metrics: pd.DataFrame, stage6_metrics: pd.DataFrame) -> None:
1065     for stage, metrics, k in [("Stage 5b", stage5_metrics, 2), ("Stage 6b", stage6_metrics, 6)]:
1066         if metrics.shape[0] != 96:
1067             raise ValueError(f"{stage} expected 96 candidates, found {metrics.shape[0]}")
1068         if set(metrics["n_subjects"].unique()) != {EXPECTED_PD_SUBJECTS}:
1069             raise ValueError(f"{stage} contains a non-PD subject count")
1070         if set(metrics["k"].unique()) != {k}:
1071             raise ValueError(f"{stage} contains wrong k values")
1072         forbidden_text = " ".join(stage5_metrics["candidate_id"].tolist() + stage6_metrics["candidate_id"].tolist()).lower()
1073         for token in ["sex_male", "record_count", "stage5_cluster", "stage6_cluster"]:
1074             if token in forbidden_text:
1075                 raise ValueError(f"Forbidden token appeared in candidate ids: {token}")
1076
1077
1078 def main() -> None:
1079     RESULTS_DIR.mkdir(exist_ok=True)
1080     FIGURES_DIR.mkdir(exist_ok=True)
1081
1082     _, pd_subjects, all_features, top20_features, top50_features = load_inputs()
1083     schemes, audit, retained_detail = build_feature_schemes(
1084         pd_subjects, all_features, top20_features, top50_features
1085     )
1086
1087     write_csv(audit, RESULTS_DIR / "stage5b_pd_only_feature_sets_audit.csv")
1088     write_csv(audit, RESULTS_DIR / "stage6b_pd_only_feature_sets_audit.csv")
1089     write_csv(retained_detail, RESULTS_DIR / "stage5b_pd_only_retained_features.csv")

```

```

1090 write_csv(retained_detail, RESULTS_DIR / "stage6b_pd_only_retained_features.csv")
1091
1092 stage5_metrics = add_normalized_metrics(
1093     run_clustering_stage(pd_subjects, schemes, 2, "stage5b"), 2, "stage5b"
1094 )
1095 stage6_metrics = add_normalized_metrics(
1096     run_clustering_stage(pd_subjects, schemes, 6, "stage6b"), 6, "stage6b"
1097 )
1098 verify_outputs(stage5_metrics, stage6_metrics)
1099
1100 stage5_sensitivity, stage5_rank = weight_sensitivity(stage5_metrics, "stage5b")
1101 stage6_sensitivity, stage6_rank = weight_sensitivity(stage6_metrics, "stage6b")
1102
1103 write_csv(stage5_metrics, RESULTS_DIR / "stage5b_clustering_metrics_no_bonus.csv")
1104 write_csv(stage6_metrics, RESULTS_DIR / "stage6b_clustering_metrics_no_bonus.csv")
1105 write_csv(stage5_sensitivity, RESULTS_DIR / "stage5b_weight_sensitivity.csv")
1106 write_csv(stage6_sensitivity, RESULTS_DIR / "stage6b_weight_sensitivity.csv")
1107 write_csv(stage5_rank, RESULTS_DIR / "stage5b_candidate_rank_frequency.csv")
1108 write_csv(stage6_rank, RESULTS_DIR / "stage6b_candidate_rank_frequency.csv")
1109
1110 plot_weight_sensitivity(
1111     stage5_rank,
1112     "stage5b",
1113     FIGURES_DIR / "stage5b_weight_sensitivity_top_candidates.png",
1114 )
1115 plot_weight_sensitivity(
1116     stage6_rank,
1117     "stage6b",
1118     FIGURES_DIR / "stage6b_weight_sensitivity_top_candidates.png",
1119 )
1120 plot_metric_comparison(
1121     stage5_metrics,
1122     "stage5b",
1123     FIGURES_DIR / "stage5b_feature_set_metric_comparison.png",
1124 )
1125 plot_metric_comparison(
1126     stage6_metrics,
1127     "stage6b",
1128     FIGURES_DIR / "stage6b_feature_set_metric_comparison.png",
1129 )
1130 plot_original_vs_pdonly(
1131     stage5_rank,
1132     stage6_rank,
1133     FIGURES_DIR / "stage5b_stage6b_original_vs_pdonly_comparison.png",
1134 )
1135
1136 write_report(stage5_metrics, stage6_metrics, stage5_rank, stage6_rank, audit)
1137 write_revision_suggestions(stage5_rank, stage6_rank)
1138
1139 print("Stage 5b/6b PD-only sensitivity analysis complete.")
1140 print(f"Stage 5b top candidate: {stage5_rank.iloc[0]['candidate_id']} ({stage5_rank.iloc[0]['top1_frequency']:.1%})")
1141 print(f"Stage 6b top candidate: {stage6_rank.iloc[0]['candidate_id']} ({stage6_rank.iloc[0]['top1_frequency']:.1%})")
1142
1143
1144 if __name__ == "__main__":
1145     main()

```